



SEVN (and SEVNpy) User guide

Version S2.10.0.P0.3:12.06.24

Development line: zealous redgiant

For any question/comment:

giuliano.iorio@unipd.it

giuliano.iorio.astro@gmail.com

sevnpeople@gmail.com

Foreword...

- The most updated version of this document (and of the SEVN code) can be found in the public SEVN repository: <https://gitlab.com/sevncodes/sevn>
- This user-guide evolves together with the code. If you find typos and/or inconsistencies with respect to the the current SEVN code, please let use know.
- Any feedback on how to improve and extend this user guide is greatly appreciated and encouraged

Summary

- **Introduction to SEVN**
 - Single stellar evolution
 - Binary stellar evolution
 - Adaptive timestep
 - Change of track
- **Getting SEVN**
- **Compiling SEVN**
 - Requirements and compatibility
 - SEVN through Docker, Docker from terminal
 - Compilation
- **Running SEVN**
 - How to run
 - Inputs
 - Runtime options
- **SEVN outputs**
 - Evolved and Failed files
 - Output files
 - Logfile
 - Summary of used parameters
 - Launch line
 - Hint and tips for output analysis

Summary

The SEVNpy python module

- [Introduction to SEVNpy](#)
 - [Getting SEVNpy](#)
 - [Requirements](#)
 - [Installation](#)
 - [API documentation](#)
 - [The sevn module](#)
 - [The SEVNmanager class](#)
 - [The Star class and SSE](#)
 - [The io module](#)
 - [The SEVNdata class](#)
 - [Reading log data](#)

Summary

Advanced Guide

- [SEVN as a library](#)
 - [Intro](#)
 - [Compiling - extra options](#)
 - [Installing the library](#)
 - [Using the library](#)
 - [Linking the library](#)
 - [Linking the library with CMake](#)
 - [Linking the library with SEVNENV](#)

Brief introduction to SEVN



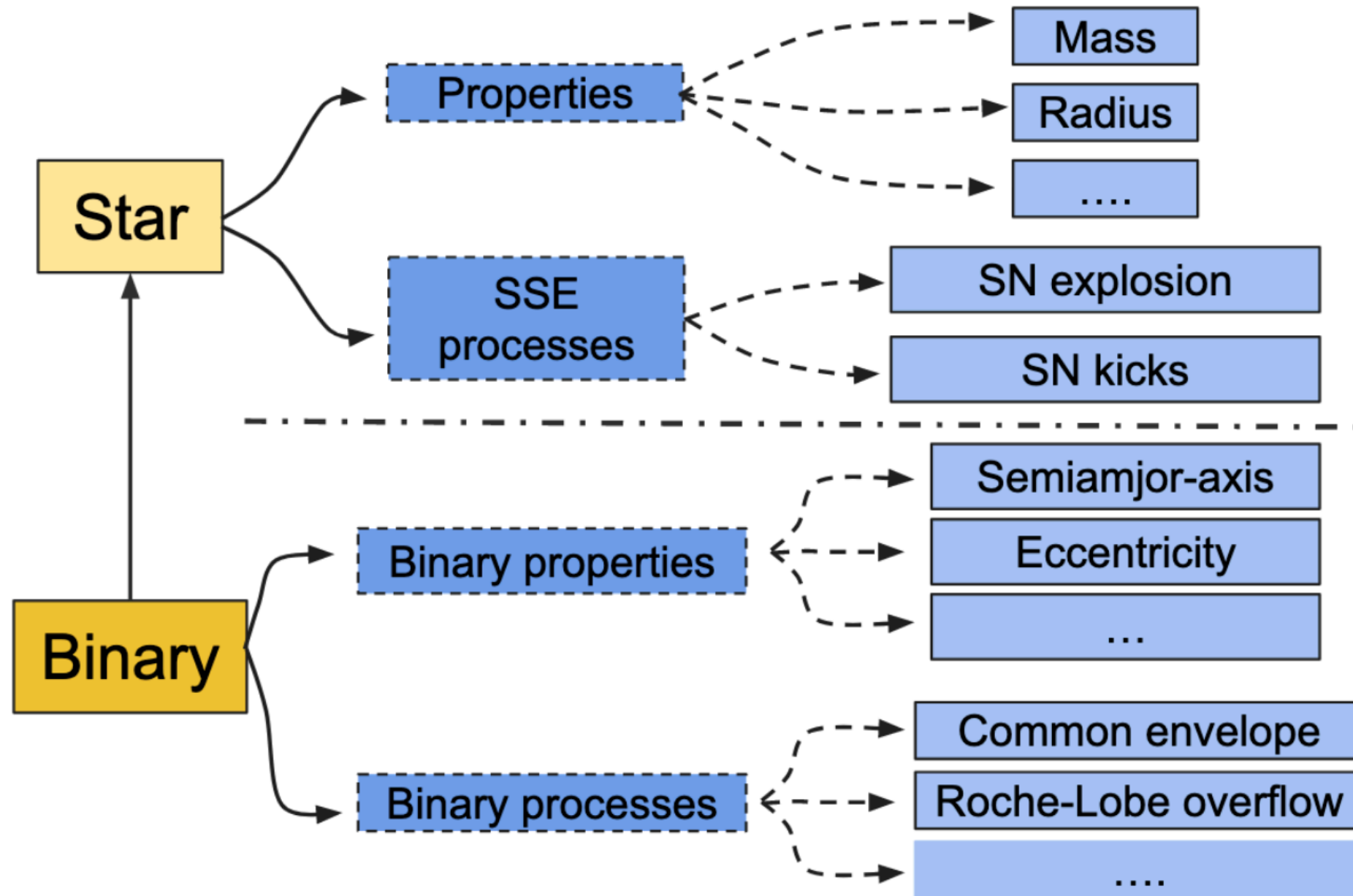
What is SEVN?

Stellar EVolution N-body
is a rapid population synthesis code

- Written completely in C++ (C++14 standard, but compatible with older compilers), parallelised with OpenMP
- Stellar evolution through **interpolation of precomputed stellar tracks**
- Precomputed stellar tracks can be easily added to use the **most updated stellar evolution models**
- Binary processes through analytic and semi-analytic methods
- The evolution is performed with an **adaptive time step** schema giving more time resolution where needed

What is SEVN?

Overview



In SEVN, single stars, binary systems, properties and processes are represented with C++ classes.

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

Basic concepts:

- The SSE of a star is univocally defined by two values:
 M_s , **Z_s** , i.e. the zero-age main sequence mass and its metallicity.

For each couple of (**M_s** , **Z_s**) the SSE will be always the same (for a given set of evolutionary tracks).

Single Stellar Evolution (SSE)

SSE through **interpolation of precomputed stellar tracks**

Basic concepts - stellar phases/types:

- The lifetime of a star is divided into different stellar phases:

Phase Int	Phase str	Physics	Hurley+02 Phase
0	PreMainSequence		
1	MainSequence	Core H-burning start	MS
2	TerminalMainSequence	Creation of a He core	HG/GB
3	ShellHBurning	He core almost formed	GB
4	CoreHeBurning	Core He-burning start	CHEB/HeMS*
5	TerminalCoreHeBurning	Creation of a CO core	AGB/HeHG*/HeGB*
6	ShellHeBurning	CO core almost formed	AGB
7	Remnant	Star is dead	WD/NS/BH

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

Basic concepts - stellar phases/types:

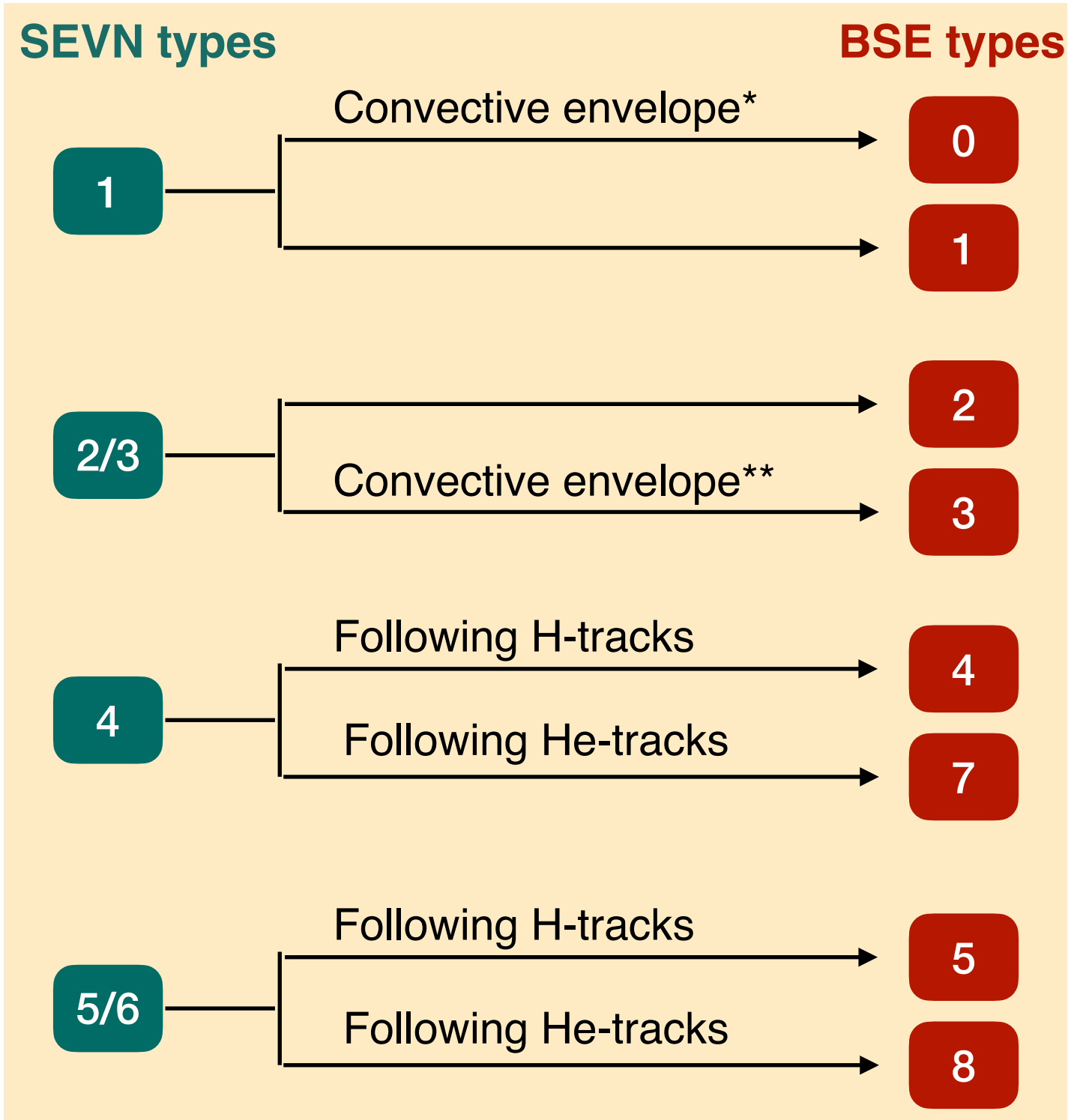
- The “after life” of a star is divided in different remnant types:

RemType Int	RemType str	Physics
0	NotARemnant	Star is not a remnant
1	HeWD	Helium White-Dwarf
2	COWD	Carbon-Oxygen White-Dwarf
3	ONeWD	Oxygen-Neon White-Dwarf
4	NS_ECSN	Neutron Star born after an Electron capture Supernova
5	NS_CCSN	Neutron Star born after a Core Collapse Supernova
6	BH	Black Hole
-1	Empty	Massless remnant

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

Basic concepts - stellar phases/types:



Used to Hurley+02 types?

***Convective envelope**: if the table with fraction of convective envelope mass is available (Q_{sup}), envelope is convective if $Q_{\text{sup}} > 0.8$, otherwise we use the condition $M_{\text{zams}} < 0.7$ (see [Hurley+02](#)).

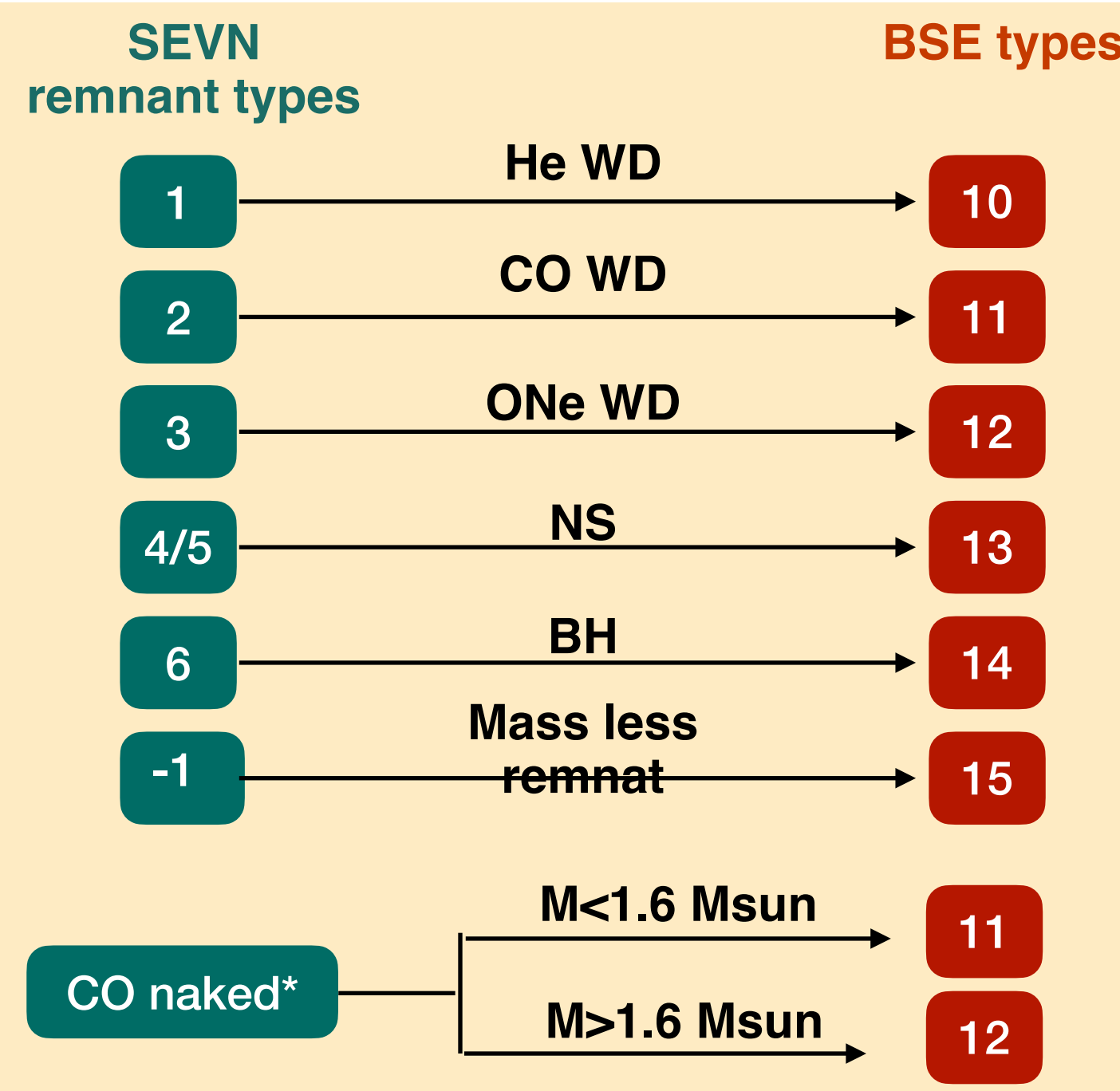
****Convective envelope**: if mass of the convective envelope is larger than 33% of the mass of the envelope, the envelope is considered convective (see [Hurley+00](#)). If the table with the fraction of convective envelope mass is not available, Q_{sup} is estimated following [Hurley+00](#) (Sec. 7.2).

****H/He tracks**: stars following H-tracks are star with an H-envelope and a He/CO core; stars following He-tracks have lost they H-envelope. See [here](#) for further details

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

Basic concepts - stellar phases/types:



Used to BSE/MOBSE types?

***CO naked**: during the BSE evolution is possible to turn a star into a bare CO core. These stars have not a special type, but they have a special treatment inside SEVN. When conversion to BSE types is needed they are considered as WD (the type of WD depend on the mass).

Single Stellar Evolution (SSE)

SSE through **interpolation of precomputed stellar tracks**

Basic concepts:

- SEVN needs precomputed stellar tracks inserted in lookup tables.

Each stellar track refers to a star with zero-age main sequence mass M_{zams} and metallicity Z .

The lookup tables contain the stellar properties (e.g. Mass, Radius, Luminosity) at given times.

There are two kind of tables:

- **H-rich tables:** tracks containing the stellar evolution. of “standard” stars with Hydrogen envelopes.
- **pure-He tables:** tracks containing the stellar evolution of pure-HE stars without Hydrogen envelopes.

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

Basic concepts: stellar tables overview

SEVN tables			
Table	Units	Type	Interpolation
Time	Myr	M	R
Phase†	Myr†	M	R
Mass	$M_{\odot}$	M	LIN
Luminosity	$L_{\odot}$	M	LOG
Radius	$R_{\odot}$	M	LOG
He-core mass	$M_{\odot}$	M	LIN
CO-core mass	$M_{\odot}$	M	LIN
He-core Radius	$R_{\odot}$	O	LIN
CO-core Radius	$R_{\odot}$	O	LIN
Stellar inertia	$M_{\odot}R_{\odot}^2$	O	LOG
H envelope binding energy	$M_{\odot}^2R_{\odot}^{-1}G^{-1}$	O	LOG
Convective envelope			
mass	normalised to star mass	O	LIN
depth	normalised to star radius	O	LIN
turnover time	yr	O	LIN

Type: M tables are mandatory in SEVN, O tables are optional

Interpolation: type of interpolations used by default in SEVN: rational (R), linear (LIN), log (LOG), see [here](#) for additional information

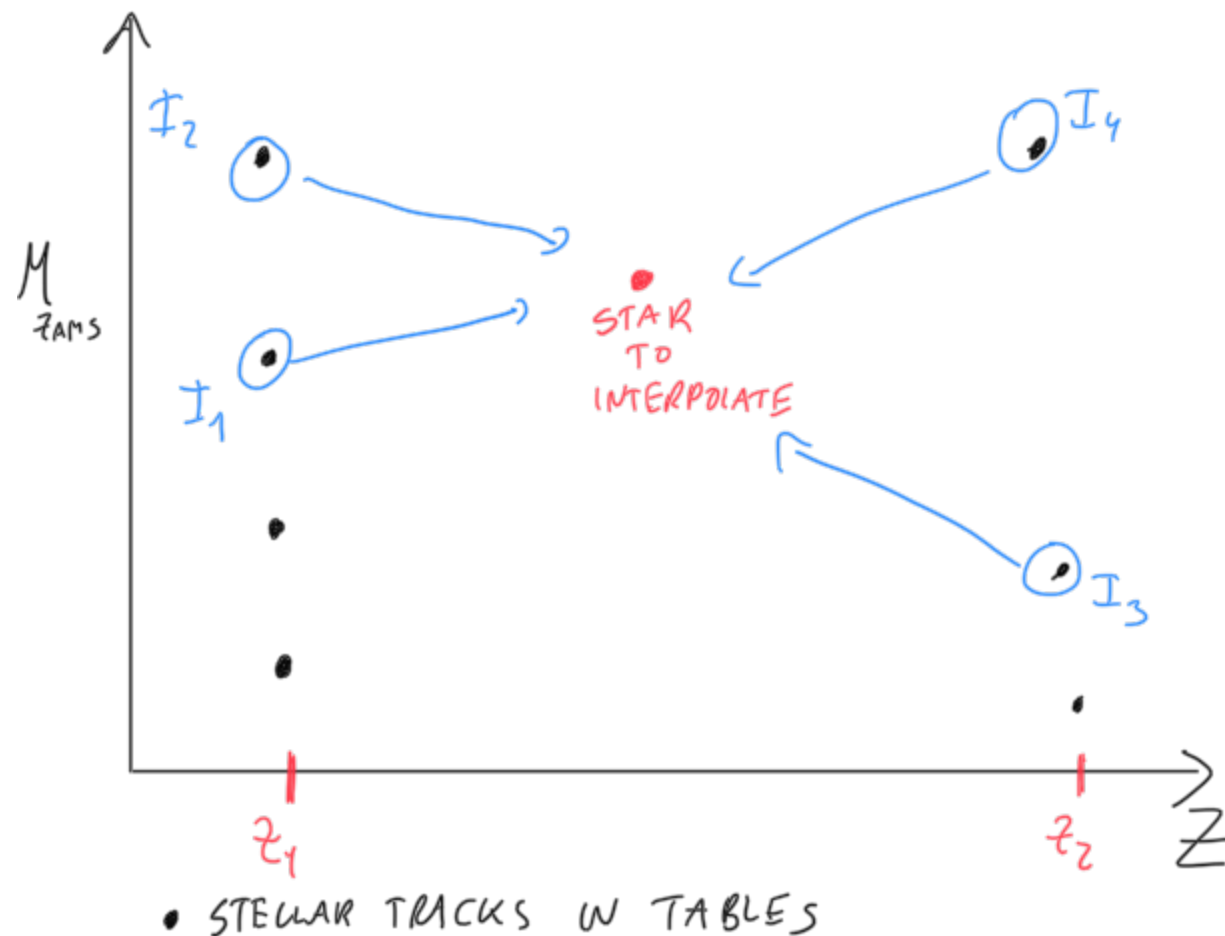
Phase: The phase table reports the starting time of each [sevn phase](#)

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

Basic concepts:

- For each couple of (**Ms**, **Zs**) SEVN assigns **four interpolating tracks** taken from the tables: I_1 ($M_1 \leq M_s, Z_1 \leq Z_s$), I_2 ($M_2 \geq M_s, Z_1 \leq Z_s$), I_3 ($M_3 \leq M_s, Z_2 \geq Z_s$), I_4 ($M_4 \geq M_s, Z_2 \geq Z_s$). Where the value of M^* and Z^* are the closest upper and lower (or equal) values wrt (**Ms**, **Zs**) found in the tables.



Single Stellar Evolution (SSE)

SSE through **interpolation of precomputed stellar tracks**

Basic concepts:

- Properties are obtained from the interpolating tracks at the **same life percentage** in a given stellar phase:

$$\text{plife} = \frac{\tilde{t} - t_{0,\text{phase}}}{\Delta t_{\text{phase}}}$$

Using plife instead of time we are guaranteed that all the interpolating tracks are in the same stellar phase of the interpolated star (i.e. they all have or not have a HE/CO core).

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

SSE Evolution - Some definitions:

Interpolating tracks: evolutionary tracks at given M and Z used to interpolate the properties and their evolution of a star. Their properties are stored in lookup tables. They are evolved in time using linear interpolation.

Fake star: it is evolved weighting the values of the four interpolating tracks.

Real star: it is evolved using the relative difference of the fake star. This is star that we actually got as output.

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

SSE Evolution - Step-by-step:

Estimate plife of the Real star

$$\text{plife} = \frac{\tilde{t}_s - t_{0,\text{phase},s}}{\Delta t_{\text{phase},s}}$$

Evolve the interpolating tracks

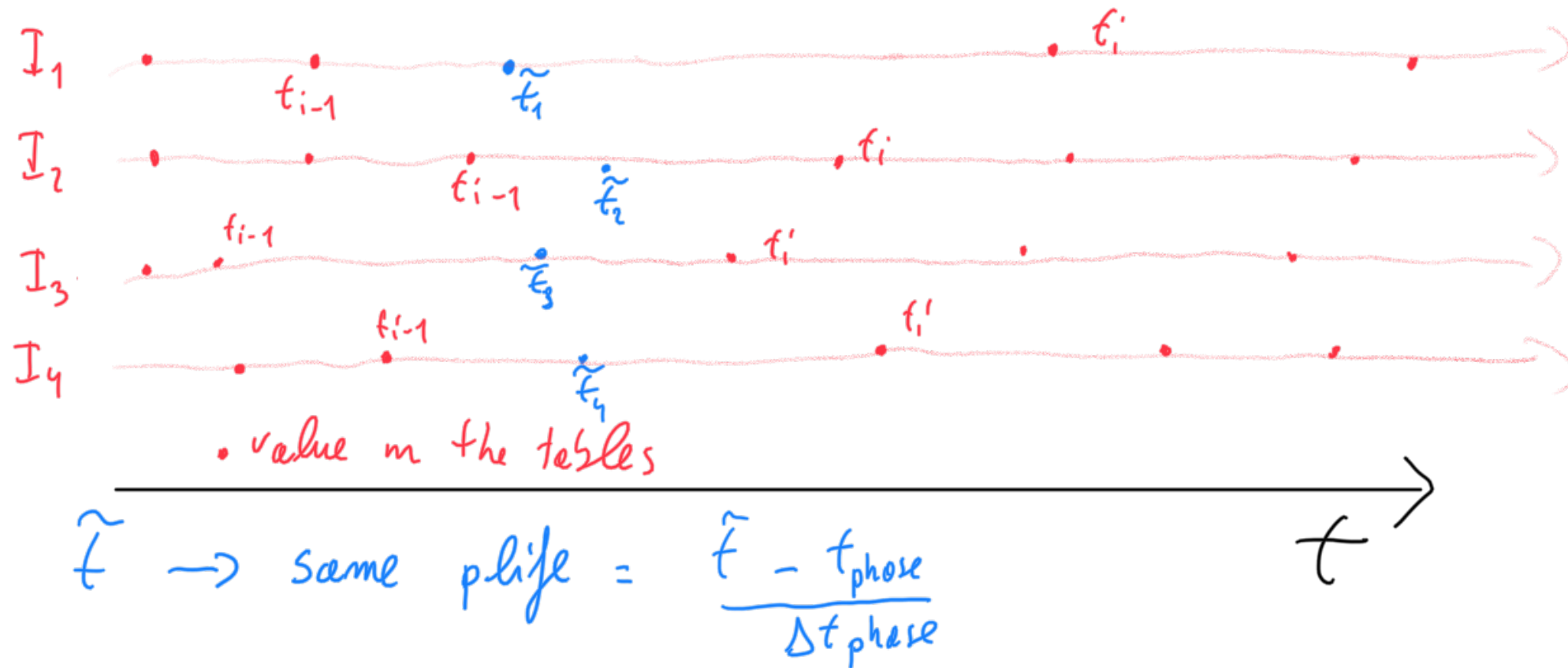
Evolve the Fake star

Evolve the Real star

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

SSE Evolution - Step-by-step- Evolve the interpolating tracks:



$$P_{\tilde{t}} = \frac{P_{t_i} - P_{t_{i-1}}}{t_{i-1} - t_i} (\tilde{t} - t_{i-1})$$

linear interpolation

Single Stellar Evolution (SSE)

SSE through **interpolation of precomputed stellar tracks**

SSE Evolution - Step-by-step- Evolve the Fake star:

Weighted combination from the interpolate tracks at the same plife.

$$Y_1 = \beta_1 P_{I_1}(\tilde{t}_1) + \beta_2 P_{I_2}(\tilde{t}_2)$$

$$Y_2 = \beta_3 P_{I_3}(\tilde{t}_3) + \beta_4 P_{I_4}(\tilde{t}_4)$$

$$P_{\text{fake}}(\tilde{t}_s) = \gamma_1 Y_1 + \gamma_2 Y_2$$

$$\gamma_1 = \frac{Z_2 - Z_s}{Z_2 - Z_1}$$

$$\gamma_2 = \frac{Z_s - Z_1}{Z_2 - Z_1}$$

See next slide for the beta weights

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

SSE Evolution - Step-by-step- Evolve the Fake star:

$$Y_1 = \beta_1 P_{I_1}(\tilde{t}_1) + \beta_2 P_{I_2}(\tilde{t}_2) \qquad Y_2 = \beta_3 P_{I_3}(\tilde{t}_3) + \beta_4 P_{I_4}(\tilde{t}_4)$$

Linear weights

$$\beta_{1/3}^* = \frac{M_{\text{zams},2/4} - M_s}{M_{\text{zams},2/4} - M_{\text{zams},1/3}} \qquad \beta_{2/4}^* = \frac{M_s - M_{\text{zams},1/3}}{M_{\text{zams},2/4} - M_{\text{zams},1/3}}$$

Mostly used for quantities such as the stellar masses (Mass, MHE, MCO)

Log weights

$$\beta_{1/3}^* = \frac{\log M_{\text{zams},2/4} - \log M_s}{\log M_{\text{zams},2/4} - \log M_{\text{zams},1/3}} \qquad \beta_{2/4}^* = \frac{\log M_s - \log M_{\text{zams},1/3}}{\log M_{\text{zams},2/4} - \log M_{\text{zams},1/3}}$$

Mostly used for quantities that show a power-like proportionality with the Mass, e.g. Radius, Luminosity

Rational weights

$$\beta_{1/3}^* = \frac{M_{\text{zams},1/3}}{M_s} \frac{M_{\text{zams},2/4} - M_s}{M_{\text{zams},2/4} - M_{\text{zams},1/3}} \qquad \beta_{2/4}^* = \frac{M_{\text{zams},2/4}}{M_s} \frac{M_s - M_{\text{zams},1/3}}{M_{\text{zams},2/4} - M_{\text{zams},1/3}}$$

Best weights to set the times of the stellar phases

The weights type can be set at runtime ([see here](#))

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

SSE Evolution - Step-by-step- Evolve the Fake star:

$$Y_1 = \beta_1 P_{I_1}(\tilde{t}_1) + \beta_2 P_{I_2}(\tilde{t}_2) \qquad Y_2 = \beta_3 P_{I_3}(\tilde{t}_3) + \beta_4 P_{I_4}(\tilde{t}_4)$$

Linear weights

$$\beta_{1/3}^* = \frac{M_{\text{zams},2/4} - M_s}{M_{\text{zams},2/4} - M_{\text{zams},1/3}} \qquad \beta_{2/4}^* = \frac{M_s - M_{\text{zams},1/3}}{M_{\text{zams},2/4} - M_{\text{zams},1/3}}$$

Mostly used for quantities such as the stellar masses (Mass, MHE, MCO)

Log weights

$$\beta_{1/3}^* = \frac{\log M_{\text{zams},2/4} - \log M_s}{\log M_{\text{zams},2/4} - \log M_{\text{zams},1/3}} \qquad \beta_{2/4}^* = \frac{\log M_s - \log M_{\text{zams},1/3}}{\log M_{\text{zams},2/4} - \log M_{\text{zams},1/3}}$$

Mostly used for quantities that show a power-like proportionality with the Mass, e.g. Radius, Luminosity

Rational weights

$$\beta_{1/3}^* = \frac{M_{\text{zams},1/3}}{M_s} \frac{M_{\text{zams},2/4} - M_s}{M_{\text{zams},2/4} - M_{\text{zams},1/3}} \qquad \beta_{2/4}^* = \frac{M_{\text{zams},2/4}}{M_s} \frac{M_s - M_{\text{zams},1/3}}{M_{\text{zams},2/4} - M_{\text{zams},1/3}}$$

Best weights to set the times of the stellar phases

The weights type can be set at runtime ([see here](#))

Single Stellar Evolution (SSE)

SSE through interpolation of precomputed stellar tracks

SSE Evolution - Step-by-step- Evolve the Real star:

Evolve following the relative difference of the Fake star

$$P_{\text{real}}(\tilde{t}_s) = P_{\text{real},0} + \frac{P_{\text{fake}}(\tilde{t}_s) - P_{\text{fake},0}}{P_{\text{fake},0}} P_{\text{real},0}$$

Why not using directly the fake star?



The Fake and the Real stars are equivalent in the case of the SSE only. When BSE is included, a star may need to change track due to the mass-loss/accretion. The match of the current stellar mass is not always “perfect”, therefore the introduction of the Real/Fake stars method allows to follow the evolution the stellar properties smoothly.

Binary stellar evolution (BSE)

BSE through **analytics** and **semi-analytics** methods

- Wind Mass Transfer
- Roche-Lobe Overflow (stable/unstable)
- Orbital circularisation
- Common Envelope
- Stellar Merger/Collision
- Stellar tides
- Orbital decay due to Gravitational Wave emission
- SN explosion
- Hardening



- Implemented as analytic/semi-analytic simplification of complex physical processes
- **Choice of different parameters can completely change the fate of a binary system** (e.g. survive or not after a Common Envelope)

Change of track

During the BSE, a star may accrete or lose mass. When this happens SEVN looks for a new stellar track (i.e. a star with a new Mzams) that is more consistent with the current stellar properties, i.e. having a given mass at a given [plife](#).

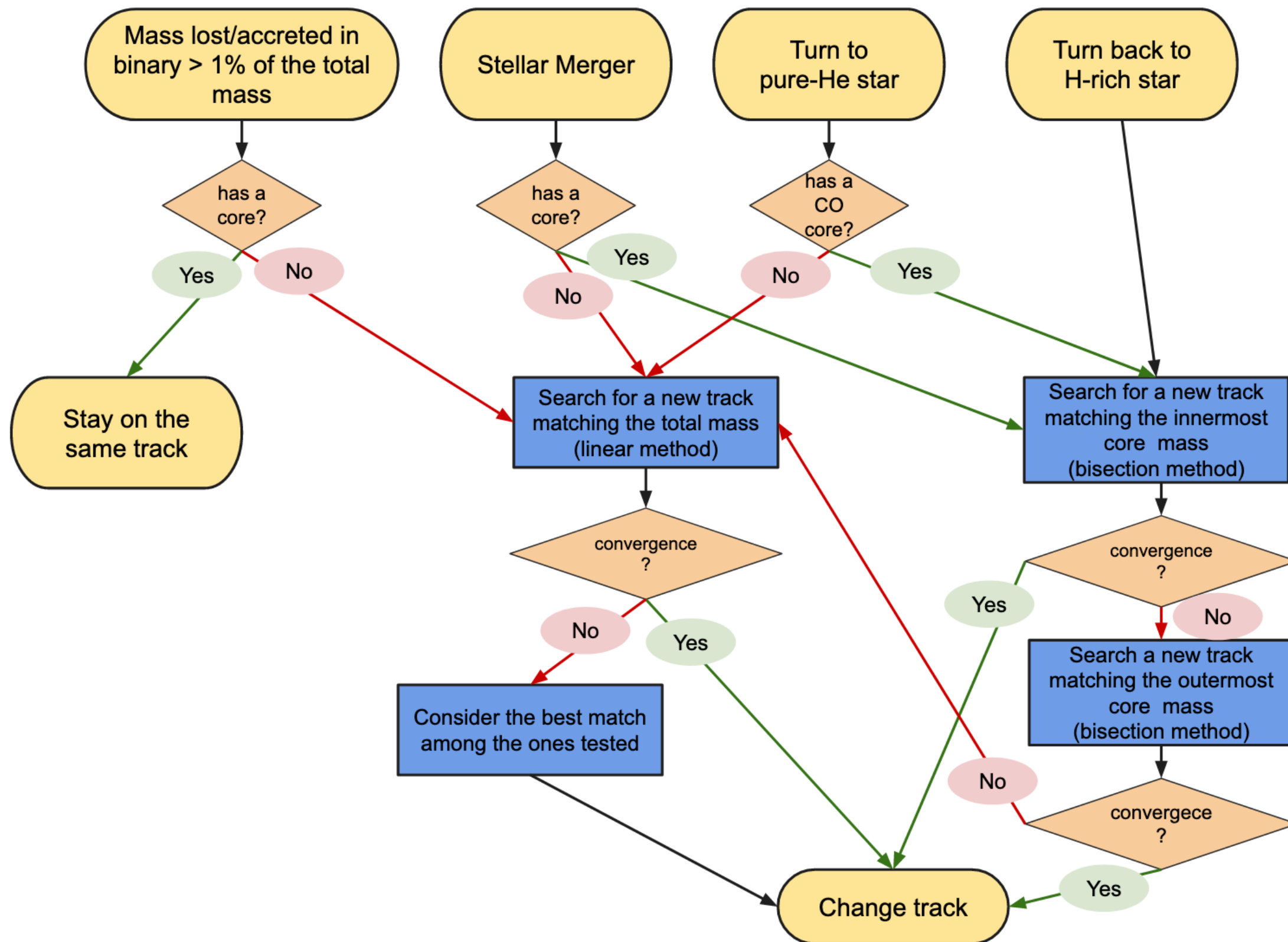
Change of track criteria:

- **During the MS, the star always jumps to a new track** if it accretes mass or lose mass (not considering stellar winds) more than a given threshold (see [here](#)).
- **Outside the MS, when a core is present, a star jumps only if it is forced by some process** (e.g. after a merger).
- **If a star loses its envelope it always jumps** to a new HE-track.

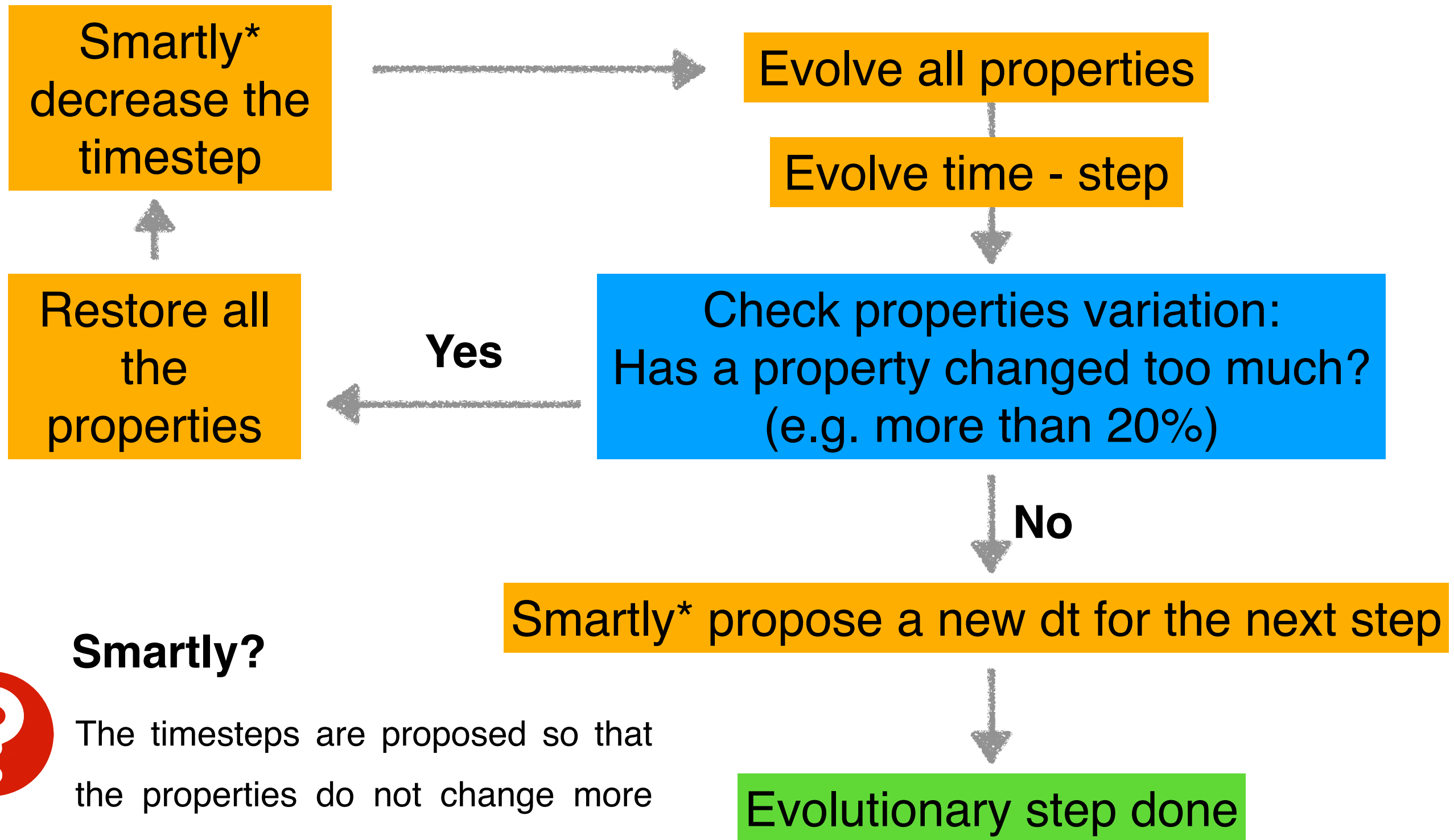
Change of track strategy:

- **A MS star jumps to a new track trying to match the total mass** at the same [plife](#).
- **Outside the MS, a star jumps to a new track trying to match the total mass or the core mass (higher priority)** at the same [plife](#).

Change of track - overview



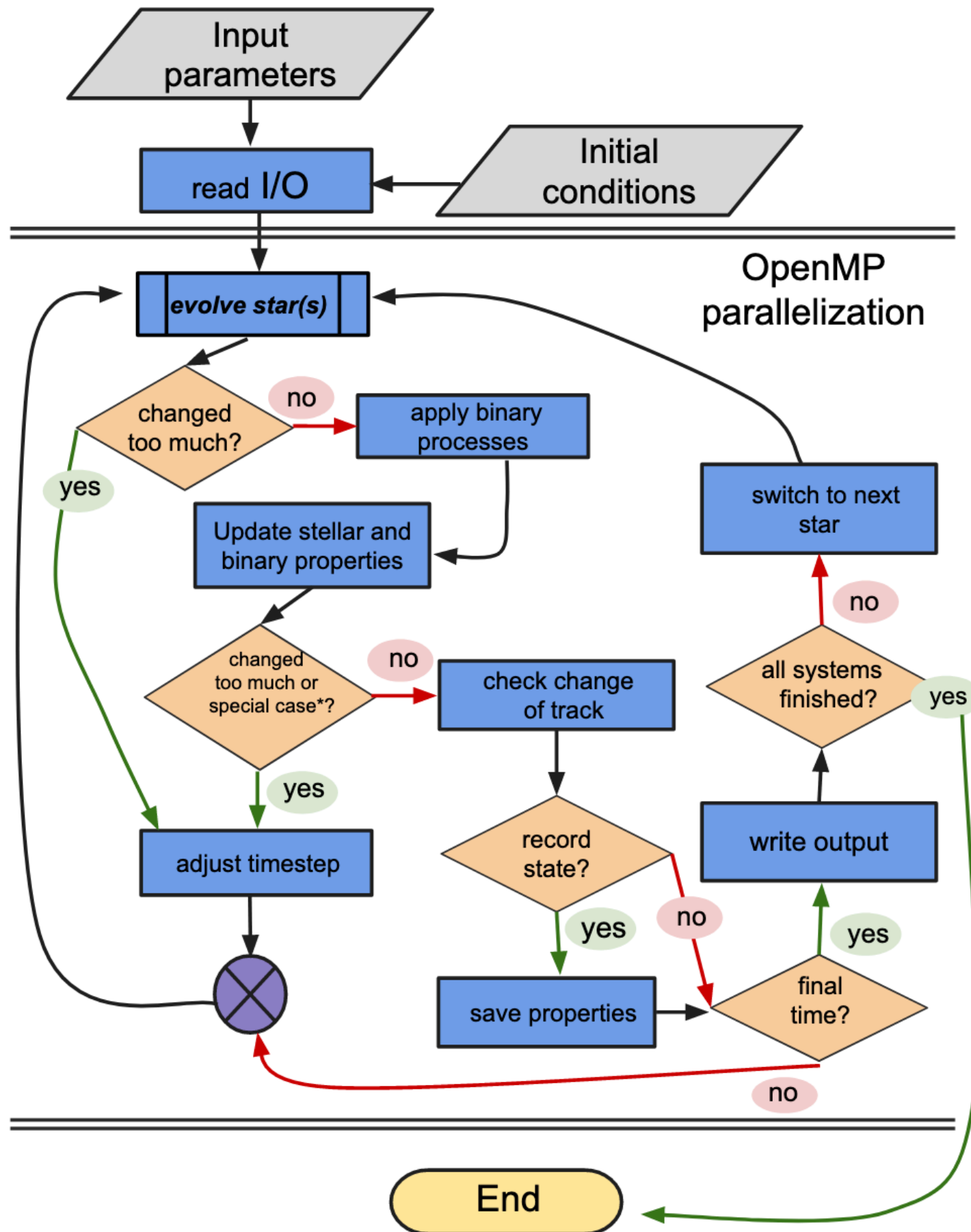
The adaptive time-step schema



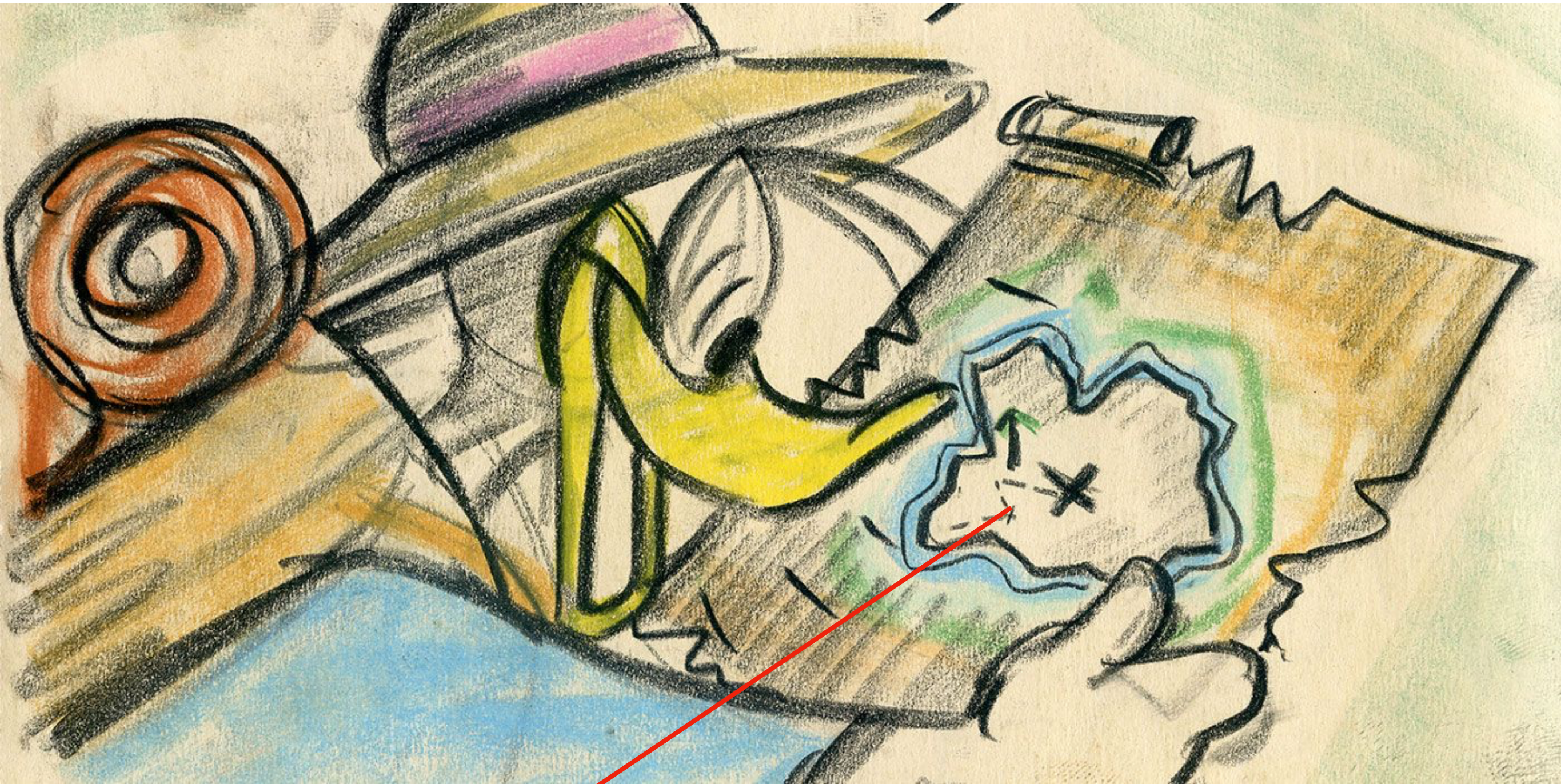
Smartly?

The timesteps are proposed so that the properties do not change more than a given limit (e.g. 5%) considering the last step differences and assuming a linear evolution.

Overall evolution algorithm



Getting SEVN

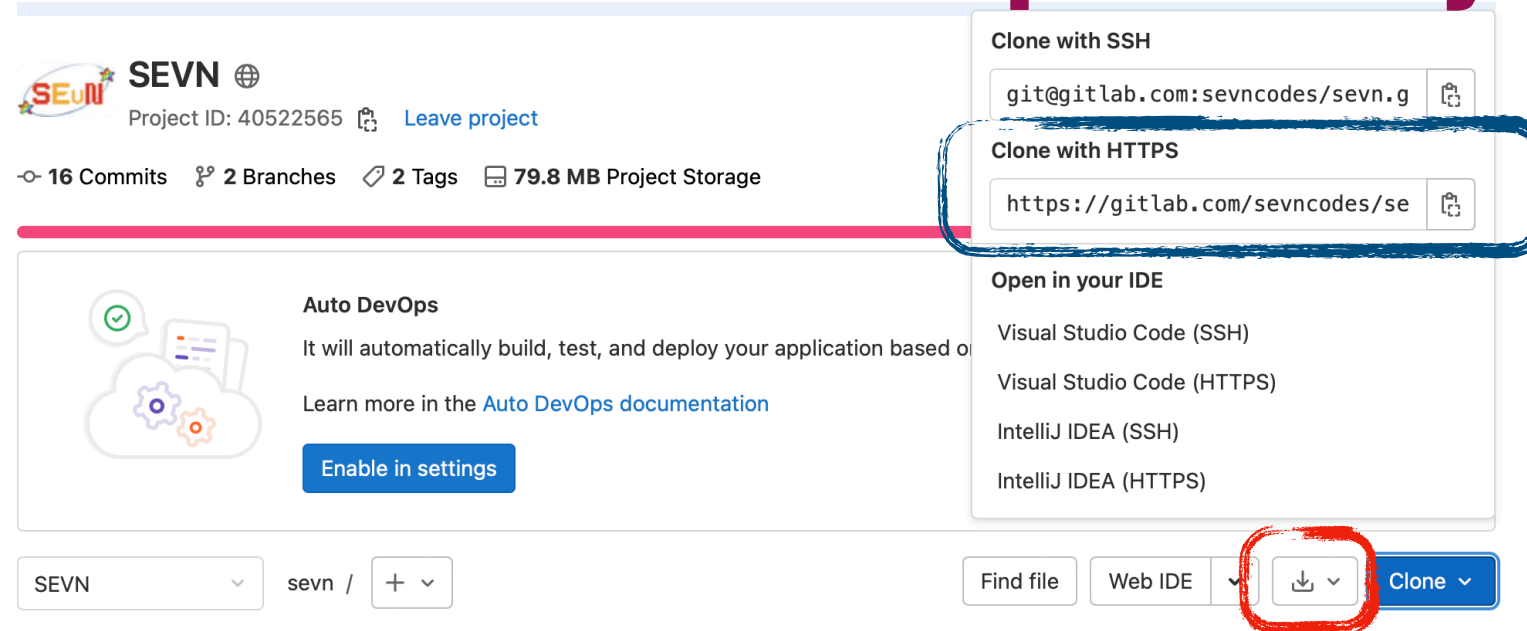


Interested to a shortcut? Give a look to the Docker alternative [here](#)

SEVN repository

SEVN is in public a Gitlab repository:
<https://gitlab.com/sevncodes/sevn>

Getting SEVN from the repository



Two methods to get SEVN from the repository:

1- **Get the source file**: use the red highlighted icons to download the source in a given archive format or use this [link](#).

2- **Clone with git (git required, recommended)**: if you have [git](#), it is possible to clone locally the repository using the blue highlighted link (NOT use SSH), i.e.

```
git clone https://gitlab.com/sevn codes/sevn.git
```

The local repository can be updated with:

```
git pull
```

SEVN structures

The SEVN folder contains many subfolders and files

Code folders:

- include*: contains the basic code headers to use [SEVN as a library](#) (.h)
- src*: contains the code headers (.h) and src file (.cpp)
- extern/catch*: contain the src code for unit test
- auxiliary_data*: data/tables used in the code (not stellar tables)
- tables*: contains the stellar tables used in SEVN to evolve stars
- test*: contains the src file to perform code testing
- main*: contains the main src file to perform BSE and SSE evolution

[Compilation](#):

- CMakeList.txt*: main instruction for Cmake
- cmake*: contains additional cmake modules used in Cmake
- compile.sh***: user-friendly script to compile the code

Utilities:

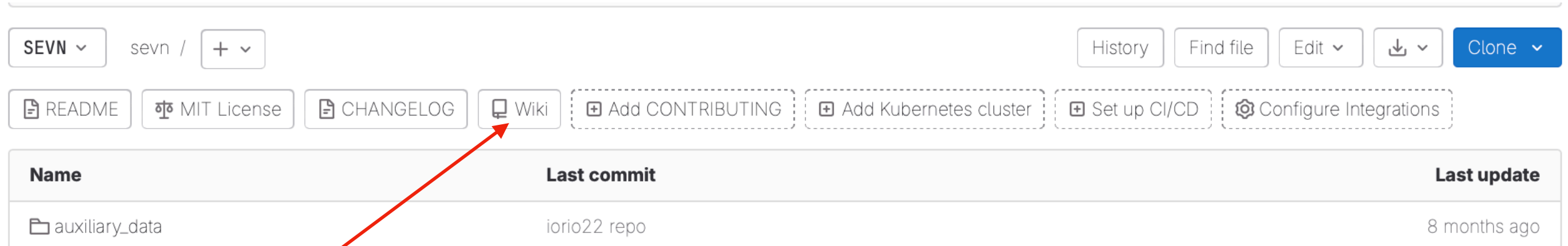
- run_scripts***: contains user-friendly script to facilitate the SEVN run
- resources*: useful resources for the SEVN users (documentation, tutorial)
- pyscript*: contains a collection of useful python scripts
- doc*: folder containing the documentation generated with doxygen
- Docker*: folder containing the [Dockerfile used to produce docker images](#)
- [SEVNpy](#): folder contains the SEVN companion Python module ([see here](#))

A standard SEVN user only needs the **compile.sh** and the **run scripts** (maybe the python scripts) or consider to use directly the **SEVNpy python module**

SEVN documentation and issue

The git repository contains also:

Official Wiki/Documentation

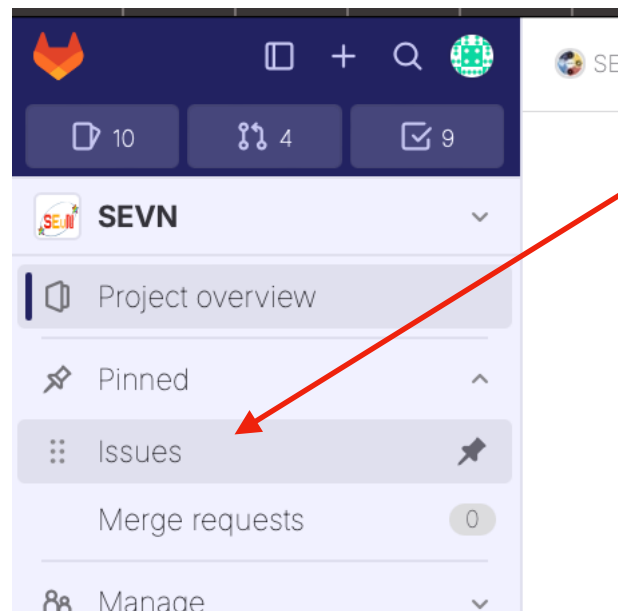


The wiki

(<https://gitlab.com/sevncodes/sevn/-/wikis/home>)

contains some additional documentation about the code.

Issues



The issues page

(<https://gitlab.com/sevncodes/sevn/-/issues>) lists all the known issues/bugs of the code. If you find something “anomalous” while running SEVN, please open an issue here.

SEVN version

A SEVN version is identified by the following variable

In the Cmake:

- **SEVN DEVELOPMENT LINE:** (e.g. zelaous_redgiant) It represent the label identifying a given development line. When the development introduces significant differences with respect to current version a new development line is created. If a new development line is promoted to the stable version, it is not merged, but rather the old version is moved to a new branch and the new version becomes the stable one.
- **SEVN VERSION:** In the form MAJOR.MINOR.PATCH, the PATCH is updated everytime we perform a bug fixes or a minor modification, the MINOR is updated when a substantial modification (both about the structure of about the code is introduced) is introduced. The MAJOR is update only when the code is significantly refactored or if its input/output is modified so much that the old inputs could not work and the analysis scripts could not be able to analyse the SEVN outputs. **The changelog file** stores the main changes correlated to the VERSION updates



These information can be found in the main Cmake file

SEVN version

A SEVN version is identified by the following variable
Related to Git:

- **SEVN BRANCH:** The Gitlab branch of the current SEVN version
 - **GIT_COUNTER:** Sequential number identifying for the last local commit.
 - **GIT_SHA:** Has number of the last local commit
 - **GIT_SHATIMESTAMP:** Timestamp of the last local commit
 - **GIT_COUNTER_REMOTE:** Same as GIT_COUNTER but for the last commit pushed to the remote
 - **GIT_SHA_REMOTE:** Same as GIT_SHA but the for last commit push to the remote
 - **GIT_SHATIMESTAMP_REMOTE,** but for last commit push to the remote
- **The content of the variables are stored in include/sevn.h**
 - **If the remote and the local GIT version variables are not aligned, it means that the local version contains modification with respect to the original remote version**
 - **This quantities are automatically updates from git, they are not available without git**



Compiling SEVN



Requirements and compatibility

Requirements:

- **Cmake** (V>2.8, recommended V>3.2): SEVN uses Cmake for compilation.
- **C++ compiler:**
 - ✓ - **GNU C++:** SEVN has been tested with GNU compilers with version > 4, a version > 7 is recommended.
 - ✓ - **Intel:** SEVN is compatible with the C++ compiler from the [Intel one API](#), however it has not been extensively tested.
 - ✓ - **Clang:** SEVN can be compiled with clang. However, notice that the default clang compiler in macOS does not support openMP. In this case, the compiled SEVN executables can be run just in serial mode.

Requirements and compatibility

Compatibility:



SEVN has been extensively tested on Linux machines.

The Cmake and C++ compiler can be get with the apt package manager:

- **Cmake:** *apt-get install cmake*
- **GNU C++:** *apt-get install g++*

Requirements and compatibility

Compatibility:



- SEVN has been extensively tested on macOS >10.14.
- SEVN is compatible with the new M1 and M2 processors



The default macOS C++ compiler is clang, it does not support openMP.

A guide to satisfy SEVN requirements:

- **Command Line Tools:** this is required to enable all the other command, open a terminal and type *xcode-select --install*
- **Homebrew:** homebrew is a package manager for macOS and facilitate the installation of Cmake and the C++ compiler. To install homebrew follow the instruction at brew.sh
- **Cmake:** *brew install cmake* ([brew link](#))

Requirements and compatibility

Compatibility:



- SEVN2 has been extensively tested on macOS >10.14.
- SEVN2 should be compatible with macOS >10.7.
- It has never been tested on new macOS with the new M1 processors.

 The default C++ compiler is Clang that is not [compatible](#) with SEVN2

A guide to satisfy SEVN2 requirements:

Notice, after the installation of the homebrew gcc, the default gcc and g++ compiler (the one in /usr/local/bin) will be still the clang ones, therefore the SEVN [compilation](#) will fail. In order to use the right version you can:

- **overwrite** the gcc and g++ in the /usr/local/bin
- **execute** the command `export CXX=g++-<version>` and `export CC=gcc-<version>`, where version is the c++ version installed with homebrew. Only the major version has to be considered, i.e. if the gcc version is 10.5.2, the command to run is `CXX=g++-10`. Important: these commands have to be run each time SEVN has to be compiled

In order to find the installed version and to find the executable to overwrite the standard one in /usr/local/bin, notice that the homebrew gcc can be found in **/usr/local/Cellar/gcc**

Requirements and compatibility

Compatibility:



If you want to fully exploit SEVN running it in parallel, we suggest to install the GNU C++ compiler through homebrew

- **GNU C++:** *brew install gcc* ([brew link](#))

Notice, after the installation of the homebrew gcc, the default gcc and g++ compiler (the one in /usr/local/bin) will be still the clang ones.

Before to run cmake or the compilation script:

- **execute** the command *export CXX=g++-<version>* and *export CC=gcc-<version>*, where version is the C++ version installed with homebrew. Only the major version has to be considered, i.e. if the gcc version is 12.2.0, the command to run is *CXX=g++-12*. Important: these commands have to be run each time SEVN has to be compiled (unless you set the GNU compiler as the default compiler)

In order to find the installed version notice that the homebrew gcc can be found in **/usr/local/Cellar/gcc** for Macs with the intel processor or in **/opt/homebrew/Cellar** for newer Macs with the M1 and M2 ARM processor.

Requirements and compatibility

Compatibility:



We get you covered thank to Docker!

Give a look at this [slide](#)

Anyway, there is a way to install a Linux terminal in Windows:
give a look to <https://allthings.how/how-to-use-linux-terminal-in-windows-10/>

Requirements and compatibility

Alternative solution:



Sometime is not possible to updated packages or install them **directly** (e.g. in hpc clusters). A possible solution is to exploit the CONDA package manager.

The easiest way to get CONDA is to install [ANACONDA](#).

Once Conda is installed:

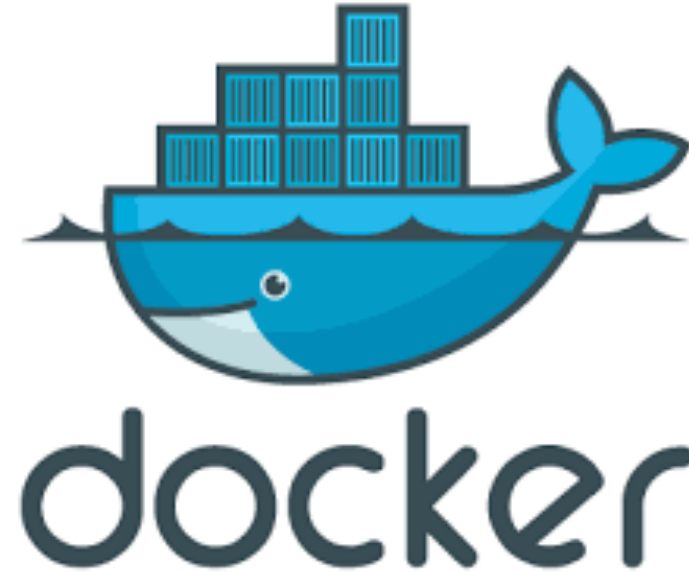
- Download the environment file:
 - [conda_sevn_env.txt](#) (basic environment to install SEVN)
 - [conda_sevn_python_env_V2.txt](#) (basic + python installation for [SEVNpy](#))

Notice: these environments are only for linux systems

- Creat the conda environment with
`conda create --name <env_name> --file conda_sevn_env.txt` or
`conda create --name <env_name> --file conda_sevn_env_V2.txt`
- Activate the conda environment with
`conda activate <env_name>`
- Compile and use SEVN as described in this tutorial

Requirements and compatibility

Alternative solution:



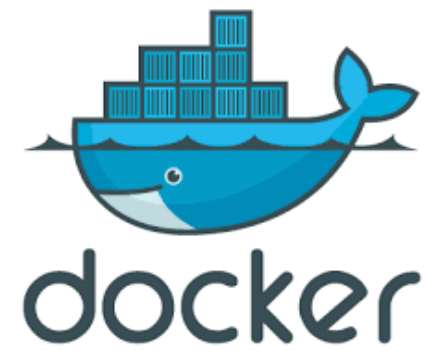
Docker allows to “ship” container that includes everything needed to run an app. If you want to know more, give a look to their [website](#) and to thousands of resources online.

Getting SEVN through docker is easy and the only thing you need to do is to install the Docker desktop app. The beautiful part is that once you get the container all the SEVN requirements are already included, SEVN will just work! See the next slides for a complete guide.



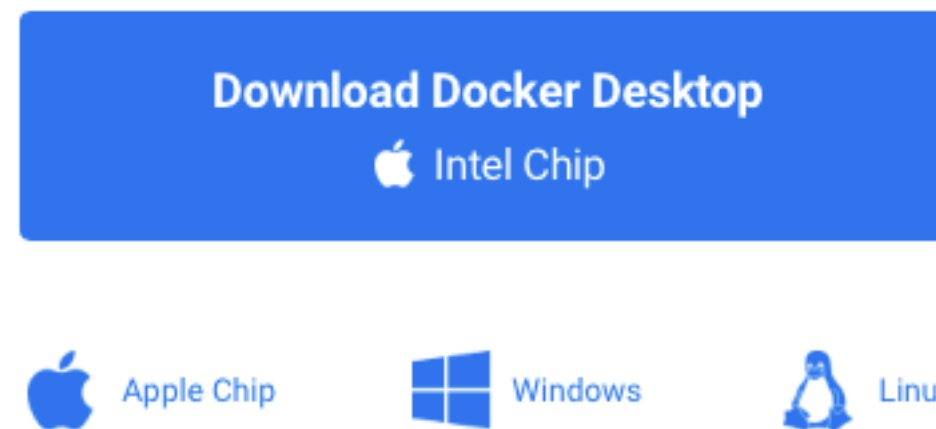
The Docker solution is warmly suggested for **Windows user, but can be exploited also by Linux and Mac users.**

Using docker



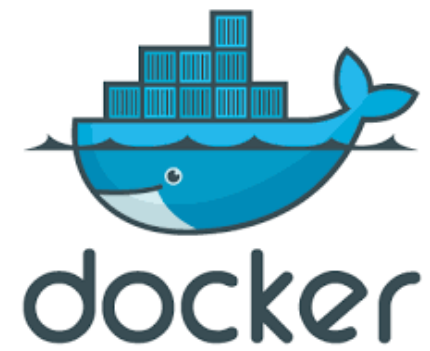
1- Step: Getting docker

- Download the Docker desktop-app for your OS from:
<https://www.docker.com/products/docker-desktop>



- Install it:
guide for Mac:
<https://docs.docker.com/desktop/install/mac-install/>
guide for Linux:
<https://docs.docker.com/desktop/install/linux-install/>
guide for Windows:
<https://docs.docker.com/desktop/install/windows-install/>

Using docker

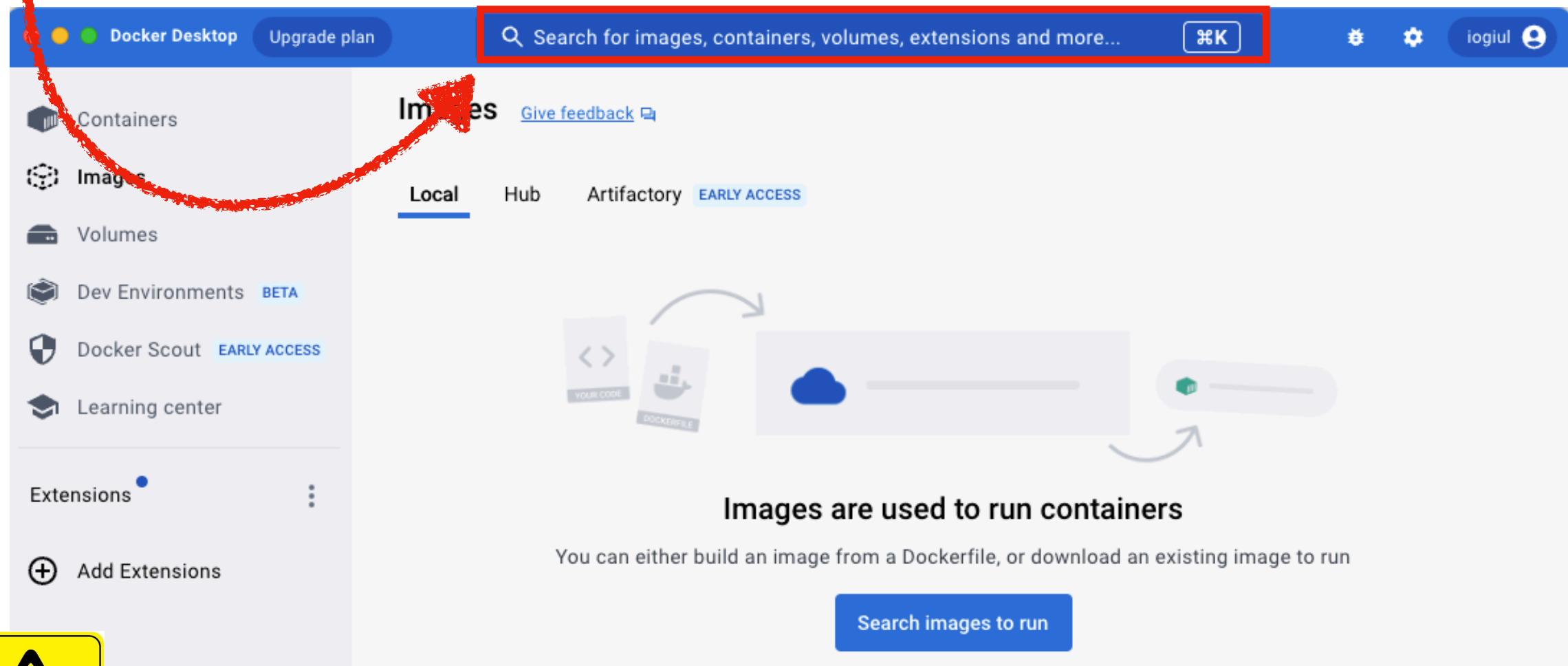


2nd- Step: Getting the SEVN container

- Open the just installed docker app
- Use the built-in search engine to look for the SEVN containers:

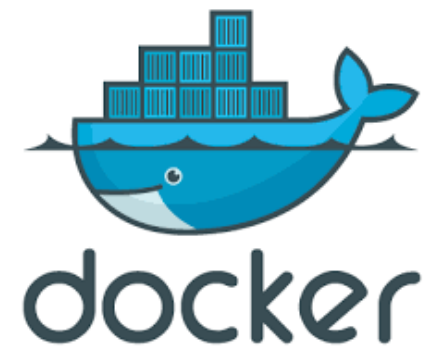
sevnpydocker: contains SEVN plus Python (~2 Gb)

sevndocker: most basic SEVN container (~1 Gb)



The images store 1-2 Gb of contents, it can take some time to download them, be patient!

Using docker

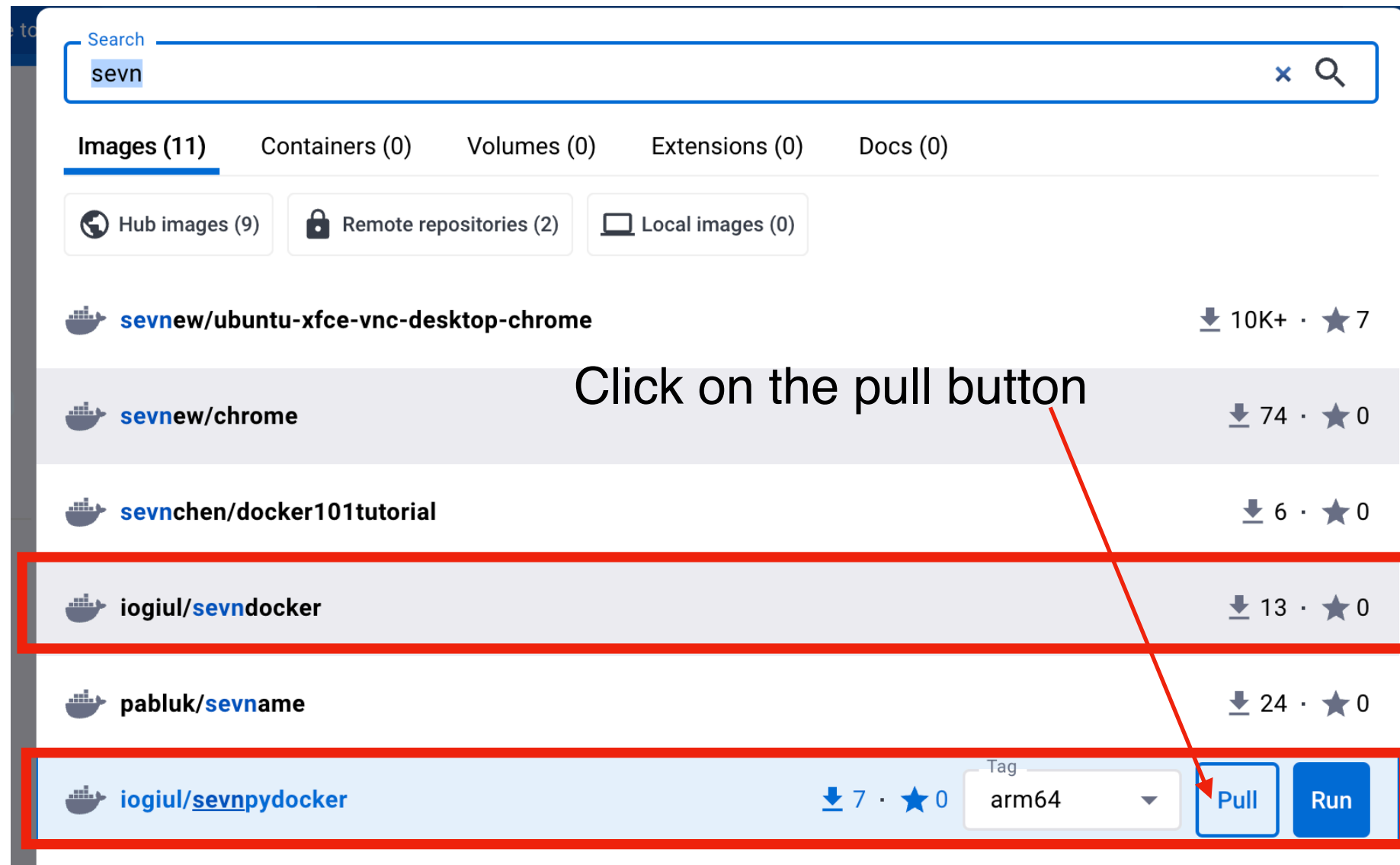


2nd- Step: Getting the SEVN container

- Use the built-in search engine to look for the SEVN containers:

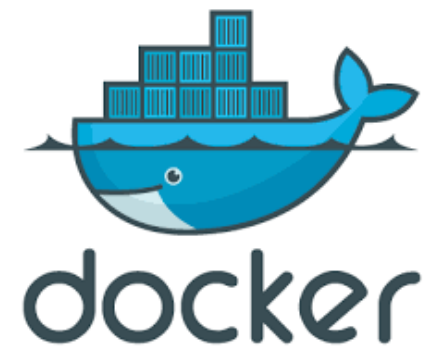
sevndocker: most basic SEVN container (~1 Gb)

sevnpydocker: contains SEVN plus [SEVNpy](#) and Python (~2 Gb)



The images store 1-2 Gb of contents, it can take some time to pull them, be patient!

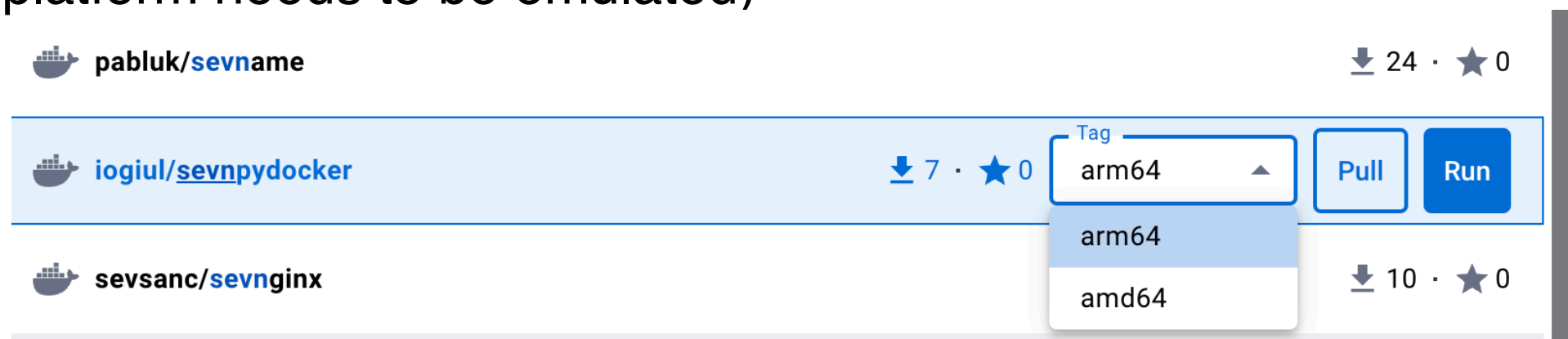
Using docker



2nd- Step: Getting the SEVN container

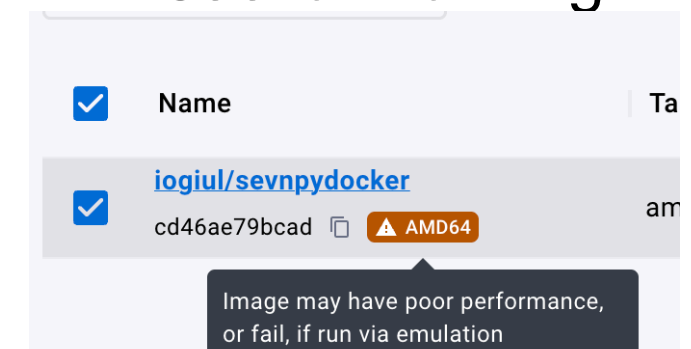
- WHAT TAG?

The two containers have two tags **amd64** and **arm64**, they refers to different building platform. In general, both can work in any system, but if the specific platform of the host systems is not chosen, the execution will be extremely slow (the platform needs to be emulated)

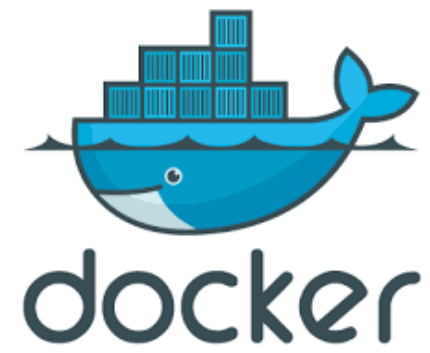


- Choice **arm64** if you have a Mac with new Apple silicon chip (M1, M2) or a Raspberry.
- Choice **amd64** in all the other cases.

 Anyway, if the chosen platform is not the right one, you will see a warning message in the Docker image window ([see next slide](#))



Using docker



2nd- Step: Getting the SEVN container

- After pulling the container (it could require some time), the image(s) will appear in the images section. They are now ready to be run

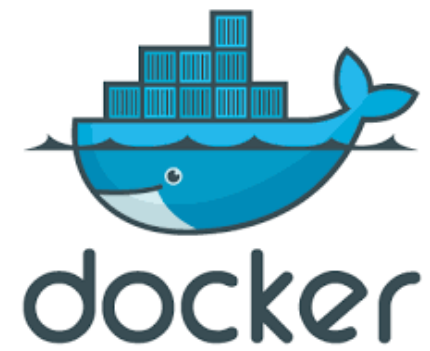
Docker Desktop interface showing the 'Images' section. The 'Images' tab is selected in the left sidebar. The main area shows a list of two images:

Name	Tag	Status	Created	Size	Actions
iogiul/sevndocker 44383173938b	arm64	Unused	9 minutes ago	1.05 GB	⋮
iogiul/sevnpdocker e6e71765f184	arm64	Unused	23 hours ago	2.15 GB	▶ ⋮

The 'Run' button is highlighted for the 'iogiul/sevnpdocker' image.

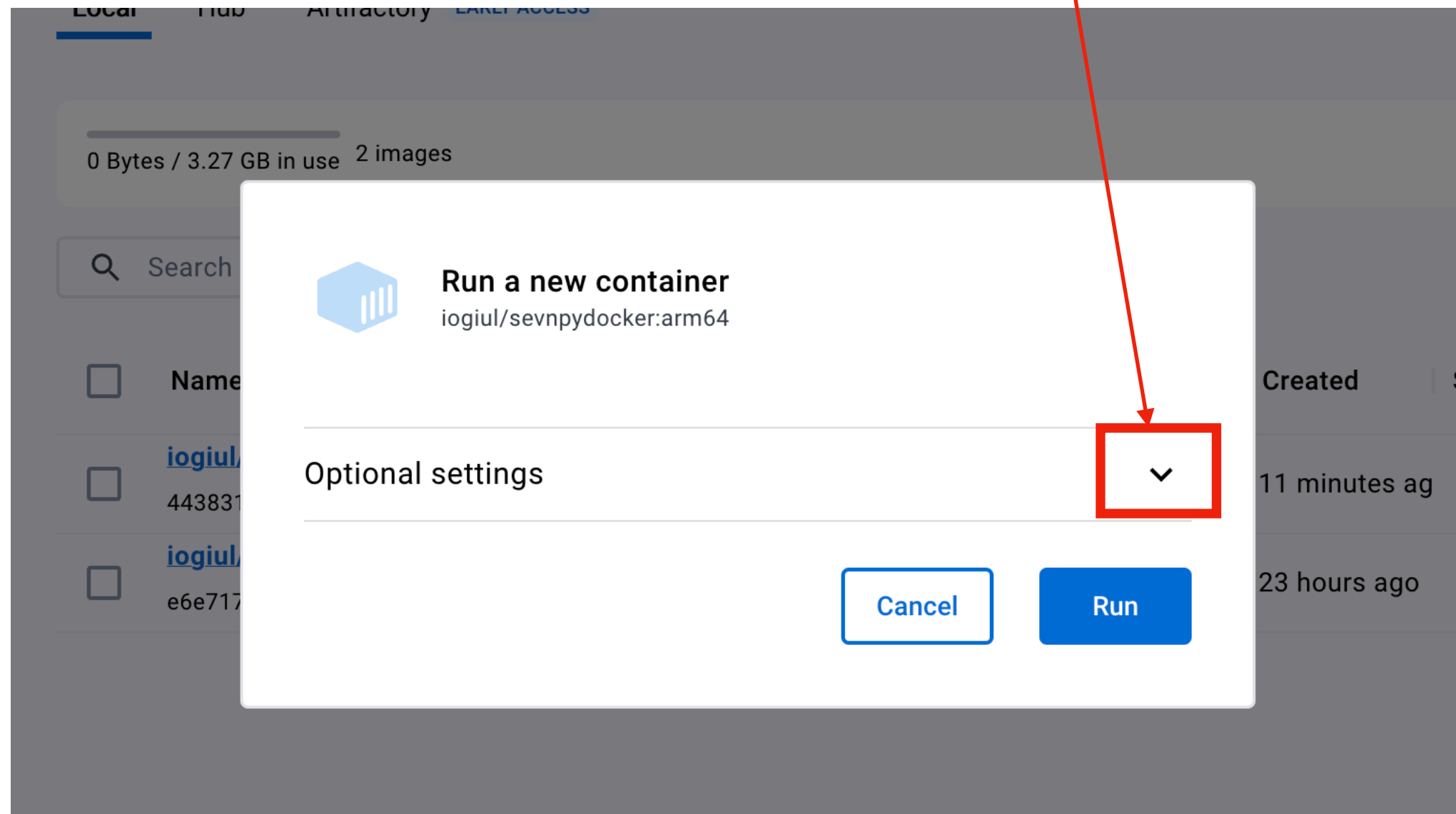
Click here to run an image

Using docker

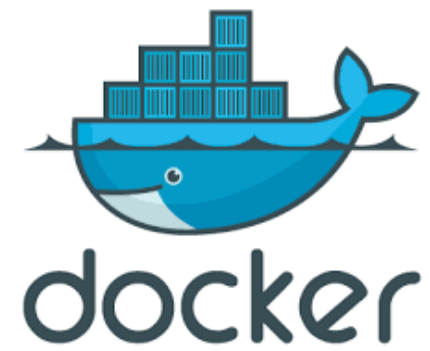


3rd- Step: Running the SEVN container

- Once clicked on the run button, a new window will appear click on the options settings arrow



Using docker



3rd- Step: Running the SEVN container

- **Container name:** you can chose a name for the running container, otherwise a random (funny one!) will be assigned

 **Run a new container**
iogiu/sevnpodocker:arm64

Optional settings 

Container name

A random name is generated if you do not provide one.

Ports
Enter "0" to assign randomly generated host ports.

Host port

:8888/tcp

Volumes

Host path

...

Container path

+

Environment variables

Variable

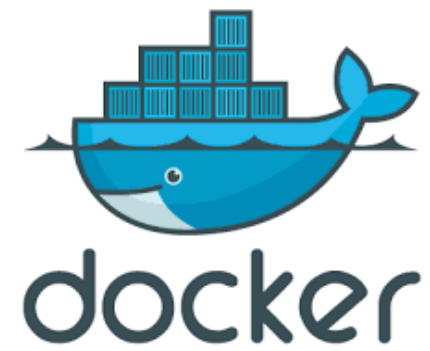
Value

+

Cancel

Run

Using docker



3rd- Step: Running the SEVN container

- **Container ports:** select the host port to communicate from the host to the container, putting 0 will produce a random port. It is useful for the sevnpy container to access the Jupyter notebook(s) created on the container

Optional settings ^

Container name

A random name is generated if you do not provide one.

Ports

Enter "0" to assign randomly generated host ports.

Host port :8888/tcp

Volumes

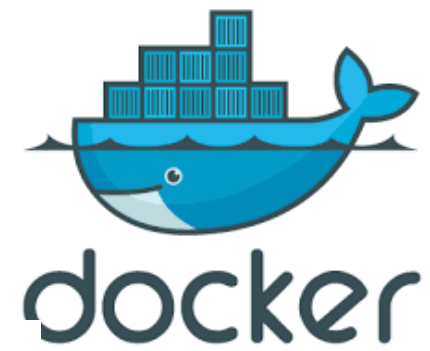
Host path ... Container path +

Environment variables

Variable Value +

Cancel Run

Using docker



3rd- Step: Running the SEVN container

- **Volumes:** Click on the ... to select a local path to mount as a volume in the container. The data in the path will be accessible for read and write. So they can be used to store the SEVN initial conditions and the SEVN outputs so that they will be available also outside the container. The volume will be accessible in the container in the folder set in the container path. Use the standard value *home/sevnuser/ext_data*

Container name

A random name is generated if you do not provide one.

Ports

Enter "0" to assign randomly generated host ports.

Host port :8888/tcp

Volumes

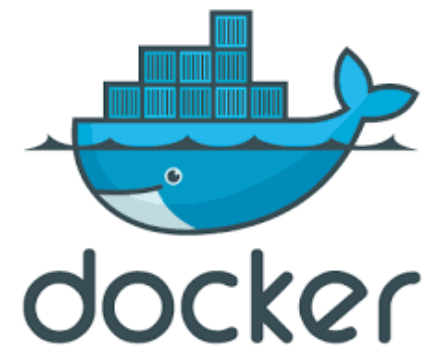
Host path ... Container path +

Environment variables

Variable Value +

Cancel Run

Using docker



3rd- Step: Running the SEVN container

- **Environment variables:** if you want to add environment variables, use this section. For example you may want to add the SEVNPATh variable containing the PATH to the SEVN folder (*/home/sevuser/sevn*)

Optional settings

Container name

A random name is generated if you do not provide one.

Ports

Enter "0" to assign randomly generated host ports.

Host port:8888/tcp

Volumes

Host path ... Container path +

Environment variables

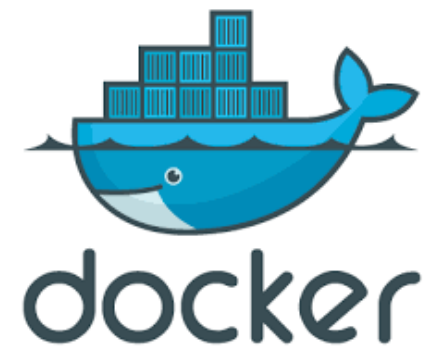
Variable

Value

+

Cancel Run

Using docker



3rd- Step: Running the SEVN container

- Example:

- The container will be called “mysevncont”
- The container exposes the port 8888 that will be listen by the local port 8888
- The contained will access all the data in my main account folder through the folder ext_volume in the working directory /app/ SEVN
- When run the SEVNPATH env variables will be set to /app/SEVN/sevn

 iogiul/sevnpydocker:arm64

Optional settings ^

Container name

A random name is generated if you do not provide one.

Ports

Enter “0” to assign randomly generated host ports.

Host port :8888/tcp

Volumes

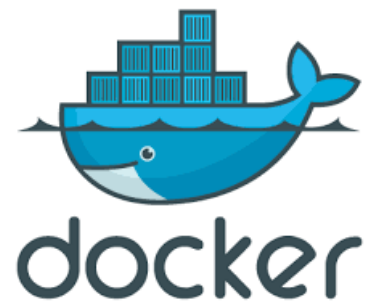
Host path	Container path	
/Users/giulianoiorio/	/home/sevnuser/ext_volu	+

Environment variables

Variable	Value	
SEVNPATH	/home/sevnuser/sevn	+

When satisfied with the configuration, click on run

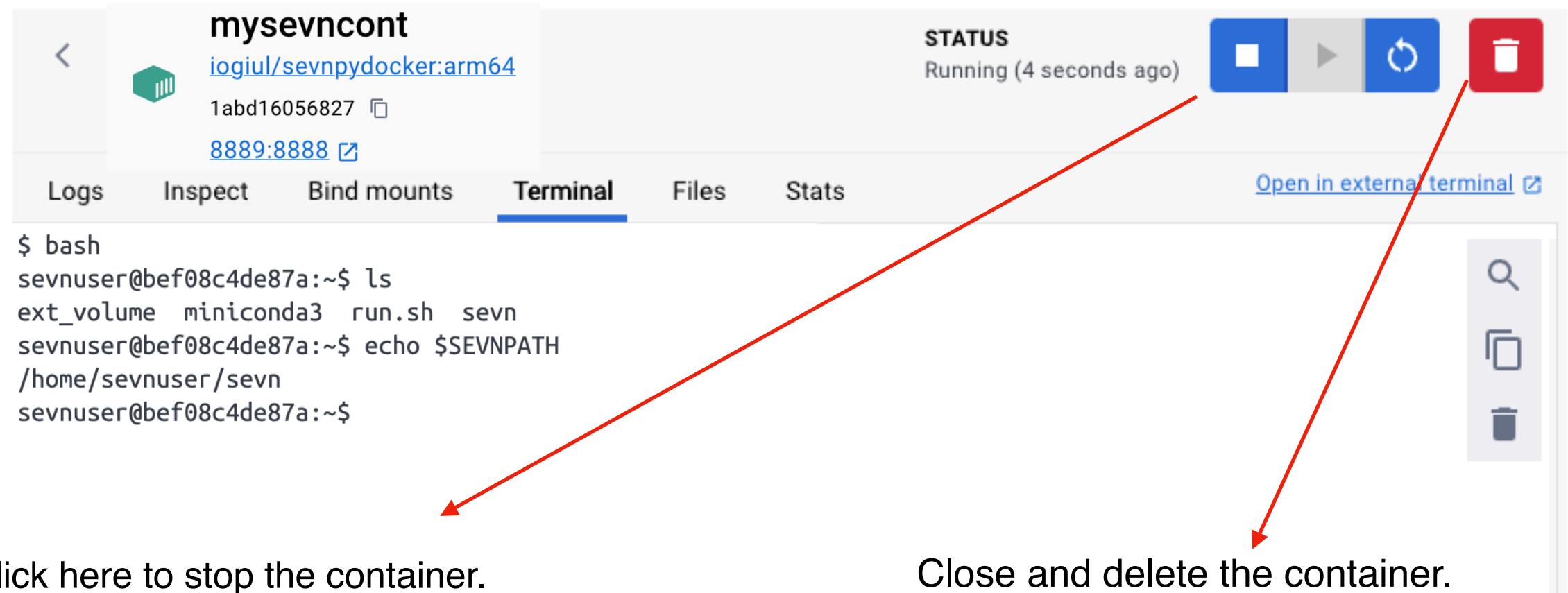
Using docker



4rd- Step: Using the internal docker terminal

- Congratulation, your SEVN container is running. A window like the one before will appear. Click on the terminal section to work directly on a terminal inside the container. First of all type `bash` to activate the bash shell, then you can check that you land in the default working directory `/app/SEVN`, and that the env variable `SEVNPAT`H is set to the value we give during the configuration.

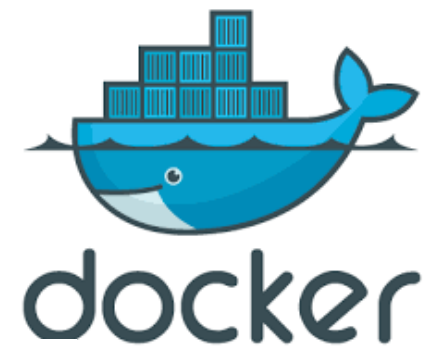
Now you can follow the rest of the userguide (actually you can skip the installation part since SEVN is already installed in the container)



Click here to stop the container.
It can be re-activated later-on
(click on the next icon)

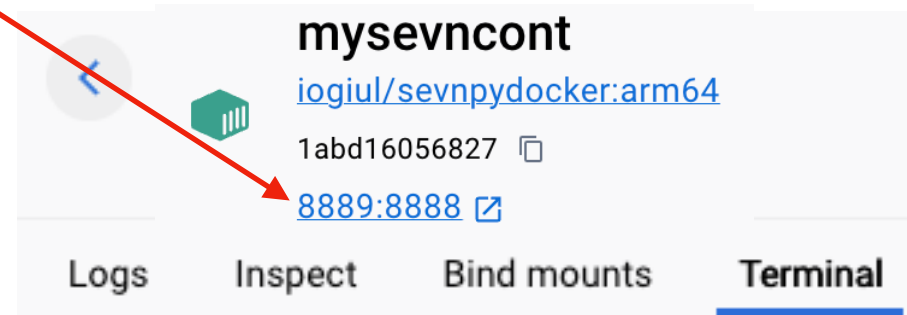
Close and delete the container.
Notice: the local file (the one not in the
external volume will be lost)

Using docker

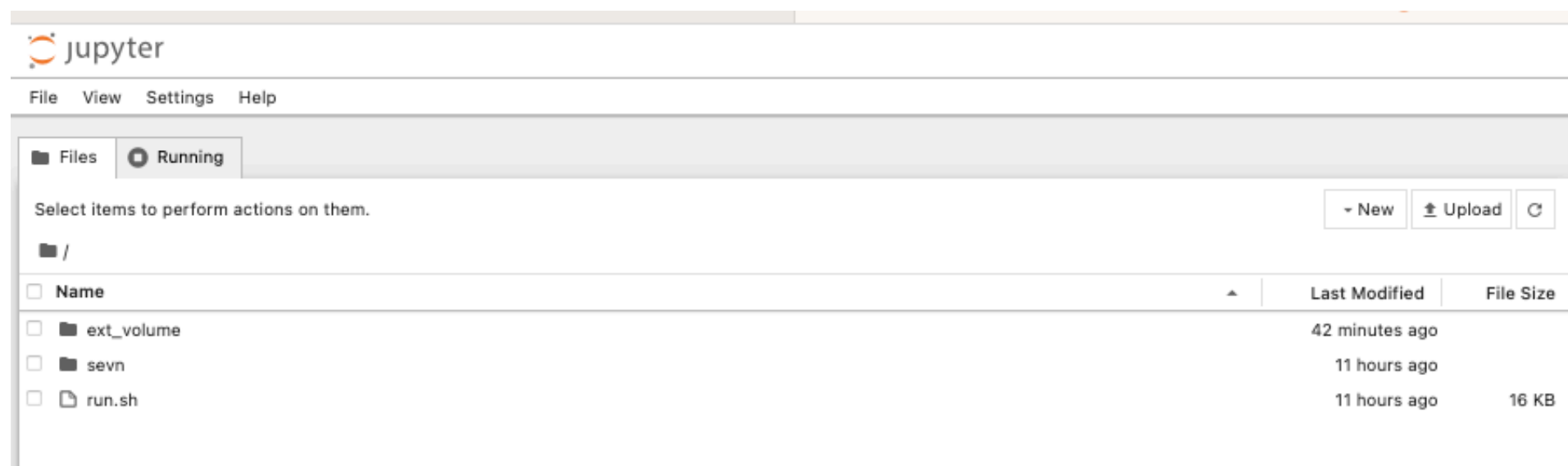


Special: The running Jupyter-notebook

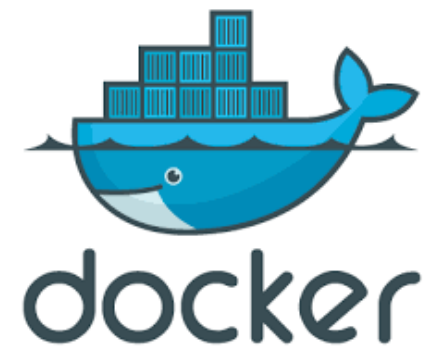
- In case you are running the **sevnpydocker** container there is a running process with a Jupyter notebook that can be directly accessed with your browser. You can click on the port information on the docker window



- or just open a browser and type <http://localhost:8890>
- A Jupyter notebook will appear. It is run directly inside the container



Using docker from terminal



In Linux based system it could be simpler to use docker directly from terminal. To install the Docker engine please follow the instruction <https://docs.docker.com/engine/install>.

Once the docker engine are installed, give a look to the post-installation steps (<https://docs.docker.com/engine/install/linux-postinstall/>) to complete the Docker setup.

At this point you can use the Docker command from the terminal, use

```
docker --help
```

to give a look to the possible commands.

To get the the sevpv or sevn images from the docker hub just use

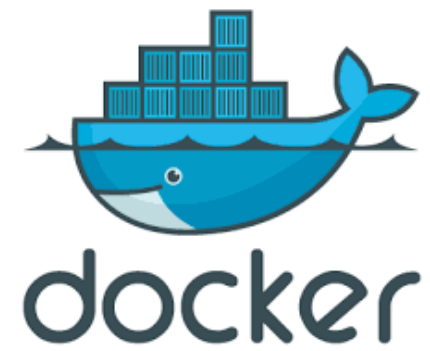
```
docker pull iogiul/sevnpydocker:amd64  
docker pull iogiul/sevndocker:amd64
```

To list of all the loaded images uses

```
docker images
```

To run the docker [see the next slide](#)

Using docker



Special: Container resources

- The container accesses directly all the resources of your machine (cpus, rams), so you can run SEVN exactly as you were using directly your machine, for example concerning the number of threads

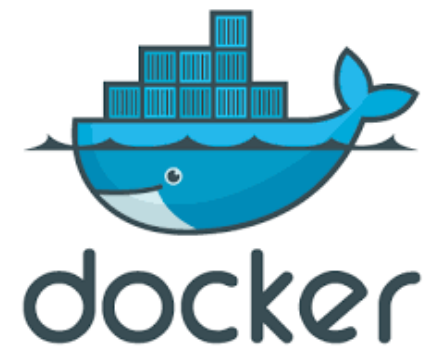
Special: command line docker

- This guide has been written thinking to a GUI use of the Docker app, but Docker can also be used from the command line using the terminal in Linux and MacOS and the power-shell in Windows (the command are the same). For example to run an image in the way equivalent to what seen [previously](#). One can use:

```
docker run -it
--name mysevncont \
-p 8890:8888 \
-e SEVNPATh="/home/sevenuser/sevn" \
-v /Users/giulianoiorio:/home/sevnuser/ext_volume \
iogiu/sevnpydocker:amd64(or arm64) \
/bin/bash
```

After running the command, the terminal will enter directly in the bash shell of the container. See the [official docker documentation](#) for other command line docker commands.

Using docker



Special: command line docker 2

The previous command just initialise a container with SEVN already installed, if you want to run the property container with the Jupyter notebook running in background, this is a two step procedure:

First step, run the container in detach mode

```
docker run -d
--name mysevncont \
-p 8890:8888 \
-e SEVNPATh="/home/sevnuser/sevn" \
-v /Users/giulianoiorio:/home/sevnuser/ext_volume \
iogiu1/sevnpydocker:amd64(or arm64)
```

Now the container is running in background, you can type `docker ps` to see it

Second step, enter in the running container

```
docker exec -it mysevncont bin/bash
```

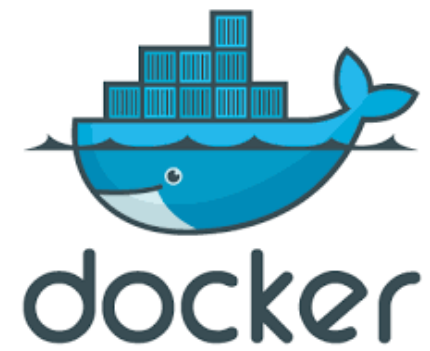
You will enter in the bash terminal of the container. To accesse the running Jupyter notebook, open a browser and write <http://localhost:8890>

To stop the container: `docker stop mysevncont`

To restar it: `docker start mysevncont`

To remove it: `docker rm mysevncont`

Using docker



Special: user

By default running the Docker container you will enter as the generic non-root and non-sudo user *sevnuser*.

The user can anyway install python packages through pip or running `python setup.py install` and write/read and execute files in the home and in the attached external volume (if the original user can actually access the external resources).

We do not suggest to use the Docker container as user with root privilege, however if a sudo access is really required it is possible to build a new image from the Docker files in the folder Docker uncommenting the row starting with [Optional]



```
# [Optional] Add sudo support. Enable this at your own risk.
#RUN apt-get update \
    #&& apt-get install -y sudo \
    #&& echo $USERNAME ALL=\(root\) NOPASSWD:ALL > /etc/sudoers.d/$USERNAME \
    #&& chmod 0440 /etc/sudoers.d/$USERNAME
```

Compiling

Two options:

1. Manually use CMake
2. Use an automated script

Compiling

CMake:

1. Open the terminal and go to the SEVN folder
2. Remove the *build* folder, if present, and create a new one

```
rm -rf build; mkdir build
```

3. Enter in the build folder and run cmake

```
cd build; cmake ..
```

4. If everything run smooth, you should see the following message

```
--Configuring done  
--Generating done  
--Build files have been written to: <current build path>
```

5. Compile with Make

```
make
```

6. If everything run smooth, you should see the following message

```
Linking CXX executable sevn.x  
Built target sevn.x  
Linking CXX executable sevnB.x  
Built target sevnB.x
```

Compiling

1. The installation script *compile.sh* is inside the SEVN folder
2. Make the script executable

```
chmod u+x compile.sh
```

3. Run it (from anywhere, the script does not need to remain in the SEVN folder)

```
./compile.sh -p <PATH_TO_SENV_FOLDER>
```

- 3b. Alternatively, you can insert the path to SEVN folder directly in *compile.sh* file at the line `SEVN="<Insert absolute SEVNpath>"`, in that case just use

```
./compile.sh
```

4. If everything runs smooth, you should see this message

```
*****  
SEVN has been successfully compiled  
...
```



This script has other options, call `./compile.sh -h` to list them

Compiling - only make



CMake is used to “make” the Makefile, then the Makefile is used to compile SEVN. **Therefore to recompile SEVN it is not needed to call cmake again.**

CMake:

1. Go inside the SEVN folder and type

```
make
```

Installation script:

1. Run the installation script with the the option *-m*

```
./compile.sh -p <PATH_TO_SENV_FOLDER> -m
```

- 1b. If the path to the SEVN folder is written inside the script

```
./compile.sh -m
```

Compiling - parallel compilation

If your machine has multiple CPUs, it is possible to speed-up the compilation performing it in parallel

CMake:

1. Go inside the SEVN folder and type

```
make -j
```

Installation script:

1. Run the installation script with the option *-j*

```
./compile.sh -p <PATH_TO_SEVN_FOLDER> -j
```

- 1b. If the path to the SEVN folder is written inside the script

```
./compile.sh -j
```



The parallel compilation uses a large amount of RAM memory. 4 GB of available memory at compilation time should be a safe threshold. If for any reason such amount of RAM is not available in your machine, please just use the standard serial compilation.

Compiling - extra options

The SEVN cmake has a number of compiling option:

```
cmake .. [-Dbin=ON*/OFF] [-Dsin=ON*/OFF] [-Dopenmp=ON*/OFF] [-Dh5=ON/OFF*]  
[-Ddebug=ON/OFF*] [-Dtest=ON/OFF*] [-Ddoc=ON/OFF*]
```

* indicates the default value

Standard users

- Dbin** Enable the compilation of the BSE executable *sevnB.x*
- Dsin** Enable the compilation of the SSE only executable *sevnB.x*
- Dopenmp** Enable OpenMP parallelization (if available)
- Dh5** Enable the hdf5 compilation for output (not yet available)

Developers

- Ddebug** Enable the debug run mode, notice this can be set at runtime
- Dtest** Compile the test suit, only possible for CMake V>3.2
- Ddoc** Produce the doxygen documentation

Library

Other additional options regarding the SEVN library can be found [here](#)



The *compile.sh* script always uses the default cmake options.

Running SEVN



Executables

After a successful compilation the executables can be found in the folder

SEVN/build/exe

sevnB.x

Executable for the SSE+BSE evolution. Input systems are binaries

sevn.x

Executable for the SSE evolution only. Input systems are single stars



Both are executed in the same way and accept the same parameters, the only difference is in the input list format. The parameters acting on the binary processes will not be used in the SSE-only executable

Executables

Executing `sevnB.x` and `sevn.x` without arguments will return the list of [available runtime parameters](#) (with the default values) and the info [about the SEVN version](#):

```
(base)   ./sevnB.x
#USED PARAMS
ce_alpha:          3          //[N][PS] alpha in binding energy (Eq. 73 in H
ce_kce:            1          //[N][PS] Fraction of non core mass participa
scaled version of eq. 77 In Hurley just for che CE mass
ce_knce:           1          //[N][PS] Fraction of non core mass not partic
  in Hurley 2002 (ce_kce is ignored unless it is -1, in this case the SEVN1 binding energy method is used)
eddington_factor:  1          //[N][PS] Eddington factor to limit accretion
on accretion
ev_Nchunk:         1000       //[N][PS] Evolve Nchunk at time
ev_max_repetitions: 100       //[N][PS] Maximum number of repetitions allowed
ev_naked_tshold:   0.0001     //[N][PS] Mass difference threshold (Msun) betw
gw_tshold:         1          //[N][PS] Time in units of Hubble time. If GWt
```

```
ts_check_spin_bse: false      //[B][PS] If true ta
use_thg_hurley:    false      //[B][PS] If true es
xspin_bavera:      false      //[B][PS] If true th
```

SEVN version info:

```
-----
SEVN VERSION:2.1.6
SEVN BRANCH:SEVNrc
-----
```

Git local info:

```
COMMITTS COUNTER:84
LAST COMMIT HASH:5453583393f514285d6eb7c26f4a69840f1d9f05
LAST COMMIT DATE:2023-01-28 01:34:55 +0100 (TIMESTAMP: 1674866095)
```

Git remote info:

```
COMMITTS COUNTER:84
LAST COMMIT HASH:5453583393f514285d6eb7c26f4a69840f1d9f05
LAST COMMIT DATE:2023-01-28 01:34:55 +0100 (TIMESTAMP: 1674866095)
```

How to run

Two options:

1. Manually run the executable

```
./sevnB.x -list <path_to_input_list> [optional params]
```

list is the only required parameters, all the others have default values and can be omitted.

Notice, in this example we assume that we are in the same folder of the *sevnB.x* executable. If this is not the case use

```
<path_to_the_executable_directory>/./sevnB.x  
-list <path_to_input_list> [optional params]
```

How to run

Two options:

2. Use automated scripts

The automated scripts are in the folder *SEVN/run_scripts/*

1. Make the script executable

```
chmod u+x run.sh
```

2. Open the *run.sh* script with a text editor

3. Set the variable SEVN as the path to the SEVN folder

```
SEVN=<Insert_absolute_sevn_path>
```

4. Set other optional variables. **Notice all the options are already set in the file with their default value**

5. Run the script, inserting as argument the path to the input list

```
./run.sh <path_to_input_list>
```



You don't have to run the script in the folder you find it (*SEVN/run_scripts/*), the idea is that they can be moved, while the SEVN executables remain in the SEVN folder

How to run - from download to run

A complete walkthrough

1- We start downloading (cloning) the code from gitlab

```
(np_veclib) giulianoiorio@MacBook-Air-di-Giuliano ~ % git clone https://gitlab.com/sevncodes/sevn.git
Cloning into 'sevn'...
remote: Enumerating objects: 3569, done.
remote: Counting objects: 100% (166/166), done.
remote: Compressing objects: 100% (157/157), done.
remote: Total 3569 (delta 27), reused 107 (delta 8), pack-reused 3403
Receiving objects: 100% (3569/3569), 78.98 MiB | 2.22 MiB/s, done.
Resolving deltas: 100% (1768/1768), done.
```

Clone the repository

```
(np_veclib) giulianoiorio@MacBook-Air-di-Giuliano ~ % cd sevn
(np_veclib) giulianoiorio@MacBook-Air-di-Giuliano sevn % ls
CMakeLists.txt      compile.sh           listBin.dat         run_scripts
LICENSE             doc                 listSin.txt         src
README.md           extern             main                tables
auxiliary_data      hdf5_interpreter    pyscript
cmake               include            resources
```

Check the folder

```
(np_veclib) giulianoiorio@MacBook-Air-di-Giuliano sevn % git branch
* SEVN
(np_veclib) giulianoiorio@MacBook-Air-di-Giuliano sevn %
```

Check the branch

How to run - from download to run

A complete walkthrough

2- Locate the important files

```
(np_vecLib) giulianoiorio@MacBook-Air-di-Giuliano sevn % ls
CMakeLists.txt      compile.sh           listBin.dat         run_scripts
LICENSE             doc                 listSin.txt         src
README.md           extern             main                tables
auxiliary_data      hdf5_interpreter   pyscript
cmake                include            resources
```

```
(base) iogiul@Giulianos-MacBook-Pro SEVN % cd run_scripts
(base) iogiul@Giulianos-MacBook-Pro run_scripts % ls
listBin.dat  listStar.dat  run.sh        run_sse.sh
```

compile.sh: script to compile SEVN

run.sh: script to run SEVN (BSE version)

run_sse.sh: script to run SEVN (SSE version)

listBin.dat: template to be used as input file for BSE (it contains only 1 system)

listSin.dat: template to be used as input file for SSE (it contains only 1 system)

How to run - from download to run

A complete walkthrough

3- Create a “run” folder and copy the script files

Notice: the “run” folder can be created everywhere and it do not need a specific name.

```
(base) iogiul@Giulianos-MacBook-Pro SEVN % cd ..
(base) iogiul@Giulianos-MacBook-Pro SEVN_tutorial % mkdir SEVN_run
(base) iogiul@Giulianos-MacBook-Pro SEVN_tutorial % ls
SEVN      SEVN_run
(base) iogiul@Giulianos-MacBook-Pro SEVN_tutorial % cd SEVN_run
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % cp ../SEVN/{compile.sh,run_scripts/run.sh,run_scripts/listBin.dat} .
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % ls
compile.sh listBin.dat run.sh
(base) iogiul@Giulianos-MacBook-Pro SEVN_run %
```

Create folder

copy files



Reasons behind the compilation and run scripts:

The standard and suggested SEVN usage assumes that for each run a dedicated folder is created and the compilation and run scripts are used.

The reason behind this guideline is to leave the SEVN folder as clean as possible.

The run script directly contains the SEVN runtime parameters, so that executables and parameters are bound together improving the reproducibility and reducing the risk of confusing the set of parameters related to a given output.

How to run - from download to run

A complete walkthrough

3- Compile SEVN

open the compile.sh file with your favourite editor

```
(base) iogiu1@Giulianos-MacBook-Pro SEVN_run % vi compile.sh
```

locate the variable SEVN

```
#!/usr/bin/env bash
```

```
SEVN="<Insert absolute SEVNpath>" #Complete path to the SEVN folder
```

Set the variable SEVN with the path of the SEVN folder

```
#!/usr/bin/env bash
```

```
SEVN="/Users/iogiu1/SEVN_tutorial/SEVN" #Complete path to the SEVN folder
```

How to run - from download to run

A complete walkthrough

3- Compile SEVN

Make the script executable

```
[(base) iogiul@Giulianos-MacBook-Pro SEVN_run % ls  
compile.sh listBin.dat run.sh  
[(base) iogiul@Giulianos-MacBook-Pro SEVN_run % chmod u+x compile.sh  
run it
```

```
[(base) iogiul@Giulianos-MacBook-Pro SEVN_run % ./compile.sh
```

-- Generating Makefile for the SEVN code

```
CMake Deprecation Warning at CMakeLists.txt:1 (cmake_minimum_required):  
Compatibility with CMake < 2.8.12 will be removed from a future version of  
CMake.
```

If SEVN has been successfully compiled, this message appears

```
*****  
SEVN has been successfully compiled  
We recommend to set all the relevant parameters in one the run script inside the run_scripts folder and to run  
SEVN through the same script (e.g. ./run.sh).  
Have fun!  
*****
```

How to run - from download to run

A complete walkthrough

4- Prepare the run script

open the run.sh file with your favourite editor

```
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % ls
compile.sh  listBin.dat  run.sh
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % vi run.sh
```

locate the variable SEVN

```
#-----
#SEVNPATH
#-----
SEVN="<Insert absolute SEVNpath>" #Complete path to the SEVN folder
```

Set the variable SEVN with the path of the SEVN folder

```
#-----
#SEVNPATH
#-----
SEVN="/Users/iogiul/SEVN_tutorial/SEVN" #Complete path to the SEVN folder
```

How to run - from download to run

A complete walkthrough

4- Prepare the run script

Locate the variable LISTBIN

```
#-----  
#Input and Tables  
#-----  
if [ "$cmdargument" = "" ]; then  
    LISTBIN="${SEVN}/run_scripts/listBin.dat" #Complete path to input file (list of b  
-
```

Set the variable LISTBIN with the path to the input file (in our case listBin.dat)

```
#-----  
#Input and Tables  
#-----  
if [ "$cmdargument" = "" ]; then  
    LISTBIN="listBin.dat" #Complete path to input file (list of binaries or si  
-
```



Notice, in this example we assume that the input file is the file *listBin.dat* in the same directory of the run script. If this not the case, use the right path to set LISTBIN

How to run - from download to run

A complete walkthrough

5- run the script

Make the script executable

```
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % chmod u+x run.sh  
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % █
```

Run the script

```
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % ./run.sh █
```

If SEVN run successfully, this message appears

```
SEVN execution was SUCCESSFUL!  
The output can be found in sevn_output/
```

How to run - from download to run

A complete walkthrough

6- check the output

The output are inside the sevn_output folder

```
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % ls
compile.sh  listBin.dat  run.sh      sevn_output
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % cd sevn_output
.....
(np_vecLib) giulianoiorio@MacBook-Air-di-Giuliano sevn_output % ls
evolved_0.dat          logfile_0.dat          sevnB.x
launch_line.txt        output_0.csv           used_params.svpar
.....
```

Congratulations! You run your first SEVN simulation

How to run - from download to run

A complete walkthrough

Change the [runtime options](#):

Suppose we want to run SEVN changing some of its default parameters, we can do it directly in the script. E.g. suppose we want to use 2 threads in the run and change the name of the output folder

Locate the name of the parameter you want to change

```
#-----  
#RUN OPTIONS  
#-----  
NTHREADS="1" #Number of OpenMP threads
```

```
#-----  
#OUTPUT  
#-----  
OUTPATH="sevn_output/" #Complete path to the
```

Set new parameter value

```
#-----  
#RUN OPTIONS  
#-----  
NTHREADS="2" #Number of OpenMP threads
```

```
#-----  
#OUTPUT  
#-----  
OUTPATH="my_output" #Complete path
```

Run again the SEVN script

```
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % ./run.sh
```

```
SEVN execution was SUCCESSFUL!
```

```
The output can be found in my_output
```

```
(base) iogiul@Giulianos-MacBook-Pro SEVN_run % ls
```

```
compile.sh  listBin.dat  my_output  run.sh  sevn_output
```

Input file

The single or binary systems to simulate has to be listed in an ascii file.
Each row must contain a single binary system or a star.

Input order:

Single star

M	Z	Ω	sn	t_{start}	t_{end}	dt_{out}	$seed$ [optional]
-----	-----	----------	------	--------------------	------------------	-------------------	-------------------

Binary

M_1	Z_1	Ω_1	sn_1	$t_{\text{start},1}$	M_2	Z_2	Ω_2	sn_2	$t_{\text{start},2}$	a	e	t_{end}	dt_{out}	$seed$ [optional]
-------	-------	------------	--------	----------------------	-------	-------	------------	--------	----------------------	-----	-----	------------------	-------------------	-------------------

1 is for the primary star, 2 for the secondary

Input file

Binary

M_1	Z_1	Ω_1	sn_1	$t_{\text{start},1}$	M_2	Z_2	Ω_2	sn_2	$t_{\text{start},2}$	a	e	t_{end}	dt_{out}	$seed$ [optional]
-------	-------	------------	--------	----------------------	-------	-------	------------	--------	----------------------	-----	-----	------------------	-------------------	-------------------

1 is for the primary star, 2 for the secondary

M - Mass [Msun]

Set the stellar mass (and type).

- **Standard:** double value with the ZAMS mass in Msun, e.g. *30.5*. A H-rich star will be initialised
- **Pure-HE stars:** double value with the zero-age-helium-main-sequence mass in Msun + HE suffix, e.g. *30.5HE*. A pureHE stars will be initialised.
- **Remnants:** double value with the remnant mass + remnant-label suffix:
 - BH, e.g. *30.5BH*. A black-hole of a given mass will be initialised.
 - NS, e.g. *1.5NS*. A of a neutron star of given mass will be initialised.
 - HEWD, COWD, ONEWD, e.g. *0.9HEWD*. A white dwarf (Helium WD, Carbon-Oxygen WD, Oxygen-Neon WD) of a given mass will be initialised.



If the remnant mass is not compatible with the remnant type (e.g. $M >$ Chandasekar mass for WDs), SEVN returns an initialisation error.

Input file

Binary

M_1	Z_1	Ω_1	sn_1	$t_{\text{start},1}$	M_2	Z_2	Ω_2	sn_2	$t_{\text{start},2}$	a	e	t_{end}	dt_{out}	$seed$ [optional]
-------	-------	------------	--------	----------------------	-------	-------	------------	--------	----------------------	-----	-----	------------------	-------------------	-------------------

1 is for the primary star, 2 for the secondary

Z - Metallicity [mass fraction of Z elements]

Set the stellar metallicity

Ω - Stellar Spin [angular velocity over critical angular velocity]

Set the stellar spin as the fraction of angular velocity over critical angular velocity

$$\Omega = \frac{\Omega_s}{\Omega_c} \quad \Omega_c = \sqrt{\frac{GM}{R_{\text{eq}}^3 \text{ (equatorial radius)}}}$$

sn - Supernova type [string]

Set the formalism to handle the transition to stellar remnant for massive stars, see [here](#) for further details

rapid, rapid formalism from [Fryer et al. 2012](#)

delayed, delayed formalism from [Fryer et al. 2012](#)

rapid_gauNS, rapid formalism from [Fryer et al. 2012](#) but the NS mass is drawn from a Gaussian

delayed_gauNS, delayed formalism from [Fryer et al. 2012](#) but the NS mass is drawn from a Gaussian

compact, compactness formalism from [Mapelli et al. 2020](#)

directcollapse, all the CCSNe produce a direct collapse, i.e. $M_{\text{remnant}} = M_{\text{star_preSN}}$

deathmatrix, prescription from [Woosley+20](#)

Input file

Binary

M_1	Z_1	Ω_1	sn_1	$t_{\text{start},1}$	M_2	Z_2	Ω_2	sn_2	$t_{\text{start},2}$	a	e	t_{end}	dt_{out}	$seed$ [optional]
-------	-------	------------	--------	----------------------	-------	-------	------------	--------	----------------------	-----	-----	------------------	-------------------	-------------------

1 is for the primary star, 2 for the secondary

t_{start} - Initial stellar age [Myr or string]

Set the age of the star at the beginning of the simulation:

- **Standard:** double value with the age in Myr, e.g. *0.5*.
- **Phase:** str with the name of the phase. The star will be initialised at the beginning of the phase (see table below), e.g. *tams*.
- **Fraction of phase:** str with % + a life percentage + : + code of the phase (integer, see table below), e.g. *%30:2*, i.e. 30% of the life during the phase 2 (*tms*).

Name of the phases

- *zams*, 1 (Zero Age Main Sequence)
- *tams*, 2 (Terminal Main Sequence)
- *shb*, 3 (shell H Burning)
- *cheb*, 4 (Core-He Burning)
- *tcheb*, 5 (terminal Core-He Burning)
- *sheb*, 6 (shell He Burning)

Input file

Binary

M_1	Z_1	Ω_1	sn_1	$t_{\text{start},1}$	M_2	Z_2	Ω_2	sn_2	$t_{\text{start},2}$	a	e	t_{end}	dt_{out}	$seed$ [optional]
-------	-------	------------	--------	----------------------	-------	-------	------------	--------	----------------------	-----	-----	------------------	-------------------	-------------------

1 is for the primary star, 2 for the secondary

a - Semi-major axis [Rsun]

Set the initial semi-major axis of binary
Double value in Rsun

e - Eccentricity

Set the initial eccentricity of the stars in the binary system.
Double value between 0 (inclusive) and 1 (exclusive)

Input file

Binary

M_1	Z_1	Ω_1	sn_1	$t_{\text{start},1}$	M_2	Z_2	Ω_2	sn_2	$t_{\text{start},2}$	a	e	t_{end}	dt_{out}	$seed$ [optional]
-------	-------	------------	--------	----------------------	-------	-------	------------	--------	----------------------	-----	-----	------------------	-------------------	-------------------

1 is for the primary star, 2 for the secondary

t_{end} - End time of the simulation [Myr or string]

Set the age at which the simulation is stopped

- **Standard**: double value with the age in Myr, e.g. *10000*.
- **end**: str end, the simulation is stopped when both stars become remnants e.g. *end*.
- **broken**: str broken, the simulation is stopped when the binary system breaks (because of a SN explosion, or destruction of a star), e.g. *broken*.

Input file

Binary

M_1	Z_1	Ω_1	sn_1	$t_{\text{start},1}$	M_2	Z_2	Ω_2	sn_2	$t_{\text{start},2}$	a	e	t_{end}	dt_{out}	$seed$ [optional]
-------	-------	------------	--------	----------------------	-------	-------	------------	--------	----------------------	-----	-----	------------------	-------------------	-------------------

1 is for the primary star, 2 for the secondary

dt_{out} - time schedule for output [Myr or string]

Set the times where the output will be stored

- **Standard:** double value in Myr, the outputs will be printed each dt, e.g. *0.1*.
- **List:** list of comma separated values (in Myr) within curly braces, the output will be printed exactly at the listed times, e.g. *{0.1,0.5,1,2,5}*
- **Interval:** three numbers (in Myr) separated by semicolons, the first number is the starting time for the output, the second number is the ending time and the third number is the time interval, e.g. *1:2:0.1*, print output from 1 Myr to 2 Myr each 0.1 Myr.
- **all:** str *all*, print outputs at each timestep. Notice the number and times of the output depends on the [adaptive timestep](#).
- **end:** str *end*, print just the first and last timestep, e.g. *end*.
- **events:** str *events*, print the outputs only when an event is triggered ([see events list in the wiki](#))
- **eventsrlo:** str *eventsrlo*, same as events, but in addition print in output all the steps during a RLO (in case of SSE it is exactly the same of events)

NOTICE: when an interval is set for the output (e.g. options standard or interval) this will force the adaptive timestep to have an upper limit equal to that interval

Input file

Binary

M_1	Z_1	Ω_1	sn_1	$t_{\text{start},1}$	M_2	Z_2	Ω_2	sn_2	$t_{\text{start},2}$	a	e	t_{end}	dt_{out}	$seed$ [optional]
-------	-------	------------	--------	----------------------	-------	-------	------------	--------	----------------------	-----	-----	------------------	-------------------	-------------------

1 is for the primary star, 2 for the secondary

seed - random seed number [integer]

Set the random seed

The integer value is used as random seed, for all the stochastic processes of the evolution (e.g. SN kicks). A binary with the same initial parameters (same SEVN parameters also) and the same seed will always experience the same evolution.



This parameter is useful for the reproducibility but also to reduce the stochasticity in the parameter exploration.



This is an optional input, if you want to use it, it has to be used for all the binary systems/stars in the input file.

If it is used, the run-time parameter *rseed* (see [later](#)) must be set to true.

Input file - placeholders

All the input parameters, except for the Mass(es), spin, semimajor axis, eccentricity and seed (if present) can be replaced in the input file by the **placeholder string: xxx**

e.g.

11.8 xxx 0 xxx xxx 9.6 xxx 0 xxx 350 0.3 xxx xxx

a system with $M_1=11.8$ Msun, $M_2=9.6$ Msun, $\text{Spin1}=\text{Spin2}=0$, Sem. axis=350 Rsun, $\text{ecc}=0.3$ and placeholders for all the others parameters.

A parameter with a placeholder **must be set as a SEVN runtime option**:
-Z, -snmode, -tini, -tf, -dtout ([see this slide](#))



Placeholders are useful to create initial-conditions “templates”, where a population can be run in SEVN testing, for example, different metallicities of sn models without creating new files.

Input file

Last remarks

- Binary systems and single stars cannot be combined in the same file. However, it is possible to effectively evolve a single star in a list of binaries setting the mass of one of the two stars to 0.
- *sevnB.x* accepts only input files containing binary systems, while *sevn.x* accepts only input files containing single stars.
- An empty row interrupts the file reading, everything below this row will not be considered by SEVN.
- The numbers have to be written using decimal or exponential notation (e.g. 1e2).
- Rows with typos and/or not consistent values raise a failed_initialisation error in SEVN. This error will not halt the SEVN execution. All the initial conditions with problems will be reported in the failed_initialisation file in the outputs of the simulation.

Runtime options

Sevn accepts a long list of optional runtime parameters, the following slides describe most of them, especially the ones that a user is more likely to interface with.

For each parameter we indicate both the name of the argument to be used if SEVN is **run directly** and the name of the variable in the **run script** referring to that parameter, e.g.

-list arg **LISTBIN="arg"**

Runtime options - input

List

-list arg **LISTBIN="arg"**

arg: String containing the path to the input file

default: no default, the argument is required

Both relative and absolute paths are ok



If the run script is used, the path to the list can be given as argument

```
./run.sh <path_to_input_list>
```

Random seed

-rseed arg **RSEED="arg"**

arg: String containing true or false

default: false

If true the, input file must contain the random seed in the last column .

If the input file contains this last column, this option has to be set to true. See [here](#).

Runtime options - input

Overwrite list inputs

The following options can be used to overwrite some input values of the [systems list](#):

Z: overwrite [Z - Metallicity](#) **-Z arg** Z="arg"

snmode: overwrite [sn - Supernova type](#) **-snmode arg** SUPERNOVA="arg"

tini: overwrite [tstart - Initial stellar age](#) **-tini arg** TSTART="arg"

tf: overwrite [tend - End time of the simulation](#) **-tf arg** TEND="arg"

dtout: overwrite [dtout - time schedule for output](#) **-dtout arg** DTOUT="arg"

arg: 'list' or same input type of the [systems list](#)

default: 'list', if 'list' the value will be taken from the [systems list](#)

 If an option is not set to 'list' the parameter will be **overwritten for all the systems in the input list.**

If [placeholders](#) are used in the input list, **the option 'list' cannot be used** to set the correspondent parameter.

Runtime options - tables

Tables

H-rich stellar tracks: **-tables arg** **TABLES="arg"**

pure-He stellar tracks: **-tables_HE arg** **TABLESHE="arg"**

arg: String containing the path to the folder containing the lookup tables

default:

H-rich stars: *<SEVN_folder>/tables/SEVNtracks_parsec_ov05_AGB*

pure-He stars: *<SEVN_folder>/tables/SEVNtracks_parsec_pureHe36*

The tables for H-rich and pure-He stars are the core of the stellar evolution, they are interpolated on-the-fly by SEVN.

H-rich stellar tracks are used to evolve “standard” star with hydrogen envelopes. pure-He stellar tracks are used to evolve pure-HE stars without hydrogen envelopes.

Runtime options - tables

Tables in SEVN

H-rich stars:

Name	Mass range [Msun]	Z range	Notes
<i>SEVNtracks_parsec_ov05_AGB</i>	2.2 - 450*	1E-11 - 0.04	Most updated tracks from PARSEC with overshooting parameter = 0.5
<i>SEVNtracks_parsec_ov04_AGB</i>	2.2 - 600	1E-4 - 0.04	Most updated tracks from PARSEC with overshooting parameter = 0.4
<i>SEVNtracks_MIST_AGB</i>	0.7 - 150	1.4E-5 - 0.045	MIST tracks (made with MESA)

*Some specific metallicities have Mzams up to 600 Msun

Runtime options - tables

The complete list of runtime options can be found [here](#)

Tables in SEVN

pure-He stars:

Name	Mass range [Msun]	Z range	Notes
<i>SEVNtracks_parsec_pureHE36</i>	0.360 - 350	1E-11* - 0.05	Most updated pureHE tracks from PARSEC

*Actually, the lowest metallicity is 1E-6, we just copy their tracks in the folder for metallicity 1E-11

Runtime options - run options

Nthreads

-nthreads arg **NTHREADS="arg"**

arg: integer containing the number of parallel threads to use

default: 1

Notice: if openMP is not supported or is not found by the compiler, SEVN uses only one thread disregarding this option

Nchunk

-ev_Nchunk arg **NCHUNK="arg"**

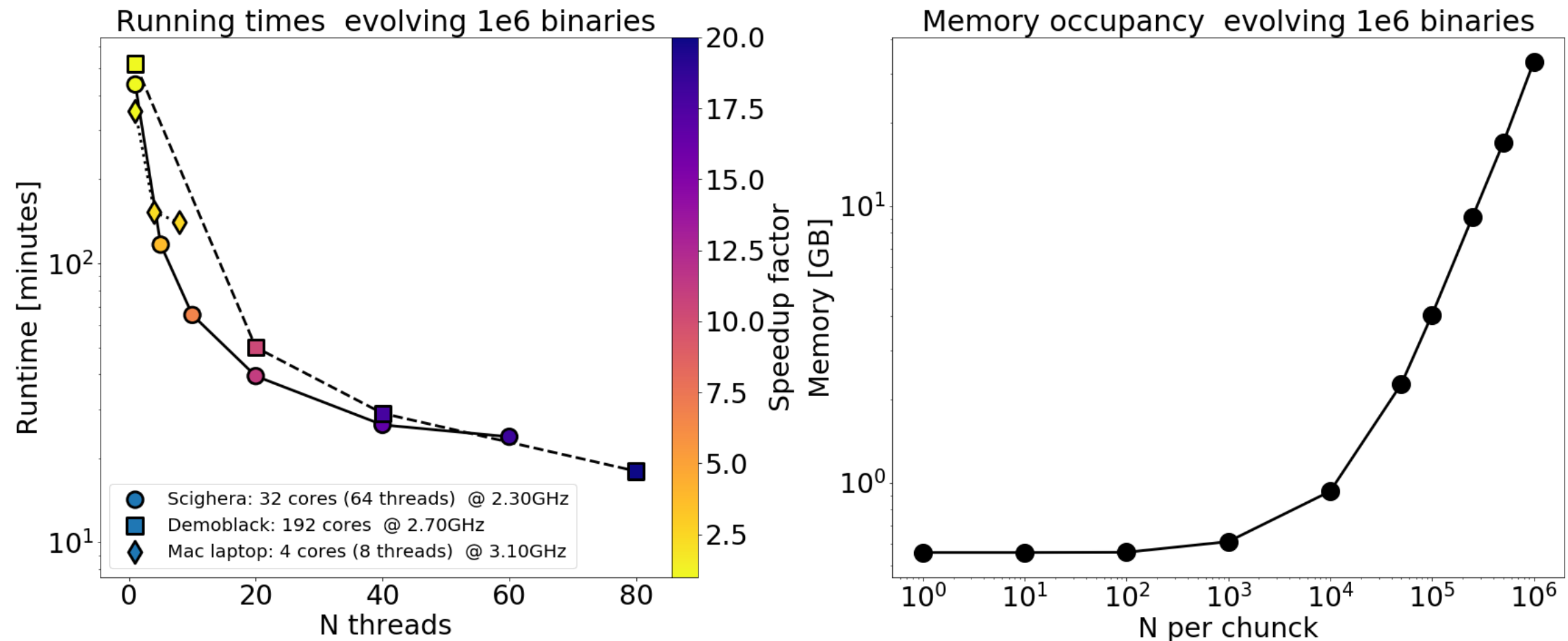
arg: integer containing the number of systems to evolve at each step

default: 1000

In SEVN only Nchunk systems at time are loaded and evolved. After the evolution of these systems is concluded a new set of Nchunk systems is loaded and evolved and so on and so forth until the end of the input file.

Larger values of Nchunk produce slightly faster run at the cost of higher memory occupancy.

Runtime options - run options



Notice: **the focus is the curves trend**, the absolute values of Runtime and memory can depend on the details of the run.

- **Use Nchunk as large as possible, leaving a safe unoccupied memory space (about 30%).** Be careful if you are using a shared machine. Use a number of **Nchunk that is multiple of Nthreads** (to balance the threads work).
- For such kind of simulations virtual multithreading is useless, therefore the **number of threads should be $\leq$ the number of physical cores**.

Runtime options - output options

Output folder

-o arg

OUTPATH="arg"

arg: path for the folder containing the outputs
default: sevn_output

 if the folder already exist, its content will be removed.

Output format

-omode arg

OMODE="arg"

arg: output format: *ascii*, *csv*, *hdf5*

default: csv

 hdf5 should be enabled at compile time, see this [slide](#)

Runtime options - output options

Literal phases

-io_literal_phases arg **LITPHASES="arg"**

arg: bool (true-false)

default: false

If true the phases, remnant types and [events](#) will be stored in output as strings, otherwise as integers (see conversion tables for [phases](#), [remnant types](#) and [events](#)).

Logfile

-io_logfile arg **LOGFILE="arg"**

arg: bool (true-false)

default: true

If true save the logfile(s) ([see here](#)) in output

Runtime options - output options

Star properties

-scol arg

SCOL="arg"

arg: list of properties separated by semicolons

default: *Worldtime:Mass:MHE:MCO:RadiusPhase:RemnantType*

These are the stellar properties that will be saved in the output files.

Binary properties

-bcol arg

BCOL="arg"

arg: list of properties separated by semicolons

default: *Semimajor:Eccentricity:BEvent*

These are the binary properties that will be saved in the output files.

Runtime options - output properties

Star properties

Notice: the * indicates that the property is interpolated from the tables if available otherwise it is estimated.
the ** indicates that the property is available only if tables are loaded (otherwise it is set to nan or zero)

Mass: stellar mass in Msun

MHE: mass of the HE core (includes also the CO core) in Msun

MCO: mass of the CO core in Msun

Luminosity: (bolometric) luminosity of the star in Lsun

Radius: radius of the star in Rsun

RHE*: radius of the He-core

RCO*: radius of the CO-core

Phase: phase of the star ([see phases tab](#), [see here](#))

RemnantType: remnant type ([see types tab](#), [see here](#))

Spin: angular velocity over critical angular velocity

OmegaSpin: stellar angular rotational velocity in yr^{-1}

AngMomSpin: stellar angular momentum $\text{Msun Rsun}^2 \text{yr}^{-1}$

Event: ID (or string) of the current SSE Event ([see here](#))

Runtime options - output properties

Star properties

Notice: the * indicates that the property is interpolated from the tables if available otherwise it is estimated.
the ** indicates that the property is available only if tables are loaded (otherwise it is set to nan or zero)

PhaseBSE: equivalent BSE phase of the star ([see phases tab](#))

Temperature: effective Temperature in K

Lambda: proxy of the binding energy ([see here](#))

Ebind: envelope binding energy in $M_{\text{sun}}^2/(R \cdot G)$

*Inertia**: moment of the inertia of the star in $M_{\text{sun}} R_{\text{sun}}^2$

Worldtime: elapsed time of the simulation in Myr

Timestep: timestep in Myr

Zams: Zams of the current [fake star](#) in M_{sun}

Zmet: Metallicity of the current [fake star](#)

*Qconv***: Mass fraction of the convective envelope ($M_{\text{cnv}}/M_{\text{tot}}$)

*Depthconv***: depth of the convective envelope over the total radius ($R_{\text{cnv}}/\text{Radius}$)

Runtime options - output properties

Star properties

Notice: the * indicates that the property is interpolated from the tables if available otherwise it is estimated.
the ** indicates that the property is available only if tables are loaded (otherwise it is set to nan or zero)

*Tconv***: convective cells turnover time in yr

dMcumul_binary: total mass accreted/lost in binary interactions in Msun.
It is reset to 0 after a change of track

dMcumul_RLO: total mass accreted/lost during a RLO. It is reset to 0 when the RLO ends

dMcumulacc_wind total mass accreted through winds since the beginning of the simulation

dMdt: mass lost through winds in the SSE (Msun/Myr)

dRdt: time derivative of the Radius variation (Rsun/Myr)

dMHEdt: time derivative of the He-core mass variation (Msun/Myr)

dMCOdt: time derivative of the CO-core mass variation (Msun/Myr)

dMRL0dt: mass accretion/loss rate during a RLO (Msun/Myr)

dMaccwinddt: mass accretion rate through winds (Msun/Myr)

Runtime options - output properties

Notice: the * indicates that the property is interpolated from the tables if available otherwise it is estimated.
the ** indicates that the property is available only if tables are loaded (otherwise it is set to nan or zero)

*Hsup***: surface H abundance (mass fraction)

*HEsup***: surface He abundance (mass fraction)

*Csup***: surface C abundance (mass fraction)

*Nsup***: surface N abundance (mass fraction)

*Osup***: surface O abundance (mass fraction)

Plife: percentage of life in the current phase (0 if remnant)

Remnant properties

These properties are available also for Stars, but most of them are set to nan before the remnant phase

Xspin: Adimensional BH spin

OmegaRem: NS remnant angular rotational velocity in s^{-1}

Bmag: Magnetic field in Gauss. Notice that atm it is just defined for NSs

Rs: Schwarzschild radius in R_{sun}

Runtime options - output properties

Binary properties

Semimajor: semimajor axis in R_{sun}

Eccentricity: eccentricity

dSemimajordt: time derivative of the semimajor axis variation ($R_{\text{sun}}/\text{Myr}$)

dEccentricitydt: time derivative of the Eccentricity variation ($R_{\text{sun}}/\text{Myr}$)

Period: orbital period in yr

AngMom: orbital angular momentum in $M_{\text{sun}} R_{\text{sun}}^2 \text{yr}^{-1}$

RL0: Roche-lobe radius of the first star in R_{sun}

RL1: Roche-lobe radius of the second star in R_{sun}

GWtime: Gravitational wave orbital decay time in Myr, it is used to estimate the merger time of the two binary objects.

It is estimated using a high-precision analytic approximation of the integration of the Peters equations (errors $<0.3\%$)

BWorldtime: elapsed time of the simulation in Myr

BTimestep: timestep in Myr

BEvent: ID (or string) of the current BSE/SSE Event ([see here](#))

Runtime options - output properties

Star/Binary properties - Events

Events list *Event*: ID (or string if [-io_literal_phases true](#)) of the current event

Event	Integer ID	str ID	Description
No Event	-1	""	
Phase change	0	"CPhase"	One of both the stars changed phase in this step
Remnant type change	1	"Crem"	The remnant star(s) change types (e.g. from NS to BH)
Quasi Homogeneous evolution	2	"QHE"	One of the two stars reach the condition to start a Quasi-Homogeneous evolution (possible only with option -rlo_QHE true)
Gravitational Wave (GW) emission	3	"GWB"	The systems reach the condition to start to consider GW emission in the evolution (it depends on the parameter -gw_tshold)
Roche-Lobe Overflow (RLO) start	4	"RLOB"	A stable RLO begins
Roche-Lobe Overflow (RLO) end	5	"RLOE"	The RLO ends in this timestep; Notice, sometime a RLOB could not be followed by a RLOE event (e.g. because of a merger or the explosion of a SN)
Collision	6	"Collision"	A Collision between two stars begins
Common Envelope (CE)	7	"CE"	A Common envelope evolution begins

Runtime options - output properties

Star/Binary properties - Events

Merger	8	"Merger"	The two stars are merged
CE + Merger	9	"CE_Merger"	A Common envelope evolution begins and end in a stellar merger
RLO start + Merger	10	"RLOB_Merger"	An unstable RLO begins ending directly in a merger
RLO start + CE	11	"RLOB_CE"	An unstable RLO begins ending directly in a Common Envelope
RLO start + CE + Merger	12	"RLOB_CE_Merger"	An unstable RLO begins ending directly in a Common Envelope that causes a stellar merger
Collision + Merger	13	"Collision_Merger"	A collision produces a direct stellar merger
Collision + CE	14	"Collision_CE"	A collision produces a Common Envelope
Collision + CE + Merger	15	"Collision_CE_Merger"	A collision produces a Common Envelope that causes a stellar merger
Swallowed	16	"Swallowed"	the donor star is completely swallowed by an due to an unstable mass transfer
RLO start + Swallowed	17	"RLOB_Swallowed"	At the onset of the RLO, the donor star is completely swallowed by an due to an unstable mass transfer
GW emission + Merger	18*	"GW_Merger"	The orbital decay due to GW emission causes a merger
System broken by SN explosion	19*	"SNBroken"	The SN explosion breaks the binary
System broken by SNIa	20*	"SNIa"	Explosion of a SNIa

Runtime options - output properties

An important caveat - New RLO without ending the previous one



Sometime a new RLO starts exactly the step after a previous RLO ends. In this case SEVN trigger the event RLO_BEGIN without concluding the previous RLO, so the event RLO_END is not triggered.

Anyway, the logfile will report both the end of the previous RLO and the begin of the new one.

Runtime options - output properties

An important caveat 2 - BWorldtime/Worldtime



Some timesteps are so tiny ($\leq 1e-6$) that the BWorldtime/Worldtime in output will not change. Therefore, it could happen that some rows report the same BWorldtime/Worldtime (especially if the [dout option is set to all](#)). Be careful if you are using the Worldtime in your analysis.

If you really care about the Worldtime and you are using the dtout option all, add also the property Timestep/BTimestep to the output. The “precise” Worldtime can be obtained summing the values in the Timestep/BTimestep column.

Runtime options - processes

Processes - Winds

-wmode arg WINDSMODE="arg"

arg: '*hurley_wind*' or '*disabled*'

default: '*hurley_wind*'

disabled: disable the process

hurley_wind: Wind formalism from [Hurley+02](#)

Related params:

alpha **-w_alpha double (def 1.5)** WALPHA="1.5"

tune the mass accretion rate (Eq. 6 [Hurley+02](#))

beta **-w_beta double (def 0.125)** WBETA="0.125"

tune the wind velocity (Eq. 9 [Hurley+02](#))

Runtime options - processes

Processes - Winds



NOTICE:

The Wind process in SEVN takes into **account only the accretion by winds and the orbital modifications** due to the mass transfer.

The wind mass loss is self-consistently accounted in the stellar evolution tracks.

Runtime options - processes

Processes - Supernova kicks

-sn_kicks arg **SNKICKS="arg"**

arg: *'unified', 'hobbs'_pure, 'hobbs', zeros'*

default: *'unified'*

zeros: disable the Supernova kicks (all kicks will have a module of 0)

hobbs_pure: kicks (module) are taken from a Maxwellian (see [Hobbs+05](#))

hobbs: same as hobbs_pure, but it includes a correction for the natal kick for the SN fallback ([Giacobbo+18](#)).

unified: kicks (module) following [Giacobbo+20](#). In practice the kick magnitude $V_{\text{kick}} \propto f_{\text{H05}} M_{\text{ej}} M_{\text{rem}}^{-1}$, where f_{H05} is drawn from the hobbs_pure Maxwellian (see above), M_{ej} is the mass of the SN ejecta and M_{rem} is the mass of the remnant.

Related params

Maxwellian sigma **-sn_kick_velocity_stdev double (def 265)** **SNVKICKSTD="265"**

1D σ parameter of the Maxwellian used to draw the natal kicks in km/s

Min Vkick **-sn_min_vkick double (def 0)** **SNMINVKICK="0"**

Minimum value of the module of the natal kick in km/s

Runtime options - processes

Processes - Supernova (SN) explosion

-snmode arg **SUPERNOVA="arg"**

arg: 'list', 'rapid', 'rapid_gauNS', 'delayed', 'delayed_gauNS', 'compact', 'deathmatrix', 'directcollapse'

default: 'list'

list: use the sn option in the input list, it could be one of the following

rapid: rapid formalism by [Fryer et al. 2012](#)

rapid_gauNS: same as rapid but the mass of the NS are drawn from a Gaussian

delayed: delayed formalism by [Fryer et al. 2012](#)

delayed_gauNS: same as delayed but the mass of the NS are drawn from a Gaussian

compact: compactness formalism by [Mapelli et al. 2020](#)

deathmatrix: sn formalism by [Woosley et al 2020](#)

directcollapse: always trigger a direct collapse, i.e. $M_{\text{remnant}} = M_{\text{star_preSN}}$

Runtime options - processes

Processes - Supernova (SN) explosion

Related params

Chandrasekar mass `-sn_Mchandra double (def 1.41)` `MCHANDRA="1.41"`

Assumed Chandrasekar mass in Msun, WDs with masses larger than this value explode as SNIa leaving no remnants.

WD/ECSN mass transition H stars `-sn_co_lower_ecsn double (def 1.38)` `SNLOWECSN="1.38"`

Mass transition (Msun) for the mass of the CO core to produce a WD or an electron capture SN leaving a NS.

WD/ECSN mass transition He stars `-sn_co_lower_ecsn double (def -1)` `SNLOWECSN="-1"`

Same as above, but for pure-He stars. If -1 use the same value of the H-stars

ECSN/ CCSN mass transition H star `-sn_co_lower_sn double (def 1.44)` `SNLOW="1.44"`

Mass transition (Msun) for the mass of the CO core to trigger an electron capture SN leaving a NS or to trigger a core collapse SN explosion/implosion leaving a NS, BH or no remnants.

ECSN/ CCSN mass transition He star `-sn_co_lower_sn double (def -1)` `SNLOW="-1"`

Same as above, but for pureHe stars. If -1 use the same value of the H-stars

Maximum NS mass `-sn_max_ns_mass double (def 3)` `NSMAZ="3"`

Maximum allowed mass of a NS in Msun

Runtime options - processes

Related params (for option “compact”):

Csi25 threshold `-sn_compact_csi25_tshold` (def -1) `SNC25TS=-1`

Compactness ($\xi_{2.5}$) threshold for explosion/implosion (see [Mapelli et al. 2020](#)).

If -1, use a stochastic explosion/implosion decision based on the distribution shown in the bottom panel of Fig. 3 in [Patton&Sukhbold20](#)

Retained H fraction `-sn_compact_fH` (def 0.9) `SNCOMPFBFS=“0.9”`

Retained H-envelope fraction before the CCSN. The final BH mass is $MBH = M_{He} + fH \cdot (M - M_{He})$, where M and M_{He} are the preSN mass and core mass

Related params (NS mass Gaussian distribution):

Some SN explosion models draw the NS mass from a Gaussian distribution. The following parameters set the properties of the Gaussian. Actually the Gaussian is truncated at a lower limit of 1.1 Msun and upper limit given by `-sn_max_ns_mass`

Centroid of the NS Gaussian mass distribution

`-sn_Mrmnant_average_NS` double (def 1.33) `NSMASSMEAN=“1.33”`

Gaussian centroid in Msun

Standard deviation of the NS Gaussian mass distribution

`-sn_Mrmnant_std_NS` double (def 1.33) `NSMASSSTD=“0.09”`

Gaussian standard deviation in Msun

Runtime options - processes

Processes - Supernova (SN) explosion pair instability

Correction for pair instability. Notice: the application of the correction for pair instability depends on the chosen sn explosion model: currently the sn explosion models *directcollapse* and *deathmatrix* do not apply the correction (*deathmatrix* already includes pair instability in the final outcomes)

-sn_pairinstability arg

SNPISN="arg"

arg: *'mapelli20', 'farmer19', 'disabled'*

default: *'mapelli20'*

- *disabled*: no corrections applied

- *mapelli20*: apply corrections to the remnant mass based on Appendix A of [Mapelli+20](#) (see also [Spera+17](#)). It is a fit to the tables in the [Woosley17](#) paper. Stars undergo PPI if the preSN MHE is within 32 and 64 Msun, while a PISN is triggered for $64 < \text{MHE}/\text{Msun} < 135$. The stars directly collapse to a BH for $\text{MHE} > 135 \text{ Msun}$

- *farmer19*: prescription based on the results of [Farmer+19](#). Stars undergo PPI if the preSN MCO is within 38 and 60 Msun, while a PISN is triggered for $\text{MCO} > 60 \text{ Msun}$. The stars directly collapse to a BH for $\text{MHE} > 135 \text{ Msun}$. The mass of the PPISN remnant is estimated using Eq. A1 in Farmer+19 where we set the minimum metallicity to $Z=1\text{E-}5$.

Runtime options - processes

Processes - Supernova (SN) explosion neutrino mass loss

Correction for neutrino mass loss during SN explosion

-sn_neutrinomloss arg **SNNML="arg"**

arg: '*lattimer89*', '*disabled*'

default: '*lattimer89*'

disabled: no corrections applied

lattimer89: apply corrections to the final remnant mass based [Lattimer&Yahil89](#), in the version discussed by [Zevin et al., 2020](#)

$$M_{\text{rem}} = \max \left[\frac{\sqrt{1 + 0.3 M_{\text{rem, bar}}} - 1}{0.15}, (M_{\text{rem, bar}} - 0.5 M_{\odot}) \right]$$

Runtime options - processes

Processes - Common Envelope

-cemode arg **CEMODE="arg"**

arg: '*energy*' or '*disabled*'

default: '*energy*'

disabled: disable the process

energy: "classical" alpha-lambda energy formalism (see [Hurley+02](#))

Related params - "energy option":

alpha **-ce_alpha double (def 3)** **CEALPHA="3"**

Common envelope efficiency (Eq. 71 [Hurley+02](#))

Lambda **-star_lambda double (def -1)** **CELAM="-1"**

Structural parameter to estimate the binding energy of the stellar envelope (Eq. 69 [Hurley+02](#)):

$$E_{\text{bind}} = - \frac{GM_{\text{star}} M_{\text{envelope,star}}}{\lambda_{\text{star}} R_{\text{star}}}$$

If >0 use a constant lambda equal to the input option, if <0 use a variable lambda following a given prescription ([see next slide](#) for available options)

Runtime options - processes

Processes - Common Envelope

Related params - “energy option”:

Lambda options `-star_lambda double (def -1)` `CELAM="-1"`

- 1: exactly as in BSE/MOBSE (based on [Claeys+14](#), but not exactly the same)
- 12: same as in -1, but considering the whole envelope as convective
- 2: exactly as in. [Claeys+14](#) (Appendix A)
- 21: same as in -2, but considering the whole envelope as convective
- 3: exactly as in [Izzard04](#)
- 31: same as in -3, but considering the whole envelope as convective
- 4: prescriptions by [Klencki+21](#), interpolating linearly in Mzams
- 41: prescriptions by [Klencki+21](#), “quantised” in Mass bins*
- 5: prescriptions by [Xi&Liu10](#)*, interpolating linearly in Mzams
- 51: prescriptions by [Xi&Liu10](#)*, “quantised” in Mass bins*

NOTICE

1- The BSE-MOBSE implementation are derived from Claeys+14 and Izzard04. However in BSE/MOBSE the fraction of convective envelope is used to define lambda, while in Claeys and Izzard the absolute mass of the convective envelope in Msun

2- The Xi&Liu (option -4 and -41) implementation is based on the [COMPAS](#) implementation.

Runtime options - processes

Processes - Common Envelope

Related params - “energy option”:

NOTICE

4- Both Xi&Liu10 and Klencki+21 contain fitting equations for lambda for given initial Mass M_{zams} and metallicity. The option -41 and -51 are “quantised”, i.e. the fitting equations for a star at given M_{zams_s} and Z_s are taken from the closer tracks (in the Xu&Li10 or Klencki21 tables) for which $M_{\text{zams}} > M_{\text{zams}_s}$ and $Z > Z_s$

5- In the option -4 and -5 the lambda values are estimated only for the tracks in the Xiu&Li and Klencki tables and then they are interpolated linearly in M_{zams} and Z in the same way [other stellar properties are interpolated in SEVN](#) ([using linear weights](#) for M_{zams})

5- Notice that the Klencki+21 tables cover the ZAMS mass range 10-80 M_{sun} , and the metallicity range 0.01 - 1 Z/Z_{sun} . In order to estimate the lambda for stars with M_{zams} outside the limits, in SEVN we consider the lambda fitting equations of the tracks with the closest M_{zams} and Z

Runtime options - processes

Processes - Common Envelope

Related params - “energy option”:

Lambda ionization `-star_lambda_fth double (def 1)` `CELAMFTH=“1”`

Parameter to consider the thermal/ionization energy in the estimate of lambda. The meaning of this parameter depends of the lambda formalism:

star_lambda > 0: not used

star_lambda = -1,-12,-2,-21,-3,-31,-4,-41: fraction of internal energy that is used in the unbinding of the stellar envelope

star_lambda = -5,-51: not used, the Klencki+21 fitting equations already take into account the internal energy

Lambda pureHe `-star_lambda_pureHe double (def 0.5)` `CELAMHE=“0.5”`

Same as *star_lambda* but for pure-He stars, atm only positive constant values are allowed. In case the chosen *star_lambda* options do not have a prescriptions for pure-He stars, this constant value is used. Atm only the *star_lambda* options -4 and -41 have a prescription for pure-He stars

Runtime options - processes

Processes - Roche Lobe Overflow (RLO)

-rlmode arg **RLMODE="arg"**

arg: 'hurley_rl', 'hurley_rl_propeller', 'hurley_rl_bse', 'disabled'
default: 'hurley_rl'

disabled: disable the process

hurley_rl: Roche-lobe overflow formalism by [Hurley+02](#) but with a modification on the fraction of accreted mass (see the parameter accreted mass fraction)

hurley_rl_propeller: Same as *hurley_rl*, but including the propeller effect to limit the accretion on NS following ([Campana+18](#)). **Notice:** if you use this model it is better to turn on the timestep check on the NS spin (see [here](#))

hurley_rl_bse: Roche-lobe overflow formalism by [Hurley+02](#)

Related params:

Accreted mass fraction **-rlo_f_mass_accreted double (def 0.5)** **RLOMACCR="0.5"**
Fraction of the mass loss during the RLO that is accreted on the companion, available only for *hurley_rl*

Eddington factor **-eddingtion_factor double (def 1)** **RLOEDD="1"**
The accretion on compact objects is limited to this fraction of the Eddington accretion limit. Notice this is used for all the accretion events

Runtime options - processes

Processes - Roche Lobe Overflow (RLO)

Related params:

Ang. Momentum loss `-rlo_gamma_angmom` (def -1) RLOGAM="-1"

Fraction of angular momentum loss from the system during the RLO, special cases:

if -1 the angular momentum is lost from the donor

if -2 the angular momentum is lost from the accretor

Quasi Homogeneous evolution `-rlo_QHE` bool (def false) SQHE="false"

Enable the quasi-homogeneous evolution after accreting mass during the RLO (other conditions have to be fulfilled, see [Eldridge&Stanway2012](#), Sec. 2.2) The stars that are flagged as QHE freeze the evolution of the radius during the Main Sequence, then they become pureHE stars after the Main Sequence.

Enable/Disable Collision during RLO

`-rlo_enable_collision` bool (def false) RLOCOLLISION="false"

If true enable the process collision (collision at periastron) during a RLO

Enable/Disable stable mass transfer for MS donors

`-rlo_mtstable_ms` bool (def true) RLOSMTMS="true"

If true the RLO mass transfer is always stable for Main Sequence donors

Runtime options - processes

Processes - Roche Lobe Overflow (RLO)

Related params:

Mass transfer stability (*qcrit*)

RLOSTABILITY="qcrit_Hradiative_stable"

-rlo_stability string or double (def *qcrit_Hradiative_stable*)

In the *hurley_rl* formalism the RLO mass transfer is stable if the stars mass ratio (*q*) is smaller than a given value *qcrit*. The *qcrit* depends on the stellar type of the donor and, for some prescription, of the accretor. The available options in SEVN are:

- *stable*: the mass transfer is assumed always stable (but special conditions in the RLO formalism can trigger anyway a CE, e.g. double RLO in the Hurley formalism)
- *unstable*: the mass transfer is assumed always unstable
- *qcrit_hurley*: *qcrit* from [Hurley+02](#)
- *qcrit_hurley_webbink*: *qc* from [Hurley+02](#) with Webbink1988 formalism for giant donors
- *qcrit_hurley_webbink_shao*: as above + [Shao+21](#) conditions for BH accretor
- *qcrit_cosmic_neijssel*: formalism from [Neijssel19](#) as used in [COSMIC](#)
- *qcrit_cosmic_claeyes*: [Clayes+14](#) formalism as used in [COSMIC](#)
- *qcrit_Hradiative_stable*: same as *qcrit_hurley_webbink*, but always stable for MS and HG donors
- *qcrit_radiative_stable*: same as *qcrit_Hradiative_stable*, but always stable with pure-He donors

Runtime options - processes

Processes - Collision at periastron

We define collision at periastron the case in which the sum of the radii of the two stars larger than the stellar separation at the periastron $R_1 + R_2 > (1-e)a$

-collmode arg **COLLMODE="arg"**

arg: '*hurley*' or '*disabled*'

default: '*hurley*'

disabled: disable the process

hurley: collision formalism by [Hurley+02](#). A collision can produce a Common Envelope or a direct Merger depending on the stellar phases

Runtime options - processes

Processes - Circularisation

Force the orbit circularisation when some conditions are satisfied, usually the onset of the RLO

-circmode arg

CIRCMODE="arg"

arg: '*periastron*', '*periastron_full*', '*angmom*', '*semimajor*', or '*disabled*'

default: '*angmom*'

disabled: disable the process

RLO-onset options: the following options circularise the system at the onset of the RLO

periastron: circularise at the periastron: $a_{\text{new}}=a_{\text{old}}(1-e_{\text{old}})$, $e_{\text{new}}=0$

angmom: circularise conserving the binary angular momentum
 $a_{\text{new}}=a_{\text{old}}(1-e_{\text{old}}^2)$, $e_{\text{new}}=0$

semimajor: circularise at constant semimajor axis $a_{\text{new}}=a_{\text{old}}$, $e_{\text{new}}=0$

Runtime options - processes

Processes - Circularisation

Force the orbit circularisation when some conditions are satisfied, usually the onset of the RLO

-circmode arg

CIRCMODE="arg"

arg: '*periastron*', '*periastron_full*', '*angmom*', '*semimajor*', or '*disabled*'

default: '*periastron*'

If activated ($\neq$ disabled) force the orbit circularisation when some conditions are satisfied, usually the onset of the RLO

Other options:

periastron_full: same as *periastron*, but the circularisation is triggered when the RLO condition is satisfied at the periastron.

In practice, it is activated when the stellar radius is larger than the $RL_periastron$ estimated as $r_periastron/a * RL_eggleton$.

Runtime options - processes

Processes - Tides

-tmode arg **TIDES="arg"**

arg: *'tides_simple'* or *'disabled'* **default:** *'tides_simple'*

disabled: disable the process

tides_simple: Equilibrium tides by [Hut1981](#) (see also [Hurley+02](#))

Processes - Gravitational Wave (GW)

-gwmode arg **GWMODE="arg"**

arg: *'peters'* or *'disabled'* **default:** *'peters'*

disabled: disable the process

peters: GW orbital decay by [Peters1964](#)

Related params:

GW activation **-gw_tshold double (def 1)** **GWTSHOLD="1"**

Enable the GW process only if GW time scale < gw_tshold*Hubble_time

GW only in BCO **-gw_onlybco bool (def false)** **GWONLYBCO="false"**

If true activate the GW orbital decay only for binary compact objects

Runtime options - processes

Processes - Hardening

Binary hardening due to external perturbers assuming the binaries are living in a cluster

-hardmode arg **HARDMODE="arg"**

arg: '*fastcluster*' or '*disabled*'

default: '*fastcluster*'

disabled: disable the process

fastcluster: hardening formalism by [Mapelli+21](#)

Related params:

Kappa

-hard_kappa (def 0.1)

HARDKAPPA="0.1"

used with option *fastcluster*: kappa parameter in Eq. 13 in [Mapelli+21](#)

Mass_average stars

-hard_mass_average (def 1)

HARDMASS="1"

Average mass of the perturbers in the clusters M_{sun}

Central density

-hard_rhoc (def 39900)

HARDRHOC="39900"

Central density of the cluster in $M_{\text{sun}}/\text{pc}^3$

Velocity dispersion

-hard_sigma (def 5)

HARDSIGMA="5"

Velocity dispersion of the cluster in km/s

Xi

-hard_xi (def 3)

HARDXI="5"

used with option *fastcluster*: xi parameter in Eq. 11 in [Mapelli+21](#)

Runtime options - optional tables

Tabuse - Inertia

Enable or disable the tables for stellar inertia

-tabuse_inertia false

TABINERTIA="false"

In case the tables are disabled, the estimate of the stellar inertia depends on the parameter inertia mode.

Notice If the inertia table are not present in the used stellar tracks this option must be set to false, otherwise SEVN will return an error

Related params:

Inertia model

-inertiamode (def "Hurley")

INERTIAMODE="Hurley"

hsphere: inertia of a uniform sphere $I = \frac{2}{5}MR^2$

hspherecore: envelope -> hollow uniform sphere + core -> uniform sphere

$$I = \frac{2}{5}M_{\text{envelope}} \frac{R^5 - R_{\text{core}}^5}{R^3 - R_{\text{core}}^3} + \frac{2}{5}M_{\text{core}}R_{\text{core}}^2$$

Hurley: similar to hspherecore, but tuning the factors to consider the density profile of a polytrope ([Hurley+02](#))

$$I = k_2 M_{\text{envelope}} R^2 + k_3 M_{\text{core}} R_{\text{core}}^2 \quad \text{with } k_2 = 0.1 \text{ and } k_3 = 0.21$$

DeMink: same as Hurley, but with factor k2 that depends on the stellar properties [DeMink+13](#) (Appendix A, A1,A2,A3) and k3=0

Runtime options - optional tables

Tabuse - RHE

Enable or disable the tables for the radius of the He core

-tabuse_rhe true

TABRHE="true"

Notice If the rhe table is not present in the used stellar tracks this option must be set to false, otherwise SEVN will return an error

In case the tables are disabled, the He core radius is estimated as

$$R_{\text{core,He}} = 0.1075 \frac{1.1685 M_{\text{core,He}}^{4.6}}{M_{\text{core,He}}^4 + 0.162 M_{\text{core,He}}^3 + 0.0065}$$

The fitting equation is based on [Hurley+00](#), with the parameters tuned on the parsec stellar tracks

Runtime options - optional tables

Tabuse - RCO

Enable or disable the tables for the radius of the CO core

-tabuse_rco true

TABRCO="true"

Notice If the rco tables are not present in the used stellar tracks this option must be set to false, otherwise SEVN will return an error

If the tables are disabled, the CO core radius is estimated as

$$R_{\text{core,CO}} = 0.0415 \frac{1.1685 M_{\text{core,CO}}^{4.6}}{M_{\text{core,CO}}^4 + 0.162 M_{\text{core,CO}}^3 + 0.0065}$$

The fitting equation is based on [Hurley+00](#), with the parameters tuned on the parsec stellar tracks

Runtime options - optional tables

Tabuse - Convective envelope

Enable or disable the tables regarding the convective envelope:

- *tconv.dat*, turnover time of the most superficial convective cells
- *depthconv.dat*, depth of the most superficial convective zone (from the surface)
- *qconv.dat*, mass fraction of the most superficial convective zone

-tabuse_envconv true

TABCONV="true"

Notice If at least one of the convective tables is not present in the used stellar tracks this option must be set to false, otherwise SEVN will return an error

If the tables are disabled, the *depth con* and *qconv* are estimated following [Hurley+02](#) (Eq. 36) and [Hurley+00](#) (Section 7.2)

The properties of the convective envelope are used to estimate the BSE Phases, especially the [HG phase](#). If the convective tables disabled, it is better to enable the estimate of the HG time following [Hurley+02](#):

HG time

-use_thg_hurley (def "false")

THGHURLEY="false"

Estimate the end of the HG phase following the fitting formulas in [Hurley+00](#)

Runtime options - optional tables

Tabuse - Chemical surface abundance

Enable or disable the tables regarding the chemical surface abundances:

- *hsup.dat*, Hydrogen surface abundance
- *hesup.dat*, Helium surface abundance
- *csup.dat*, Carbon surface abundance
- *nsup.dat*, Nitrogen surface abundance
- *osup.dat*, Oxygen surface abundance

-tabuse_Xsup false

TABXSUP="false"

Notice If at least one of the surface abundance tables is not present in the used stellar tracks this option must be set to false, otherwise SEVN will return an error

If the tables are disabled all the surface abundance properties (Hsup, HEsup, Csup, Nsup, Osup) are set to 0

Runtime options

Change of track (see [here](#))

Naked threshold `-ev_naked_tshold double (def 1E-4)` `NAKEDTS="1E-4"`

When a star has a mass difference between the $Mass$ and MHE lower than this parameter (M_{sun}), the stars becomes a pure-HE jumping in a new track. If the star is already a pureHE and MHE and MCO are within this parameter, the star becomes a nakedCO stopping its evolution.

Turn to pureHe `-turn_WR_to_pureHe bool (def false)` `WRTOPHE="false"`

If this option is set to true, when a star is flagged as Wolf-Rayet (see later), the star is turned into a pureHe with mass equal to the total mass of the star disregarding the rest of the Hydrogen stellar evolution track.

Notice: Up to the SEVN Version 2.8.0 this parameter did not exist and all the stars flagged as WR were immediately turned into a pureHe.

WR threshold `-star_tshold_WR_envelope double (def 0.021)` `WRTS="0.021"`

It sets the threshold of mass fraction of Hydrogen envelope over the total mass of the star required to be flagged as Wolf-Rayet. Stars with $(M-MHE)/M$ lower than this value are flagged as WR.

Runtime options

Change of track (see [here](#))

Jump trigger threshold error

`-jtrack_tshold_dm_rel double (def 0.01)` `JTDMTSHOLD="0.01"`

Minimum relative change in mass to trigger the change of track. E.g. if a star accretes more than 1% of its mass, a change of track is triggered.

Jump max error

`-ev_naked_tshold double (def 1E-4)` `JTEMAX="5E-3"`

Maximum relative error to reach [convergence](#) when jumping on a new track. If convergence is not reached the track given the best match (among the tested ones) is chosen. Notice: the best track can be also the current track.

Jump max iterations `-jtrack_max_iteration integer (def 0.01)` `JTMAXITER="10"`

Maximum number of iterations to perform searching for the best match. If this limit is reached, the track that gives the best match (among the tested ones) is chosen. Notice: the best track can be the current track.

Runtime options

BH spin

BH spin option

`-xspinmode str (def disabled)`

`BHXSPIN="disabled"`

Set the [*BH adimensional spin*](#) Xspin when the BH is formed, option:

disabled : do not set, Xspin=nan

geneva : set Xspin using Eq. 3 in [*Belczynski+20*](#) (based on Geneva models)

fuller : set Xspin=0.01 (Eq. 5 in [*Belczynski+20*](#))

mesa : set Xspin using Eq. 4 in [*Belczynski+20*](#) (based on MESA models). Notice, this model has been estimate for $M_{\text{CO}} < 70 M_{\text{sun}}$, for larger masses the equation can return negative values. In these cases we set the Xspin to 0

zeros : set Xspin=0.

accretion : Xspin is 0 at the BH creation and can increase through mass accretion following Eq. 4 in [*Zevin&Bavera22*](#)

maxwellian : Xspin is drawn from a Maxwellian with sigma set by the parameter *xspin_sigma_maxwell* (see below).

BH spin maxwellian

`-xspin_sigma_maxwell double (def 0.1)`

`MAXWSDXSPIN="0.1"`

Parameter of the Maxwellian used to draw the Xspin if *xspinmode=maxwellian*

Bavera correction

`-xspin_bavera bool (def false)`

`BAVERAXSPIN="false"`

If true, apply the BH spin up described in [*Bavera21*](#)

Runtime options

NS/Pulsar

When a NS is formed, the magnitude of the magnetic field is drawn from a uniform distribution between $1e10$ Gauss and $1e13$ Gauss.

The initial rotational period is drawn from a uniform distribution between 0.01 s and 0.1 s.

The NS then evolve spinning down and decreasing its magnetic field

Magnetic decay mass-scale

`-ns_magnetic_mscale double (def 0.15)` `NSMAGMSCALE="0.15"`

Mass scale of the exponential Magnetic field decay due to mass accretion [Msun]

Magnetic decay time-scale

`-ns_magnetic_tscale double (def 1000)` `NSMAGTSCALE="1000"`

Time scale of the exponential Magnetic field decay [Myr]

Activate detailed handling of mass accretion on a NS

`-ns_detailed_accretion bool (def false)` `NSDETACC="false"`

If True handle the mass accretion on a NS accounting for the change of spin and magnetic field (see [Sgalletta+23](#))

Notice: If in your study you particular care about such properties, remember to switch on the timestep control on Neutron Star spins (see [here](#) and [here](#))

Runtime options

Study focusing on NS/Pulsar

If you want to study in details Neutron Stars/Pulsars including the binary evolution, please set the following runtime parameters

-ns_detailed_accretion true

NSDETACC="true"

To activate the detailed accretion mode on NS accounting for change in Spin and magnetic field (see [here](#))

-rlmode hurley_rl_propeller

RLMODE="hurley_rl_propeller"

To take into account the propeller effect that could drastically reduce the amount of mass accreted in a NS ([Campana+18](#), see [here](#))

-ts_check_spin_bse true

TSSPINBIN="true"

To include in the adaptive timestep the variation of the spin of the NS (see [here](#))

Notice: The run script *run_ns_pulsar.sh* in the folder *run_scripts* has these variable already set

Runtime options

Interpolation weights

The interpolation weights are used to interpolate the stellar properties.

The stellar properties are divided in two groups: linear, log.

The log group contains all the properties that are internally stored in log, i.e. the luminosity, the radius, the inertia. All the other stellar properties are in the linear group. In addition to the stellar properties, there are other values that are interpolated but not saved as a proper stellar property. These are the times that define the boundary between different [SEVN phases](#).

For each of three groups, the user can select which one of the available interpolator weights (linear, log rational, [more details here](#)) are used:

Linear properties `-ev_setwM str (def linear)` `INTW="linear"`

Set the interpolation weights for the linear properties

Log properties `-ev_setwM_log str (def log)` `INTWLOG="log"`

Set the interpolation weights for the log properties

Phase times `-ev_setwM_tphase str (def rational)` `INTWPHASE="rational"`

Set the interpolation weights for the phase times

Runtime options

Other interpolation options

Force linear interpolation for the Radius

`-ev_R_linear_interp bool (def false)`

`FORCERLIN="false"`

If true force the interpolation of the Radius to use linear values transforming the log values stored in SEVN. If this is true the radius will be used the weights options set for for the linear quantities (ev_setwM).

Runtime options

Timestep control

The following parameters control the SEVN adaptive timestep. We suggest to leave these parameter set to the default values

Maximum variation `-ts_maximum_variation double (def 0.05)` `TSMAXVAR="0.05"`

Maximum relative variation for a given set of stellar and binary properties allowed in a single timestep. This value will be used both to predict the new timestep and to repeat the last evolution step if the variation of at least one of the checked properties is larger than the maximum.

Minimum dt `-ts_min_dt double (def -1)` `TSMIN="-1"`

Minimum value allowed for the timestep. The adaptive timestep is forced to be larger than this value. Notice, there are special evolution cases (e.g. merger) that are handled through arbitrary small timesteps ($dt=1e-15$). These are the only cases in which the timestep is allowed to be smaller than this limit. If the parameter is set to -1, no limits are used

Maximum dt `-ts_max_dt double (def -1)` `TSMAX="-1"`

Same as minimum dt but for the maximum allowed dt

Check stalling `-check_stalling bool (def true)` `CKSTALLING="true"`

If true check stalling stars: if a stellar or binary evolution is running for more than 20s, SEVN will throw an error

Check stalling time `-check_stalling_time (def 5)` `CKSTALLINGTIME="true"`

Time in seconds (units granularity) after which the system is considered stalled and the evolution is stopped and an error is raised)

Runtime options

Timestep control

The following parameters control the SEVN adaptive timestep. We suggest to leave these parameter set to the default values

Neutron star spin `-ts_check_NSspin bool (def false)` `TSNSSPIN="false"`

The variation of the property OmegaRem (angular velocity of the NS) is not check by default in the adaptive timestep. Setting this property to true include this property in the timestep check&repeat. **This is necessary if one wants to study how the accretion process can change the period and magnetic fields of the neutron stars.**

Stellar spin SSE `-ts_check_spin bool (def false)` `TSSPIN="false"`

The variation of the stellar angular velocity is not check by default in the adaptive timestep. Setting this property to true force the adaptive timestep to check the variation in the stellar angular velocity due to single stellar evolution processes (winds, angular momentum conservation).

Stellar spin BSE `-ts_check_spin_bse bool (def false)` `TSSPINBIN="false"`

The variation of the stellar angular velocity is not check by default in the adaptive timestep. Setting this property to true force the adaptive timestep to check the variation in the stellar angular velocity due to binary evolution processes (e.g. tides, mass accretion).

Runtime options

Timestep control - Stellar spin



NOTICE:

When the adaptive time step for the Spin is activated the stellar rotation is checked using two properties: Spin (that is the angular velocity over the critical angular velocity) and the stellar angular momentum of the star OmegaSpin (see [here](#)).

SSE: for the single stellar evolution the variation of OmegaSpin is checked, but only if the initial OmegaSpin is larger than $1 \text{ Msun Rsun}^2 \text{ yr}^{-1}$.

BSE: if $\text{Spin} < 0.1$, a step is accepted only if the spin variation is less than 0.01 and the new timestep are estimated assuming a maximum absolute variation of 0.01. In all the other cases the check of OmegaSpin is used if larger than $1 \text{ Msun Rsun}^2 \text{ yr}^{-1}$.

Hints and tips

Detaching SEVN from the terminal

In order to run a long SEVN run or run SEVN in a server, it is preferable to run SEVN in background

```
nohup ./run.sh <path_to_input_list> >out.log 2>err.log & disown
```

The file *out.log* contains the standard SEVN message in output, while the file *err.log* contains all the Warning and Error messages.

Hints and tips

Monitoring the SEVN run

A standard SEVN runs write in output a counter of evolved and failed stars updated each [Nchunk](#) of processed stars:

```
4000020 loaded stars!  
Evolving systems:  
200000/4000020 (Nfailed:0)
```

A more “thin grained” check can be obtained looking at the number or rows present in the evolved files (or in the error files) inside the SEVN output folder:

```
wc -l evolved*
```

```
wc -l failed*
```

The command(s) will return the number of systems (one per row) evolved (or failed) so far in each thread and their total sum.

```
(base) 🌊 🦊 wc -l evolved_*  
571 evolved_0.dat  
561 evolved_1.dat  
1132 total
```

Sevn outputs



Output files - overview

```
(base) iogiul@Giulianos-MBP SEVN_run % cd my_output
```

```
(base) iogiul@Giulianos-MBP my_output % ls
```

```
evolved_0.dat
```

```
logfile_0.dat
```

```
failed_initialisation_0.dat
```

```
output_0.csv
```

```
launch_line.txt
```

```
used_params.svpar
```

sevnB.x

Used executable

A SEVN run produces several output files:

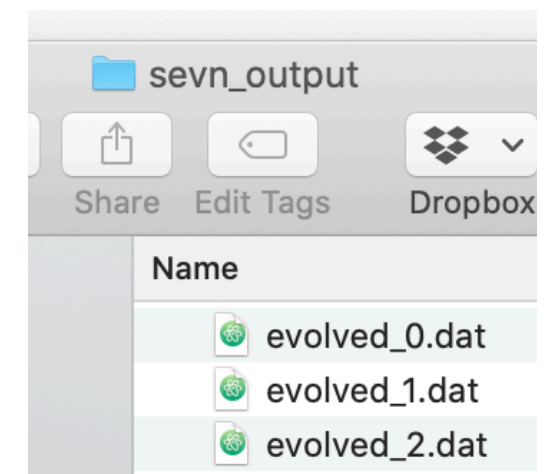
- **Evolved file(s):** *evolved_xxx.dat*,
initial properties (input) of the systems that has been successfully evolved
- **Not evolved file(s):** *failed_initialisation_xxx.dat* and *failed_xxx.dat*,
initial properties of the systems for which the simulation has been halted due to some error at system initialisation (failed_initialisation) or later on (failed).
- **Output file(s):** *output_xxx.csv* (the extension depends on the runtime option omode), proper output of the simulation
- **Log file(s):** *logfile_xxx.dat*,
info on the SSE/BSE evolution events, complementary to the output files
- **Summary of the used parameters:** *used_params.svpar*,
complete list of SEVN parameters used in the simulation
- **Launch line** (only present if the run script has been used): *launch_line.txt*,
contains the run command actually produced by the run script

Output files - overview

evolved_xxx.dat, *output_xxx.csv*, *logfile_xxx.dat*, *failed_xxx.dat*

What does xxx mean?

- SEVN exploits cpu-parallelism through openMP multithreading
- Each tread produces its own outputs
- The suffix **xxx** indicates the **thread number**
- The thread number (starting from 0) is printed without leading or trailing zeros
- **Notice:**
SEVN initialises the star/binary outside the parallel part of the code. Therefore, all the systems that trigger a failed initialisation are stored in one single file *failed_0.dat*



Evolved and failed files:

evolved_xxx.dat, *failed_initialisation_xxx.dat*, *failed_xxx.dat*

Format

- Formatted ascii files
- **Values are separated by white spaces**
- The **first row contains the header**. It starts with the character “#” and contains the column names separate by white spaces.

Content

- Input information of all the evolved (evolved_xxx), failed during evolution (failed_xxx) and failed on initialisation (failed_initialisation_xxx) systems with the addition of unique integers, *ID*, *name* and *Seed*.
- Each row refers to a given system.

evolved_0.dat						
1	#ID	name	Mass_0	Mass_1	Z_0	Z_1
2	0	438060358735771	30	2.2	0.004	0.004
3	1	438060358735771	9.013	2.2	0.004	0.004
4	2	438060358735771	9.014	2.2	0.004	0.004

Evolved and failed files:

evolved_xxx.dat, failed_initialisation_xxx.dat, failed_xxx.dat

Content - Columns - SSE

- 1-**ID** [long int]: sequential integer, unique for each binary, it indicates the position of the system in the input file
- 2-**name** [long int]: random integer, unique for each binary
- 3-**Mass** [double]: zams mass of the star in Msun
- 4-**Z** [double]: metallicity
- 5-**spin** [double]: initial spin of the star
- 6-**SN** [double]: [supernova model](#) for the star
- 7-**Tstart** [str]: [initial age](#) of the star
- 8-**Tend** [str]: [ending time](#) of the simulation
- 9-**Dtout** [str]: [time step for output](#)
- 10-**Seed** [long int]: [seed for random number generation](#) [long int]

Evolved and failed files:

evolved_xxx.dat, failed_initialisation_xxx.dat, failed_xxx.dat

Content - Columns - BSE

- 1-**ID** [long int]: sequential integer, unique for each binary, it indicates the position of the system in the input file
- 2-**name** [long int]: random integer, unique for each binary
- 3-**Mass_0** [double]: zams mass of the first star in Msun
- 4-**Z_0** [double]: metallicity the first star
- 5-**spin_0** [double]: initial spin of the first star
- 6-**SN_0** [double]: [supernova model](#) for the first star
- 7-**Tstart_0** [str]: [initial age](#) of the first star
- 8-**Mass_1** [double]: same as Mass_0 but for the second star
- 9-**Z_1** [double]: metallicity the second star
- 10-**spin_1** [double]: same as spin_0 but for the second star
- 11-**SN_1** [double]: same as SN_0 but for the second star
- 12-**Tstart_1** [double]: same as Tstart_0 but for the second star
- 13-**a** [double]: initial semimajor axis of the binary [double, Rsun]
- 14-**e** [double]: initial eccentricity of the binary [double]
- 15-**Tend** [str]: [ending time](#) of the simulation
- 16-**Dtout** [str]: [time step](#) for output
- 17-**Seed** [long int]: [seed for random number generation](#) [long int]

Output files: *output_xxx.csv*



Notice: this guide assume that the output format is csv, other options are possible ([see here](#)). The ascii format is similar to the csv except for the column delimiter, it uses white spaces instead of commas.

Format

- Comma Separated Variables ascii files **Values are separated by commas** without extra spaces (e.g. 1,2,hello)
- The **first row contains the header** with columns names separated by commas

output_0.csv

```
1 ID,name,Worldtime_0,Localtime_0,Mass_0,MHE_0,MCO_0,Lumino
2 0,438060358735771,0.000000e+00,7.195606e-02,6.993543e+01,
3 0,438060358735771,3.465333e-01,4.184893e-01,6.959649e+01,
4 0,438060358735771,5.836260e-01,6.555820e-01,6.933650e+01,
```

Output files: *output_xxx.csv*

Content

- Stellar and binary properties (set using the runtime options *scol* and *bcol*, [see here](#)) at the chosen times (set using the list or runtime option *dtout*, [see here](#) and [here](#)).
- Each system is identified by a sequential integer *ID* (original position in the input file) and a unique long integer (*name*). These two values are always the first two columns of the output files.
- The order of the other columns in output depends on the order given in the runtime options *scol* and *bcol*.
- For binary systems the stellar properties (*scol*) are printed before the binary properties (*bcol*). The name of the stellar properties columns have a suffix *_0* or *_1* for the primary and secondary star, respectively.

Output files: *output_xxx.csv*

Content - example

- Assume we set the runtime options:

- *-scol Mass:Radius:MHE*
- *-bcol Semimajor:Eccentricity:BWorlrdtime*

- The columns in output are:

col0-ID: ID of the systems (correspond to the position in the input file)

col1-Name: unique identifier of the system

col2-Mass_0: total mass of the first star

col3-Radius_0: radius of the first star

col4-MHE_0: mass of the He-core of the first star

col5-Mass_1: total mass of the second star

col6-Radius_1: total radius of the second star

col7-MHE_1: mass of the He-core of the second star

col8-Semimajor: semimajor axis of the binary

col9-Eccentricity: eccentricity of the binary

col10-BWolrdtime: time elapsed in the simulations (starting always from 0)

Output files: *output_xxx.csv*

Content - notes

- Outputs can contain *nan* values, they represent undefined values
 - The following columns are **never set to nan**:
Worldtime, *Timestep*, *Phase*, *RemnantType*, *BWorldtime*, *BTimestep*, *Zams*, *Event*.
 - **If a star does not exist anymore** (because of a stellar merger or after a SN leaving no remnants), the stellar properties are set to *nan*. In this case *Phase*=7 and *RemnantType*=-1. Notice, if at least a star is empty, the binary is considered broken (see below)
 - **If the binary does not exist anymore** (because it is broken or after a stellar merger), the binary parameters are set to *nan*.

Output files: *output_xxx.csv*

Content - notes

- **Timestep:** SEVN uses an [adaptive timestep schema](#), the timestep differs from system to system and from step to step considering the same system. In the output, the properties *Timestep* and *BTimestep* indicate the time elapsed in the current timestep.
- **Time properties:** The stellar properties *Worldtime* and *Timestep* are always synched with the binary properties *Bworldtime* and *BTimestep*. Therefore in each step :
 $Worldtime_0 = Worldtime_1 = BWorldtime$ and
 $Timestep_0 = Timestep_1 = BTimestep$
Printing in output both of them is redundant
- **Time resolution:** in some cases the timestep is smaller than the output time resolution for the property worldtime. In those cases the columns *BWorldtime* and *Worldtime* could have the same value in two consecutive rows. If this happens, the combination of *Worldtime* (*BWorldtime*) and *Timestep* (*BTimestep*) can be used to have a correct time tracking.

Log files: *logfile_xxx.dat*

Format

- “Partially” formatted ascii file.
- Each row contains an **header** part with a fixed number of values (4) + an **info** part with variable number of values.
- The **header** values are separated by “;”
- The **info** values are separated by “:”

Content

- Information about the binary and stellar evolution, complementary to the output files.
- Each row reports a particular event happened in a given star or binary system

```
B;136641911126985;3;MERGER;28.340739;0:8.472e+00:1.530e+00:0.000e+00:3:0:1:6.255e+00:0.000e+00:0.000e+00:1:0:14.727  
S;136641911126985;0;SN;30.891598;13.2745:1.92026:1.49299:1.23552:5:507.658:643.773:-181.211:305.113:537.134
```

Log files: *logfile_xxx.dat*

Content - how to read it: Header

HEADER The first part of each row is the header:
Object;name;ID;event;time;

- ***Object***: **S** indicates a single star, **B** indicates a binary
- ***name***: object name (long int), notice the stars have the same name of the binary they belong to. It can be used to link the systems to the output and evolved files
- ***ID***: id of the object (long int). If the object is a star the id is 0 if it is the first star or 1 if it is the second one. It can be used to link the systems to the output and evolved files.
- ***event***: a label in capital letter indicating the triggering event. All the possible events are listed in the [next slide](#)
- ***time***: time of the simulation (Myr) when the event is triggered.

B;621064598874129;13;CE;3.348641;

A Common envelope (*event*=CE) is triggered in the binary with *name* 621064598874129 and *ID* 13 at *time* 3.348641 Myr.

Log files: *logfile_xxx.dat*

Content - events list

Events list - Star:

- **SN:** Supernova explosion (star)
- **QHE:** Quasi Homogeneous Evolution
- **WD:** White Dwarf formation
- **HENAKED:** He naked formation
- **CONAKED:** CO naked formation

Events list - Binary:

- **BSN:** Supernova explosion (binary)
- **RLO_BEGIN:** Roche Lobe Overflow begins
- **RLO_END:** Roche Lobe Overflow ends
- **CIRC:** Orbit circularisation at the onset of RLO
- **CE:** Common Envelope
- **COLLISION:** Collision
- **MERGER:** Merger
- **SWALLOWED:** Swallowed star

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Star: Supernova explosion

Header label: **SN**

Trigger: A star explodes as SN

FORMAT: <i>d1:d2:d3:d4:i5:i6:d7:d8:d9:d10:d11</i>

- *d1*: total mass before the SN explosion in Msun
- *d2*: He core mass before the SN explosion in Msun
- *d3*: CO core mass before the SN explosion in Msun
- *d4*: mass of the SN remnant in Msun
- *i5*: remnant type (integer, [remnant type](#))
- *i6*: SN type (integer, see [sn_type](#))
- *d7*: natal kick, i.e. module of the random sampled kick velocity in km/s
- *d8*: actual module of the velocity after possible corrections in km/s
- *d9*: x-component of the corrected kick velocity in km/s
- *d10*: y-component of the corrected kick velocity in km/s
- *d11*: z-component of the corrected kick velocity in km/s

S;101931734175490;0;SN;11.84;3.53:3.53:2.17:1.23:5:2:245.96:59.45:0.56:-38.22:45.53

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Star: Supernova explosion - SN type

SN type Int	SN type str	Physics
0	Unknown	Default initialiser. It means that no SN type has been associated to the explosion. In general, a SN type is set at every explosion, therefore a value of 0 means that something went wrong.
1	Electron Capture SN	Electron capture Neutrostar
2	Core Collapse SN	Core collapse explosion
3	Pulsation Pair Instability SN (PPISN)	PPISN
4	Pair Instability SN (PISN)	PISN
5	Ia	WD explosion

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Star: White dwarf formation

Header label: QHE

Trigger: a stars starts to follow the Quasi-Homogeneous evolution

FORMAT: *d1:d2:d3*

- *d1*: total mass of the Star at the beginning of the QHE [Msun]
- *d2*: stellar radius at the beginning of the QHE [Rsun]
- *d3*: amount of mass accumulated during the ongoing RLO (if any) [Msun]

Notice: after the activation of the QHE, if the star is in the MS, the radial evolution is frozen. After the MS phase the star is turned into a pure-He star

Example:

S;101931734175490;1;QHE;4.3078;36.5701:13.0271:2.03386

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Star: White dwarf formation

Header label: **WD**

Trigger: A star becomes a White dwarf

FORMAT: *d1:d2:d3:d4:i5*

- *d1*: total mass before the WD formation
- *d2*: He core mass before the WD formation
- *d3*: CO core mass before the WD formation
- *d4*: mass of the WD in Msun
- *i5*: remnant type (integer, [remnant type](#))

Example:

S;101931734175490;1;WD;207.419314;0.542482:0.542482:0.483104:0.542482:1

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Star: Neutron star formation

Header label: NS

Trigger: A star becomes a Neutron star

FORMAT: *i1:d2:d3:d4:d5*

- *i1*: remnant type (integer, [remnant type](#))
- *d2*: mass of the SN remnant in Msun
- *d3*: Initial magnetic field (Gauss)
- *d4*: Initial angular velocity (1/s)
- *d5*: sin alpha, angle between magnetic axis and rotation axis

Example:

S;101931734175490;27;NS;1.183621e+01;5:1.44057:5.76188e+12:249.379:0.843977

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Star: Helium naked formation

Header label: **HENAKED**

Trigger: A star loses its hydrogen envelope turning into a pure-He star.

FORMAT: <i>d1:d2:d3:d4:d5:i6:d7:d8:i9</i>

- *d1*: total mass before the event in Msun
- *d2*: He core mass before event in Msun
- *d3*: CO core mass before event in Msun
- *d4*: radius of the star before the event in Rsun
- *d5*: radius of the star after the event in Rsun
- *i6*: current stellar phase (integer [phase type](#))
- *d7*: interpolating track Mzams before the event in Msun
- *d8*: interpolating track Mzams after the event in Msun
- *i9*: outcome of the new track matching: 0- track found and convergence reached, 1-track found convergence not reached 2-track not found

Example:

S;320104657184301;1;HENAKED;3.21;28.49:25.23:0:221.389:2.4119:4:87.0625:25.2393:0

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Star: CO naked formation

Header label: **CONAKED**

Trigger: A star has lost both its Hydrogen envelope and HE portion of the core remaining with a “naked” Carbon Oxygen core. Notice, the star evolves passively until it explodes as SN or it becomes a WD.

FORMAT: *d1:d2:d3:d4:d5:d6:d7:i8*

- *d1*: total mass before the event in Msun
- *d2*: He core mass before event in Msun
- *d3*: CO core mass before event in Msun
- *d4*: radius of the star before the event in Rsun
- *d5*: radius of the CO core before the event in Rsun
- *d6*: total mass after the event in Msun
- *d7*: radius of the star after the event in Rsun
- *d8*: current stellar phase (integer [phase type](#))

Example:

S;108335909573341;0;CONAKED;26.57;1.69:1.69:0.90:11.69:0.027:0.90:0.027:6

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Binary: Supernova explosion

Header label: **BSN**

Trigger: One of the star in the (still bound) binary system explodes as SN or becomes a WD

FORMAT: <i>i1:d2:d3:d4:i5:i6:i7:d8:d9:d10:i11:i12:d13:d14:d15:d16:d17:d18</i>

- *i1*: id of the star exploding as a SN
 - *d2*: mass of the first star before the event in Msun
 - *d3*: HE core mass of the first star before the event in Msun
 - *d4*: CO core mass of the first star before the event in Msun
 - *i5*: phase of the first star before the event (integer [phase type](#))
 - *i6*: remnant type of the first star before the event (integer [remnant type](#))
 - *i7* - *i12*: same as *i1* - *i6* but for the companion star
 - *d13*: semi-major axis length before the event in Rsun
 - *d14*: eccentricity before the event
 - *d15*: semi-major axis length after the event in Rsun.*
 - *d16*: eccentricity after the event in Rsun.*
 - *d17*: cosine of the angle between the normal to the old orbital plane (before the event) and the normal to the new orbital plane (after the event).*
 - *d18*: module of the velocity of the binary center of mass after the event in km/s.*
- * If the binary has been broken by the event, this value is *nan*.

Example:

B;555880334646216;407478;BSN;11.06;0:3.79:3.79:2.38:6:0:1:11.47:0.00:0.00:1:0:228.46:0:160.49:0.42:-0.85:14.72

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Binary: RLO begin

Header label: **RLO_BEGIN**

Trigger: One of the stars in the binary system fills the Roche-Lobe and a Roche-Lobe overflow begins

FORMAT:

i1:d2:d3:d4:i5:i6:d7:d8:i9:d10:d11:d12:i13:i14:d15:d16:d15:d16:d17:d18:d19:d20

- *i1*: id of the star filling the RLO (donor star)
- *d2*: mass of the donor star in Msun
- *d3*: HE core mass of the donor star in Msun
- *d4*: CO core mass of the donor star in Msun
- *i5*: phase of the donor star (integer [phase type](#))
- *i6*: remnant type of the donor (integer [remnant type](#))
- *d7*: radius of the donor in Rsun
- *d8*: RL radius of the donor in Rsun
- *i9* - *d16*: same as *i1* - *d8* but for the other star (accretor)
- *d17*: mass ratio q ($M_{\text{donor}}/M_{\text{accretor}}$)
- *d18*: critical mass ratio q_{crit} , if $q < q_{\text{crit}}$ the mass transfer is stable, otherwise unstable
- *d19*: semi-major axis length in Rsun
- *d20*: Eccentricity

Example:

B;320104657184301;24;RLO_BEGIN;3.21;1:28.58:25.13:0.0:3:0:74.14:71.371:0:53.06:0.0:0.0:7:6:2.17e-04:94.64:0.53:4:218.38:0.004

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Binary: RLO end

Header label: **RLO_END**

Trigger: the RLO started previously comes to an end both because the two stars have merged or because they stop to fill the RL

FORMAT:

i1:d2:d3:d4:i5:i6:d7:d8:i9:d10:d11:d12:i13:i14:d15:d16:d15:d16:d17:d18:d19:d20

- *i1*: id of the star filling the RLO (donor star)
- *d2*: mass of the donor star in Msun
- *d3*: HE core mass of the donor star in Msun
- *d4*: CO core mass of the donor star in Msun
- *i5*: phase of the donor star (integer [phase type](#))
- *i6*: remnant type of the donor (integer [remnant type](#))
- *d7*: radius of the donor in Rsun
- *d8*: RL radius of the donor in Rsun
- *i9* - *d16*: same as *i1* - *d8* but for the other star (accretor)
- *d17*: total mass lost by the donor during the RLO in Msun
- *d18*: total mass accreted by the accretor during the RLO in Msun
- *d19*: semi-major axis length in Rsun
- *d20*: Eccentricity

Example:

B;320104657184301;24;RLO_END;3.21;1;28.49;25.24;0.0;3;0;221.38;71.30;0;53.09;0.0;0.0;7;6;0.0002;94.70;-0.052;0.00015;218.361;0

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Binary: Circularisation

Header label: CIRC

Trigger: The orbit is circularised due to some trigger (usually onset of RLO)

FORMAT: *d1:d2:d3:d4:i5*

- *d1*: semi-major axis before the circularisation (Rsun)
- *d2*: Eccentricity before the circularisation
- *d3*: semi-major axis after the circularisation (Rsun)
- *d4*: Eccentricity after the circularisation

Example:

B;101931734175490;0;CIRC;3.337316;586.249:0.116313:578.318:0

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Binary: Common Envelope

Header label: **CE**

Trigger: a common envelope evolution is triggered due to a RLO unstable mass transfer or to a stellar collision

FORMAT: *i1:d2:d3:d4:i5:i6:i7:d8:d9:d10:i11:i12:d13:d14:i15*

- *i1*: id of the star that starts the CE (primary)
- *d2*: mass of the primary star in Msun before the event
- *d3*: HE core mass of the primary star in Msun before the event
- *d4*: CO core mass of the primary star in Msun before the event
- *i5*: phase of the primary star before the event (integer, [phase type](#))
- *i6*: remnant type of the primary star before the event (integer, [remnant type](#))
- *i7* - *i12*: same as *i1* - *i6* but for the other star (secondary)
- *d13*: semi-major axis length before the event in Rsun
- *d14*: semi-major axis length after the event in Rsun
- *i15*: 0-if the binary survives after the CE, 1-if the stars coalesce

Example:

B;633318987585808;603010;CE;33.25;0:1.43:1.43:0.77:6:0:1:4.44:0.0:0.0:1:0:37.40:7.86:0

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Binary: Collision

Header label: COLLISION

Trigger: The sum of the radii of the two stars is larger than the stellar separation at the periastron ($R1+R2>a(1-e)$)

FORMAT: *i1:d2:d3:i4:i5:d6:d7:i8:d9:d10:d11:d12*

- *i1*: id of the star filling the RL (donor star)
- *d2*: mass of the donor star before the event in Msun
- *d3*: radius of the donor star before the event in Rsun
- *i4*: phase of the donor star (integer [phase type](#))
- *i5-i8*: same as *i1 - i4* but for the other star
- *d9*: semi-major axis length before the in Rsun
- *d10*: Eccentricity before the event
- *d11*: RL radius in Rsun of the donor star before the event (ID=0)
- *d12*: RL radius in Rsun of the accretor star before the event (ID=0)

Example:

B;194272875003868;26;COLLISION;6.09;0:29.71:41.07:4:1:0.5:2.12e-06:7:57.47:0.66:39.98:6.91

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Binary: Merger

Header label: **MERGER**

Trigger: Two stars merge after a collision or a unstable RLO.

FORMAT: *i1:d2:d3:d4:i5:i6:d7:i8:d9:d10:d11:i12:i13:d14:d15:d16:d17*

NOTICE: in the following, the accretor is the star that remains after the merger, the donor is the star that will be set to empty

- *i1*: id of the star that survives after the merger (accretor)
- *d2*: mass of the accretor star in Msun before the event
- *d3*: HE core mass of the accretor star in Msun before the event
- *d4*: CO core mass of the accretor star in Msun before the event
- *i5*: phase of accretor star before the event (integer [phase type](#))
- *i6*: remnant type of accretor star before the event (integer [remnant type](#))
- *d7*: Radius of the accretor before the event
- *i8 - d14*: same as *i1 - d7* but for the other star (donor)
- *d15*: final mass of the merger product
- *d16*: semi-major axis at the onset of the merger
- *d17*: eccentricity at the onset of the merger (if >1, this is an head-on merger)

Example:

B;0;2;MERGER;5.109291;0:3.126e+01:1.302e+01:0.000e+00:3:0:681.172:1:7.748e+00:0.000e+00:0.000e+00:1:0:3.30122:39.0128:1294.95:0.0278763

Log files: *logfile_xxx.dat*

An important caveat - SN (NS, WD) duplicates



During a binary evolution, It could happen (it is rare, ~ 30 every $1e6$ binaries for typical Kroupa and Sana IMF) that in the same step a star explodes as SN, but the other star triggers a repeat due to the adaptive timestep. In this case, the just exploded star is restored and the evolution repeated.

Since the SN and the remnant log information are flushed as soon as a SN is triggered, in the logfile there will be duplicates SN (and NS or WD) events. Only the last is the correct one.

This should be taken into account when dealing with the log files and the SN, NS and WD events.

REMEMBER: One star can have only one SN, (NS, WD) event, if there are duplicates, the last one is the one reporting the correct information

NOTICE: This is not effecting the event BSN since it is printed only after the evolution of the two stars.

Log files: *logfile_xxx.dat*

Content - How to read it: Info

Event - Binary: Swallowed

Header label: SWALLOWED

Trigger: during an unstable RLO the star filling the RL is completely destroyed, e.g. the mass lost is the total stellar mass and neither a merger nor a CE is triggered. Notice: this event can be triggered even inside the CE process if it produces an unstable object formed by a compact remnant surrounded by an envelope that will be ejected.

FORMAT: *i1:d2:d3:d4:i5:i6:i7:d8:d9:d10:i11:i12:d13*

- *i1*: id of the destroyed star
- *d2*: mass of the destroyed star in Msun before the event
- *d3*: HE core mass of the destroyed star in Msun before the event
- *d4*: CO core mass of the destroyed star in Msun before the event
- *i5*: phase of the destroyed star before the event (integer [phase type](#))
- *i6*: remnant type of the destroyed star before the event (integer [remnant type](#))
- *i7* - *i12*: same as *i1* - *i6* but for the other star
- *d13*: total accreted mass

Example:

B;208699110706408;652062;SWALLOWED;23.88;1:1.38:1.380:1.38:6:0:0:1.23:0.00:0.0:7:5:23.88

Summary of used parameters: *used_params.svpar*

Format

- Formatted ascii file
- The first row contains the header “#USED PARAMS”
- The other rows are formatted as:
parameter_name : value //short_description

Content

- This file stores the values of all the SEVN parameters used in the simulation.

```
used_params.svpar
1 #USED PARAMS
2 ce_alpha:      5           //[N] [PS] alpha in binding energy (
3 ce_kce:        1           //[N] [PS] Fraction of non core mass
4 ce_knce:       1           //[N] [PS] Fraction of non core mass
5 ev_Nchunk:    1000         //[N] [PS] Evolve Nchunk at time
```

Launch line: *launch_line.txt*



This file is included in the outputs only if SEVN has been run using the [run script](#).

Format

- Text file containing two parts separated by a semicolon.

Content

- The first part contains info about the date and time of the SEVN execution and the name of users that run it.
- The second part contains the runs command actually executed through the run script

Notice: you can use the second part of this output to re-run exactly the same simulation (but if [rseed](#) is set to false, the stochastic parts of the simulation will differ)

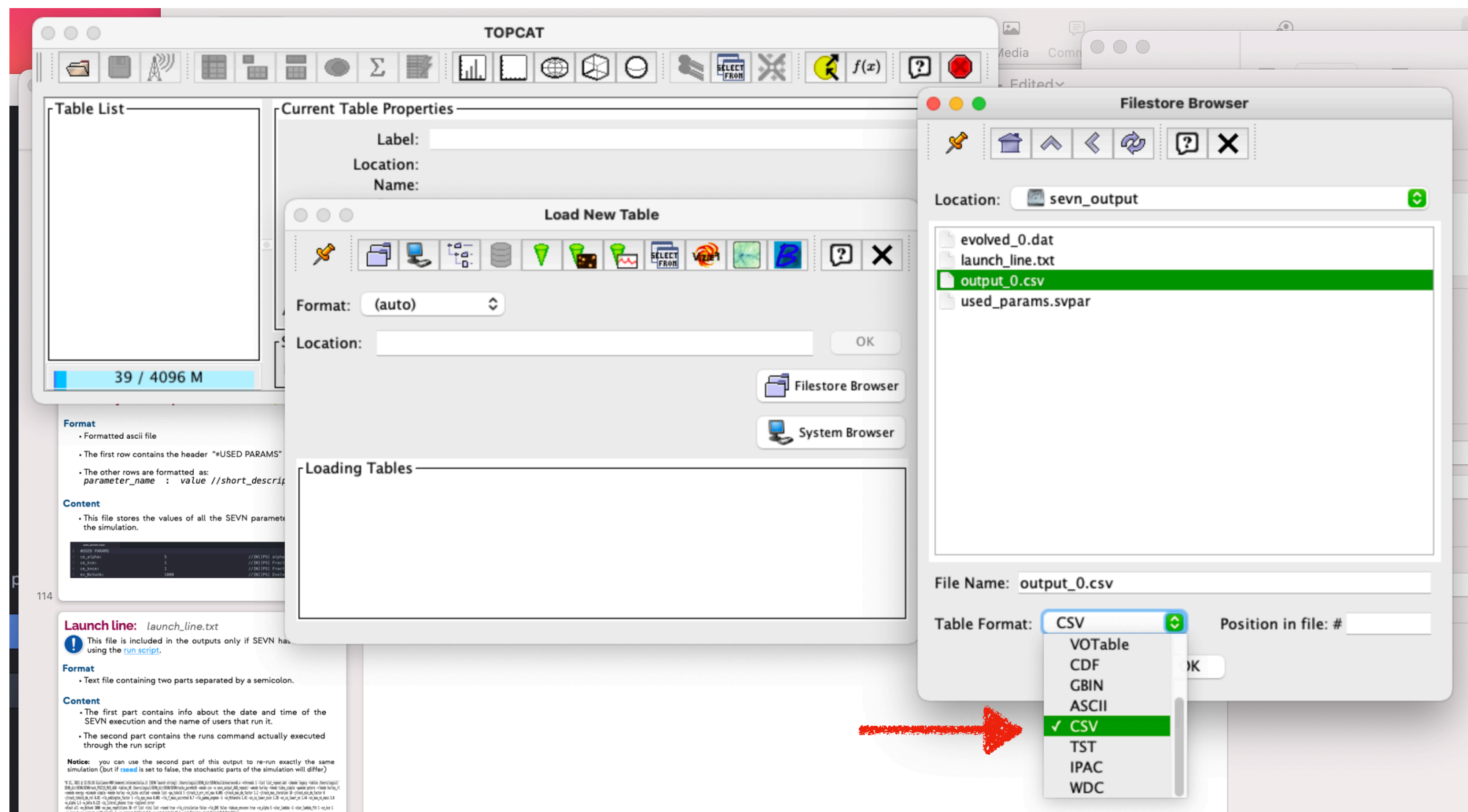
```
^B 21, 2021 @ 12:55:26 Giulianos-MBP.homenet.telecomitalia.it [SEVN launch string]: /Users/iogiul/SEVN_dir/SEVN/build/exe/sevnB.x -nthreads 1 -list list_repeat.dat -ibmode legacy -tables /Users/iogiul/SEVN_dir/SEVN/SEVNtrack_PSIC15_MCO_AGB -tables_HE /Users/iogiul/SEVN_dir/SEVN/SEVNtracks_pureHe36 -omode csv -o sevn_output_AGB_repeat/ -wmode hurley -tmode tides_simple -gwmode peters -rlmode hurley_rl -cemode energy -mixmode simple -kmode hurley -sn_kicks unified -snmode list -gw_tshold 1 -jtrack_h_err_rel_max 0.005 -jtrack_max_dm_factor 1.2 -jtrack_max_iteration 10 -jtrack_min_dm_factor 0 -jtrack_tshold_dm_rel 0.01 -rlo_eddington_factor 1 -rlo_eps_nova 0.001 -rlo_f_mass_accreted 0.7 -rlo_gamma_angmom -1 -sn_Mchandra 1.41 -sn_co_lower_ecsn 1.38 -sn_co_lower_sn 1.44 -sn_max_ns_mass 3.0 -w_alpha 1.5 -w_beta 0.125 -io_literal_phases true -loglevel error -dtout all -ev_Nchunk 1000 -ev_max_repetitions 30 -tf list -tini list -rseed true -rlo_circularise false -rlo_QHE false -tabuse_envconv true -ce_alpha 5 -star_lambda -1 -star_lambda_fth 1 -ce_kce 1 -ce_knce 1 -sn_compact_csi25_tshold -1 -sn_compact_fallback 0.9 -ev_naked_tshold 1E-4 -scol Timestep -bcol Period:RL0:RL1:GWtime
```

Hints and tips



Reading and handling the output files

- The software [TOPCAT](#) can be used to give a first quick look to the outputs. It reads both ascii and csv files, just use the right extension when loading the file



Hints and tips



Reading the output files

- The Python package [PANDAS](#) can read output files saved with the `ascii/csv` and `hdf5` [format](#).

CSV

```
import pandas as pd
df = pd.read_csv("path_to_output_file")
```

ASCII

```
import pandas as pd
df = pd.read_csv("path_to_output_file", sep=r"\s+") //read file
df=df.rename(columns={'#ID':ID}) //rename ID column to remove
the #
```

HDF5

```
import pandas as pd
df = pd.read_hdf("path_to_output_file") //read file
```

The Pandas read functions return a pandas **dataframe**

Hints and tips



Filtering the output files

- The Python package [PANDAS](#) can read output files saved with the `ascii/csv` and `hdf5` [format](#).
- [PANDAS](#) dataframes can be filtered creating boolean list through boolean operations

E.g. filter binary black holes

```
#Dataframe can be filtered using boolean operations on their columns as
done with numpy array,
#For example, let assume we want to retrieve all the BH-BH #binaries.
#Remember that the BHs hav RemnantType=6 and bounded binaries have #not nan
Semimajor

idxBHBH=(df.RemnantType_0==6) & (df.RemnantType_1==6) &
(df.Semimajor.notnull())

#now use the above index to extract a new dataframe of bounde BHBH binaries

dfbhbh=df[idxBHBH]
```

Hints and tips



Reading the evolved files

- [PANDAS](#) can also read the evolved (and failed) files

```
import pandas as pd
df = pd.read_csv("path_to_evolve_file", delimiter=r"\s+")
df = df.rename(columns={'#ID':ID}) //rename ID column to remove
the #
```

- It could be useful to rename the columns name to avoid to have the same names of the output files (this is useful when we want to join two files, see [next slide](#))

```
df = df.rename(columns={"Mass_0":"Mzams_0", "Mass_1":"Mzams_1",
"a":"Semimajor_ini", "e":"Eccentricity_ini"})
```

Hints and tips



Join evolved and output files

- Sometimes a combination of data in the output and in the evolved files is needed. For example, we want to add information of the initial Mzams and initial orbital properties for all BHBH binaries. The [PANDAS](#) method [merge](#) handles all the possible joins among two dataframes.
- Assume we want to merge two dataframes (df_left, df_right) joining together the information of data sharing the same value on a given column (e.g. an ID), this is an inner join.

```
df_merged = df_left.merge( df_right, on="ID", how="inner",  
suffixes=("_left","_right") )
```

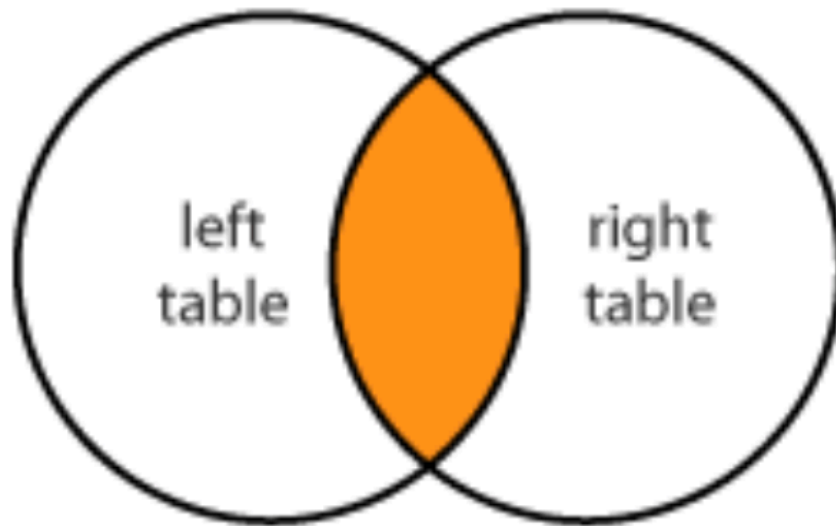
- **on:** column(s, can be a list of columns) to match during the merge of the two tables. The column(s) has(have) to be present in both the tables
- **how:** type of join to use, see documentation [here](#) and the [next slide](#)
- **suffixes:** columns with the same name in the two tables (not used in on) will be renamed adding these suffixes.

Hints and tips

Join evolved and output files - join types

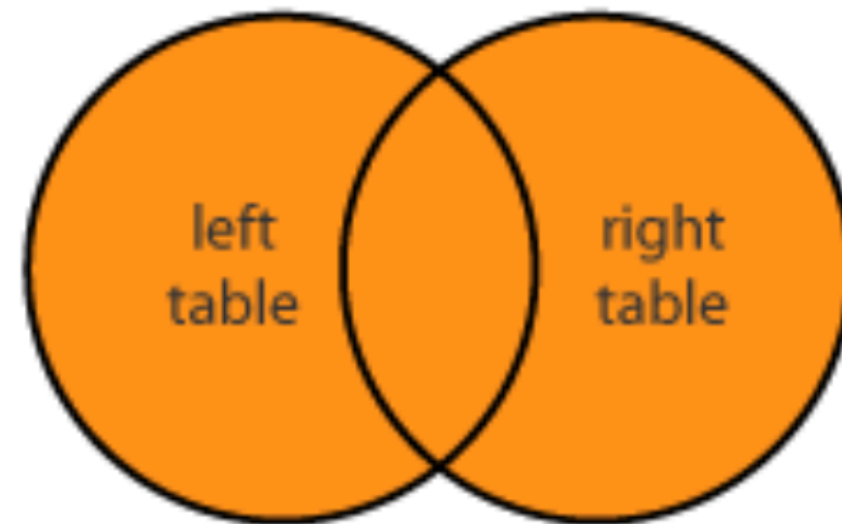


INNER JOIN



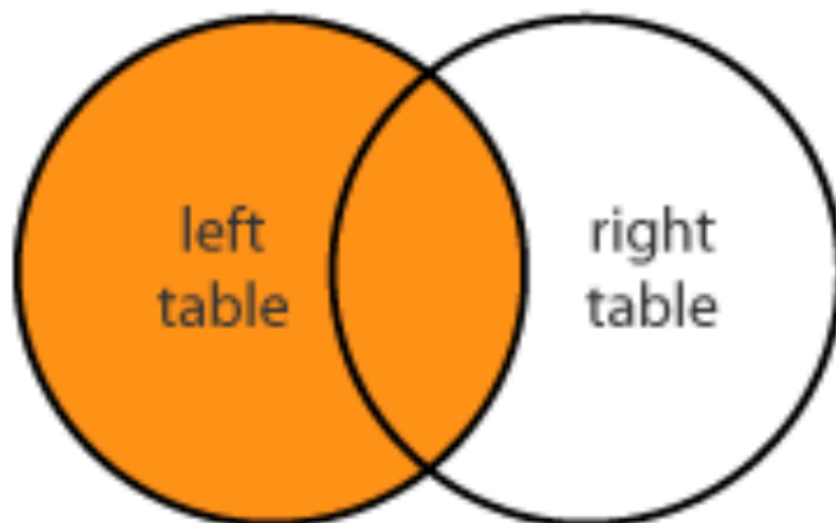
how: "inner"

FULL JOIN



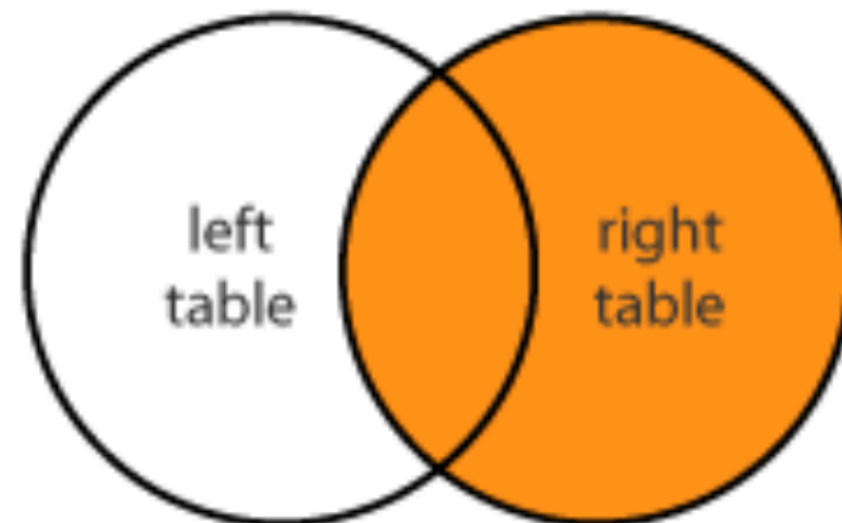
how: "outer"

LEFT JOIN



how: "left"

RIGHT JOIN



how: "right"

Hints and tips



Join evolved and output files - example

- Assume we have a dataframe containing all the binary black holes filtered from the output of a SEVN run (dfbhbh, [see here](#)). We want to add to the dataframe the info stored in the evolved output file.

Load the evolved file

```
evolved_path="../../sevn_outputs_example/sevn_output_single/evolved_0.dat"
dfe = pd.read_csv(evolved_path, sep=r"\s+") #df is a pandas dataframe
dfe=dfe.rename(columns={"#ID":"ID"}) #rename ID column to remove the #
#It could be useful to rename the columns name to avoid to have same names
of the output files
dfe = dfe.rename(columns={"Mass_0":"Mzams_0",
"Mass_1":"Mzams_1","a":"Semimajor_ini","e":"Eccentricity_ini"})
```

Join the bhbh dataframe with the evolved file

```
merged_df =dfbhbh.merge(dfe,on=["ID","name"],how="inner",suffixes=("", "_"))
```

Hints and tips



Reading and handling multiple files

- When SEVN is used with multithreading parallelisation, the [outputs are split in N files](#) where N is the number of used threads.
- Outputs can be gathered loading each single file with [PANDAS](#) and then concatenating the daframes with the method [concat](#)

```
import pandas as pd
df_0 = pd.read_csv("output_0.csv")
df_1 = pd.read_csv("output_1.csv")
df_2 = pd.read_csv("output_2.csv")
final_df = pd.concat([df_0,df_1,df_2])
```

- Inside the SEVN folder pyscript, you can find the script *pymake_unique_file*, it can be used to automatically merge the splitted outputs adding also information from the evolved and log files, its basic usage is:

```
./pymake_unique_file <path_to_the_output_folder> (e.g. sevn_output)
```

There are other options that can be set directly inside the script.

Hints and tips



Reading and handling multiple files: DASK

- An other option is to use the python package [DASK](#). The purpose of [DASK](#) is to provide parallelism to python data structures. In particular the [DASK DataFrame](#) mimics the Pandas data structure and methods.
- Most of the pandas methods can be called in dask using the same formalism. Moreover, some of them have “augmented functionalities”, the `read_csv` method can automatically handle split files.

```
import dask.dataframe as dd
dt = dd.read_csv("output_*.csv") #it will automatically load
#output_0.csv, output_1.csv etc. etc.
```

- [DASK](#) is based on the [lazy execution](#) concept, in practice the operations on dask dataframes (including the reading of files) are stored (in graphs), but not evaluated until the method `compute()` is explicitly called. The `compute()` call on a dask dataframe returns a pandas dataframe.

```
df = dt.compute() #df is a pandas dataframe containing all the
#data of the split output_file in a unique dataframe
```

Hints and tips



Reading and handling multiple files: DASK

- In order to exploit the dask dataframe, the general suggestion is to perform all the operation like filtering, joining using dask and call `compute()` only at the end.

For example: load the output files and filter the binary black holes

```
import dask.dataframe as dd

dt = dd.read_csv("senv_output/output_*")

idxBHBH=(dd.RemnantType_0==6)
        & (dd.RemnantType_1==6)
        & (dd.Semimajor.notnull())

dtbhbh=dt[idxBHBH]

df = dtbhbh.compute()
```

Hints and tips

Reading and handling logfiles: RegEx

Reg[ular]
Ex[pression]

- Logfiles are “partially” formatted (e.g. number of “columns” changes from event to event) therefore modules as numpy or pandas are not the best choice to handle them.
- A good alternative to manually loading and analysing the information row by row is to use *RegEx* and its python implementation [re](#) (built-in module, not need to install it)
- “A *Regular Expression* is a sequence of characters that specifies a search pattern. Usually such patterns are used by string-searching algorithms for “find” or “find and replace” operations on strings, or for input validation” (from Wikipedia). A tutorial for writing RegEx can be found [here: http://regular-expressions.info](http://regular-expressions.info).
- The python module [re](#) uses regular expressions to search for pattern in string and retrieve information from them.

Hints and tips

Reading and handling logfiles: RegEx

Reg[ular]
Ex[pression]

Example: RegEx and re to get all the SN kick velocities (module) from the logfile.

The information about the kick velocity is inside the [SN event in the logfile](#):

```
S;101931734175490;0;SN;11.84;3.53:3.53:2.17:1.23:5:2:245.96:59.45:0.56:-38.22:45.53
```

In particular, the info we want to get is in *d8* (59.45 km/s in the example above)

1.First, create the RexEx string, we can exploit the fact that the event identification is unique

```
regex_str="SN;.*?;(?:.*?:){7}(.*?):"
```

RegEx strings can look very cryptic at the first glance, essentially they include a list of proper characters and meta-characters with special meaning.

The site <https://regex101.com> is a very useful tool to both learn and test on the fly RegEx strings. It contains also a section where all the parts of the expression are explained.

Hints and tips

Reading and handling logfiles: RegEx

Reg[ular]
Ex[pression]

Example: RegEx and re to get all the SN kick velocities (module) from the logfile.

The information about the kick velocity is inside the [SN event in the logfile:](#)

```
S;101931734175490;0;SN;11.84;3.53:3.53:2.17:1.23:5:2:245.96:59.45:0.56:-38.22:45.53
```

In particular, the info we want to get is in *d8* (59.45 km/s in the example above)

1.First, create the RexEx string, we can exploit the fact that the event identification is unique

```
regex_str="SN;.*?;(?:.*?:){7}(.*?):"
```

;SN; matches the ;SN; part in the string, this identifies all the rows with a SN event

. * ? matches the second part of the header (the time), this string is essentially getting any character up to the ;. **.** matches any character, *** ?** matches the previous token between 0 and unlimited times, as few times as possible.

(?:. * ? :){7} matches the first 7 entries of the info part (d1-d7),

(? : ...) identify a non-capturing group, i.e. considered what is inside as a group but not storing it as output, **{7}** means replicate six times the previous group

(. * ?): matches the 8th entry (d8),

(...) identify a capturing group, i.e. considered what is inside as a group, what it is matched inside the parenthesis is captured for the output

Hints and tips

Reading and handling logfiles: RegEx

Reg[ular]
Ex[pression]

Example: RegEx and re to get all the SN kick velocities (module) from the logfile.

The information about the kick velocity is inside the [SN event in the logfile](#):

S;101931734175490;0;SN;11.84;3.53:3.53:2.17:1.23:5:2:245.96:59.45:0.56:-38.22:45.53

In particular, the info we want to get is in *d8* (59.45 km/s in the example above)

2. **Import re** `import re`

3. **Apply the *findall* method**, to the entire logfile (it has to be read)

```
logfile_path="sevn_output/logfile_0.dat"
with open(logfile_path,"r") as fo:
    ma=re.findall(regex_str,fo.read())
#ma is a list of all the matched output (they are all strings)
```

4. **Store it in a numpy array**

```
import numpy as np
vkicks=np.array(ma,dtype=float)
```

Hints and tips

Reg[ular]
Ex[pression]

Reading and handling logfiles: RegEx

Example: RegEx and re to count the number of CE for each binary of compact objects and add the information to the final output

1. **First, create the RegEx string**, we are interested only on counting CE events, therefore we need just to match the label *CE* in the [header](#) of each row and get the ID of the binary

```
regex_str=r'B;\d+;(\d+);CE;'
```

2. **Match and get the ID,**

```
logfile_path="sevn_output/logfile_0.dat"
with open(logfile_path,"r") as fo:
    ma=re.findall(regex_str,fo.read())
#ma is a list of all the matched output (they are all strings)
```

3. **Create a Dataframe and group by ID to count the number of CE events**

```
#Create a dataframe with all the ID obtained in the re findall
BID = pd.DataFrame({'ID':np.asarray(ma,dtype=int)})

#Group by ID and count occurrences producing a dataframe with the
#column ID and the column NCE containing the number of CEs
BID= BID.groupby(['ID']).size().reset_index(name='NCE')
```

Hints and tips

Reg[ular]
Ex[pression]

Reading and handling logfiles: RegEx

Example: RegEx and re to count the number of CE for each binary of compact objects and add the information to the final output

4. Load the output file and filter it, we are interested only on systems ending with a a bounded binary of compact objects (BHBH, NSNS, BHNS)

```
#Load the output file
output_path="../../sevn_outputs_example/sevn_output_singleoutput_0.csv"
df = pd.read_csv(output_path)

#Filter it
idx_compact_primary= (df.RemnantType_0==4) | (df.RemnantType_0==5) |
(df.RemnantType_0==6)

idx_compact_secondary= (df.RemnantType_1==4) | (df.RemnantType_1==5) |
(df.RemnantType_1==6)

idxcomp=(idx_compact_primary) & (idx_compact_secondary) &
(df.Semimajor.notnull())

dfcomp=df[idxcomp]
```

Hints and tips

Reg[ular]
Ex[pression]

Reading and handling logfiles: RegEx

Example: RegEx and re to count the number of CE for each binary of compact objects and add the information to the final output

5.Merge the two dataframe

```
dfmerged=pd.merge(dfcomp,BID,on='ID',how="left")
```

Notice, we use how="left" because because we want that all the binary in the dfcomp dataframe are included in the final dataframe (see [here](#)). The one with 0 CE events will have a NaN in the NCE column.
We will set this NaN to 0

```
dfmerged['NCE'] = dfmerged['NCE'].fillna(0)  
dfmerged = dfmerged.astype({"NCE": int})
```

Hints and tips

Reading and handling logfiles: RegEx

Reg[ular]
Ex[pression]

Other examples of regex usage can be found in the wiki page:
[Hints and tips](#)

SEVNpy Python module



What is SEVNpy?

SEVNpy is a Python module companion of SEVN.

The main target of the module is to **expose the SEVN main functionalities**, i.e. the evolution of Star and Binaries directly to **Python**.

In addition, it will* also contains utilities to setup the run of the main SEVN executables (sevn.x and sevnB.x) and analyse their output in a standardised way.



SEVNpy is currently under development.

If you find bugs, errors, unexpected behaviours, or differences with this userguide or the [documentation](#), please let us know and/or add a [gitlab issue](#)

*This part is currently working in progress

Getting SEVNpy?

SEVNpy is shipped [together with SEVN](#).

All the necessary files can be found in the folder SEVNpy:

Code folders:

sevnpy: folder containing the proper `sevnpy` module

Installation:

setup.py: file to install the module

setup_nosevncext.py file to install the module without the SEVN extension (i.e. we will not be able to use the functions to evolve stars and binaries)

Makefile: basic makefile to install SEVNpy

tests: folders containing testing scripts to be used with [pytest](#)

Others:

examples: folder contains examples of python scripts using seventy

docs: folders containing the documentation generated with [sphinx](#)

changelog.txt: file containing the update history

LICENCE: code licence

SEVNpy requirements

Requirements:

- **C++ compiler**, [same as SEVN](#)
- **Python**: minimum version ≥ 3.7
- [pip](#) (*optional*): package installer, warmly suggested
- [sphinx](#) (*optional*): used to (re)build the module documentation (the updated documentation [is already included in the module](#) folder, sphinx is necessary if one wants to rebuild it locally)
- **Additional Python modules:**
 - [numpy](#) (strictly required)
 - [pandas](#) (strictly required)
 - [scipy](#) (strictly required)
 - [setuptools](#) (strictly required)
 - [pytest](#) (*optional*): only needed to test the installation



See the file pyproject.toml for additional and updates requirements

SEVNpy installation

In order to install SEVNpy, open the terminal, then go inside the SEVNpy folder, e.g. from the main SEVN folder

```
cd SEVNpy
```

Then, two main methods to install SEVNpy:

- **pip installation** (preferred, but pip should be installed):

using make: `make`

direct command: `pip install .`

- direct installation:

using make: `make install_direct`

direct command: `Python setup.py install`

SEVNpy installation

If for any reason you don't want (or is not possible) to install the SEVN extension, use the alternative setup file *setup_nosevncext.py*

```
cd SEVNpy
```

```
python setup_nosevncext.py install
```

SEVNpy installation - tests

If the pytest module is available ([see here](#)), the installation can be tested with

using make: `make tests`

direct command: `pytest tests`

If all the tests passed or just raised warnings, SEVNpy is ready to be used.

If one or more test fails, please report it to the SEVN team using the [gitlab issues](#)

SEVNpy installation - Makefile

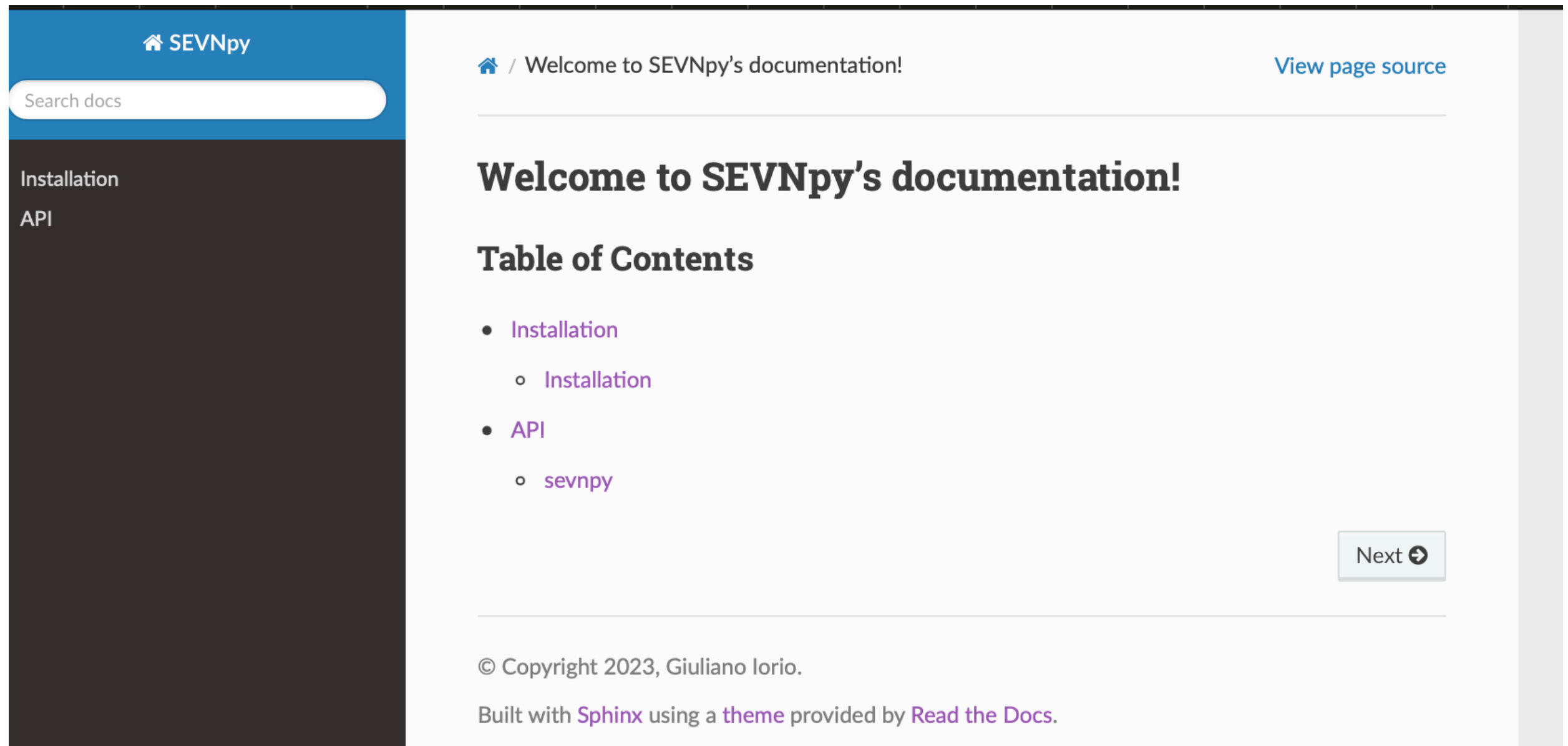
There are additional makefile commands

- *clean*: remove all the files generated by the installation
`make clean`
- *build*: just build the module without moving it to the python lib folder
`make build`
- *build_ext*: compile and build only the library containing the exposed SEVN C++ functions. The library will be build in place, e.g. the final library `sevnwrap` will be located in the same folder of the makefile. The library can be directly load in python
`make build_ext`
- *doc*: this will generate a new (html) documentation using [sphinx if available](#). The generated documentation will be placed in the folder *docs/build*
`make doc`

SEVNpy documentation

The complete SEVNpy API documentation is hosted at this link

<https://sevn.readthedocs.io>



The screenshot displays the SEVNpy documentation website. On the left is a dark sidebar with a blue header containing the 'SEVNpy' logo and a search bar. The sidebar lists 'Installation' and 'API' as main sections. The main content area has a light blue header with a home icon and the text 'Welcome to SEVNpy's documentation!', along with a 'View page source' link. Below the header is a 'Table of Contents' section with a bulleted list: 'Installation' (with a sub-item 'Installation') and 'API' (with a sub-item 'sevnpy'). A 'Next' button with a right arrow is located at the bottom right of the main content area. The footer contains copyright information for 2023 by Giuliano Iorio and mentions the use of Sphinx and Read the Docs theme.

SEVNpy

Search docs

Installation

API

Home / Welcome to SEVNpy's documentation! [View page source](#)

Welcome to SEVNpy's documentation!

Table of Contents

- Installation
 - Installation
- API
 - sevnpy

Next ➔

© Copyright 2023, Giuliano Iorio.
Built with Sphinx using a theme provided by Read the Docs.

SEVNpy: the sevn module

The sevn module inside SEVNpy can be used to easily exploit the SEVN evolve backend.

The main classes in the sevn module (and its submodules) are:

- [SEVNmanager](#): it is used to manage the connection to the SEVN backend and the setup of the SEVN parameters. It is never instantiated, rather the static (or class) methods *init* and *close* are directly called to initialise and close a SEVN session. (*from sevnpy.sevn import SEVNmanager*)
- [Star](#): this class is used to initialise a star, then the methods *evolve* and *evolve_for* can be used to evolve the Star using the SEVN backend. (*from sevnpy.sevn import Star*)
- ***Binstar***: this class is used to initialise a binary (WORKING IN PROGRESS... not available at the moment).

SEVNpy: how to initialise a SEVN session

All the classes, methods/functions inside the `sevn` module need to be called within an initialised SEVN session. The steps to initialise (and close) a SEVN session are as follow.

1- Import the `sevn` manager

```
from sevnpy.sevn import SEVNmanager
```

Be patient, it can take a while since at this point SEVN is loading the tables

2- Initialise the SEVN session with the static method [*init*](#)

```
SEVNmanager.init()
```

In this way all the SEVN parameters are set to the default value. If you want to change some (or all) of them, the *init* method accepts a dictionary containing the pair `property_name:value`. All the parameters that are not in the dictionary will be set at their default value, e.g.

```
SEVNmanager.init({"ce_alpha":1})
```

3- Insert your runtime code

4- Finalise and close the SEVN session with the static method [*close*](#)

```
SEVNmanager.close()
```

SEVNpy: notes on the SEVNmanager

- If the method *init* is called again without closing it, the SEVNmanager automatically closes the older session.
- All the methods described in this slide are static, so they can (and should) be called directly without the need of instantiate the class (e.g. SEVNmanager.get_ID())
- To check if SEVNmanager is initialised (useful for functions that are defined in external modules or outside a SEVNsession), it is possible to use the SEVN method [*check_initiliased*](#), it returns True or False depending on the initialisation status
- Each time an *init* is called, the SEVNmanager stores internally the call ID (an increasing sequential number) and the SEVN parameters referred to this call. These quantities can be retrieved with the methods [*get_sevnParams_history*](#), the method return a dictionary contains the pair *IDcall:params* where *params* is a dictionary contains all the used SEVN parameters. To get directly the last SEVN parameters it is possible to use the method [*get_sevnParams*](#) (and *get_ID* to get the current ID).
- In order to give a look to the default SEVN parameters use the method [*get_sevnParamsDefault*](#), to have a description of each parameter uses [*sevnparam_describe*](#), use a list of one or more parameter names as input or leave it blank to get the description of all the parameters. To obtain info about the loaded stellar tables, uses the method [*tables_info*](#), while to obtain the path of the main SEVN folder uses [*sevn_path*](#).

The details documentation of the SEVNmanager API can be found here

https://sevn.readthedocs.io/en/latest/api_autosummary/sevnpy.sevn.sevnmanager.SEVNmanager.html#sevnpy.sevn.sevnmanager.SEVNmanager

SEVNpy: notes on the SEVN parameters

The SEVNpy module uses a subsample of SEVN parameters since some of them are directly related to the SEVN executables (e.g. options for the outputs or inputs).

In order to retrieve the complete list of the SEVN parameters (and their default value) it is possible the SEVNmanager static method [get_sevnparams_raw](#)

To select only the SEVN parameters that are not included in SEVNpy do the following

```
from sevnpy.sevn import SEVNmanager
parall=set(SEVNmager.SEVNmanager.get_sevnparams_raw().keys())
parsevnpy=set(SEVNmager.SEVNmanager.get_sevnParams().keys())
SEVNexclusive = parall.difference(parsevnpy)
```

The details documentation of the SEVNmanager API can be found here

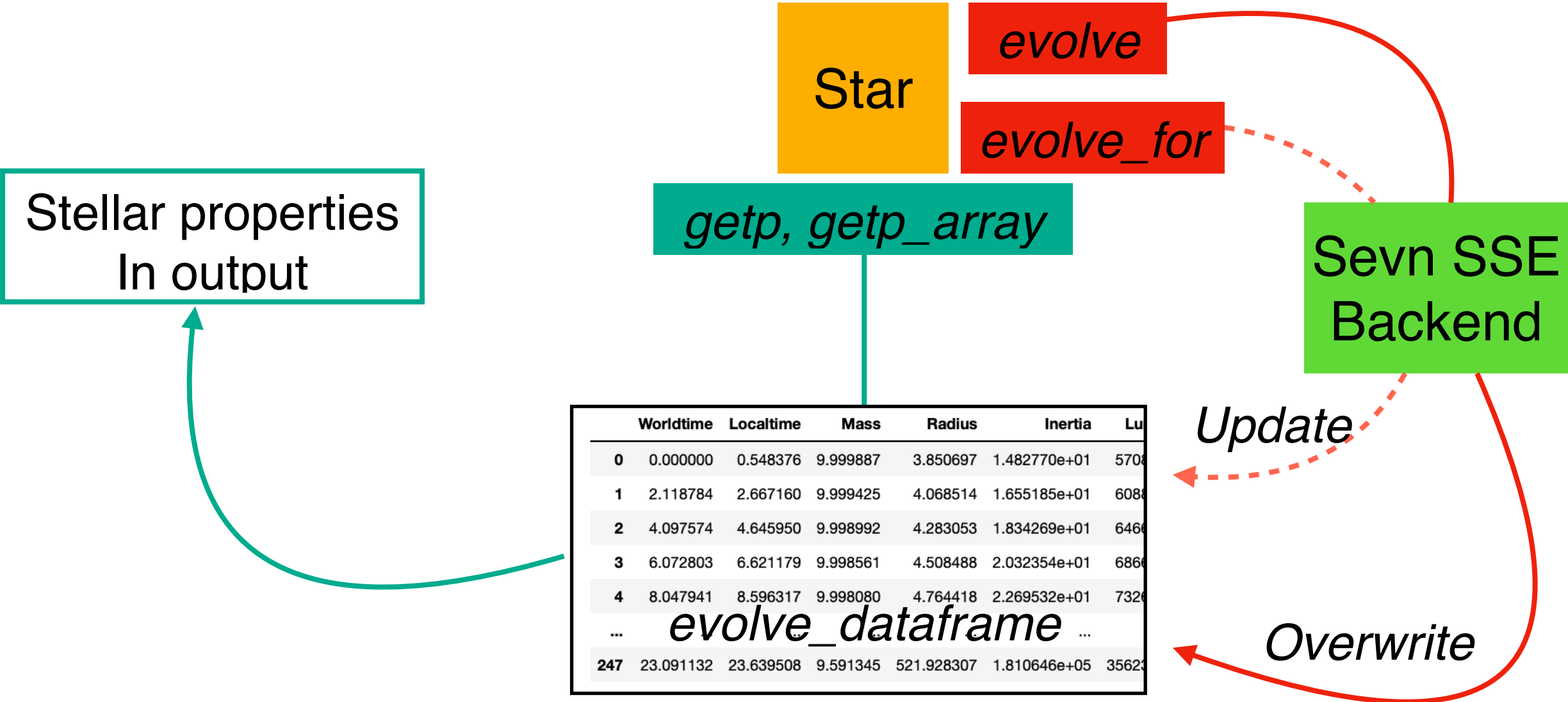
https://sevn.readthedocs.io/en/latest/api_autosummary/sevnpy.sevn.sevnmanager.SEVNmanager.html#sevnpy.sevn.sevnmanager.SEVNmanager

SEVNpy: the class Star

The class `Star` is used to initialise and evolve a star object given its initial zero age main sequence mass (M_{zams}), metallicity (Z), spin initial age and SN model to pass from the star to remnant.

Internally the stellar properties are stored in a pandas dataframe (`evolve_dataframe`). The data frame is filled at the initialisation with the initial properties and after an evolve call (methods `evolve` and `evolve_for`). Then the properties are retrieved with the `getp` and `getp_array` methods.

The diagram below shows a schematic view of the class `Star`



SEVNpy: the class Star - initialise-1

To initialise a [Star object](#), only two parameters are required:

- **Mzams**: initial zero age main sequence mass in Msun. If the Star is initialised as a pureHe star (see below) Mzams refers to the initial mass at the start of the coreHe burning phase
- **Z**: stellar metallicity

These two values (together with the loaded stellar tables) univocally define an [interpolating track](#) that is used to initialise and evolve the star (during the evolution the interpolating track can change, [see here](#)).

There are other optional parameters that have default values:

- **spin**: initial stellar spin, i.e. fraction between the angular velocity and the critical angular velocity, default: 0
- **tini**: initial age of the Star, it accepts the same input of the SEVN parameter tstart ([see here](#)), default: “zams”
- **snmodel**: SEVN option for the SNmodel, same as the SEVN parameter snmode ([see here](#)), default: “rapid_gauNS”
- **star_flag**: flag used to initialise special type of star (default, “H”)
 - “H”: initialise a standard hydrogen star
 - “HE”: initialise a pureHE star (Mzams is the initial mass of the star at the start of the core-He burning)
 - “HEWD,COWD,ONEWD,NS,BH”: initialise a remnant object of given kind (in this case Mzams is the mass of the compact remnant)
- **ID**: identifier of the star, if None a sequential ID is used, default: None
- **rseed**: random seed to be used in the SEVN backend, if None it is assigned randomly, default: None
- **Mass, MHE, MCO, Radius**: use these values to modify the initial masses and radius of the star as given by the interpolating track. The masses are in Msun, the Radius in Rsun. **Notice** there is a mass hierarchy, MHE can be set only if also Mass is set, and MCO only if also MHE is set. **Notice**, if Mass==MHE a pureHE star is initialised independently by the input star_flag, if Mass>MHE a H star is set independently by the input star_flag

SEVNpy: the class Star - initialise-2

A Star can also be initialised from another Star instance using the static method [*from_star*](#) using as input the other Star.

```
s1 = Star(10,0.02)
s1.evolve()
s2 = Star.from_star(s1)
```

The new star will be initialised with the current properties of the other star, i.e. with the properties stored in the last row of the `evolution_dataframe` ([see here](#))

The `star_flag` ([see here](#)) of the star will be automatically set considering the properties of the other star.

from_star also have the parameters `ID` and `rseed`. If `ID` and `resend` are `None` new ones are generated ([see here](#)). If `rseed` has the special (default) value “same”, the `rseed` is taken from the other star.

SEVNpy: the class Star - evolution-1

The Single Stellar Evolution (SSE) of a Star instance is run through two methods that are directly connected with the SEVN backend ([see here](#)):

- [evolve](#): it requires as an input the ending time of the evolution in Myr (or the special default value “end”) to evolve until the remnant formation. It evolves the star overwriting the evolve_dataframe, so every new evolution call remove the past Stellar properties.

```
s1 = Star(10,0.02)
s1.evolve() #evolve until the end, i.e. remnant formation. The #current
properties of the Star are the one of the remnant
s1.evolve(1) #evolve until 1 Myr, all the evolve steps done #after 1
Myr in the first evolution called are now removed and #the current
properties of the Star are the one after 1 Myr of evolution
```

- [evolve_for](#): it requires as an input the time interval for the star evolution in Myr. It takes the current properties of the Star (e.g. the last row in the evolve_dataframe, [see here](#)) and evolve the star for the amount of time in input. It updates the evolve_dataframe, so the past evolution steps are not overwritten.

```
s1 = Star(10,0.02)
s1.evolve_for(1) #evolve from initial time for 1 Myr, the current
properties are the one after 1 Myr of evolution
s1.evolve_for(2) #evolve for other 2 Myr, the current properties are
the one after 3 (1+2) Myr of evolution
```

SEVNpy: the class Star - evolution-2

evolve_for can be used to built a custom Python evolve routine.

For example we can create an evolve routine that get the values each 1 Myr until the remnant formation storing the properties only if the Luminosity is larger than a certain value:

```
s1=Star(10,0.02)
phase = s1.getp_array("Phase",mode="last")
ret_prop=s1.getp() #Store the initial value
Lcut = 100

while phase<7:
    s1.evolve_for(1) #evolve for 1 Myr
    if s1.getp_array("Luminosity")[-1]>Lcut: #check luminosity
        ret_propt=pd.concat([ret_prop,s1.getp(mode="last")]) #Store
just the last value
```

SEVNpy: the class Star - get the properties

There are two main method that can be used to get the Stellar properties from the *evolve_dataframe* (it cannot be directly accessed):

- [getp](#): return the star properties as a pandas Dataframe
- `[]`: use the `[]` operator (equivalent to `getp`, but forcing `mode="all"` and `t=None`)
- [getp_array](#): return the star properties as a numpy array (the column order depends on the properties in input see below)

Both the methods have three parameters:

- **properties**: the name of the properties to retrieve, or a list of names. If `None` (default) return all the properties. The name of the properties are the same as SEVN ([see here](#)), but not all the SEVN properties are stored, the available property names can be retrieve with the class attribute [pnames](#).
- **mode**: set what to get in input, if `"all"` (default) returns the all the rows from the *evolve_dataframe*, if `"first"/"last"` returns only the initial/current properties, i.e the first/last row from the *evolve_dataframe*
- **t**: if this value is not `None` (default), the mode parameter will not be considered and the properties will be retrieved exactly at time(s) `t` (it could be a single number or a list of times).



The values of the properties taken at times not included in the *evolve_dataframe* are interpolated, they are not been obtained with the SEVN backend, so use them wisely. The properties at times larger than the current star Worldtime are set to 0. Some integer properties such as the Phase could not make sense if interpolated. **DON'T ABUSE OF THIS OPTION AND DO NOT BLINDLY TRUST THE RESULTS.**

```
s1 = Star(10,0.02)
s1.evolve_for(2) #Evolve for 2 Myr of evolution
# Get the current Worldtime, Mass and Radius
s1.getp(properties=["Worldtime","Mass","Radius"], mode="last")
# Get the Worldtime, Mass and Radius and all the timesteps
s1.getp(properties=["Worldtime","Mass","Radius"])
# Get the Worldtime, Mass and Radius and given times (interpolated)
s1.getp(properties=["Worldtime","Mass","Radius"],t=np.linspace(1,2,100))
```

SEVNpy: the class Star - the log

When the evolution methods are called (see [here](#)) the SEVN backend produces log messages that are stored in the class Star.

It is possible to access the log through the property [log](#), the log is a string containing all the log messages separated by the special character `/n`.

Same as for the *evolve_dataframe*, the method *evolve* ([see here](#)) overwrites all the precedent log messages, while *evolve_for* ([see here](#)) update the log adding the new log messages raised during the evolution.

```
In [1]: import numpy as np
        from sevnpy.sevn import SEVNmanager, Star
```

```
In [2]: SEVNmanager.init()

        s = Star(50,0.025)
        s.evolve()

        print(s.log)

        SEVNmanager.close()
```

```
S;119634072590723;0;HENAKED;3.780092e+00;20.3296:19.8149:0:1.86895:2.081:4:50:22.9925:0
S;119634072590723;0;SN;4.084468e+00;13.9587:13.9587:11.3759:13.4587:6:2:423.556:0:-0:0:-0
```

SEVNpy: the class Star - other methods

In addition to the evolve methods ([see here](#)) and getp methods ([see here](#)) the class Star contains other useful methods or attribute such as:

- [get_remnant](#): use this method to retrieve directly the property of the remnant produced by star (using the initialisation properties). Even this method internally uses the SEVN evolution backend, it does not overwrite or update neither the *evolve_dataframe* nor the log messages.
- [get_SN_kick](#): use this method to automatically retrieve the information about the SN kick from the log messages. If the log message does not contain the SN entry, the return SN kick will contain NaN
- [to_star](#): use this method to return a new instance of the class Star initialised from the current properties of the star. This is similar to the class method *from_star* ([see here](#)).

The detailed documentation of the Star API can be found here:

https://sevn.readthedocs.io/en/latest/api_autosummary/sevnpy.sevn.star.Star.html#sevnpy.sevn.star.Star

SEVNpy: the io module

The io module include classes and functions to help the user in preparing the input for the SEVN executables and in analysing their outputs.

The main content are the following:

- The SEVNDataResults class: it is a base class used to store the data loaded from the SEVN outputs, it is specialised depending on the data (e.g. log)
- The log readers classes and functions: utilities to load the [log messages from the SEVN outputs](#)
- under development

The detailed documentation of the io API can be found here:
https://sevn.readthedocs.io/en/latest/api/_autosummary/sevnpy.io.html#module-sevnpy.io

SEVNpy: the io module - SEVNDataResults

The base class [SEVNDataResults](#) contains the following basic attributes

- [data](#): store the underling data as a pandas Dataframe
- [data_info](#): info about the stored data
- [columns_info](#): info (description) of the columns stored in the pandas Dataframe
- [sevn_files](#): files that have been used to get the data



This base class is never directly used, rather the users will access to the inherited specialised class (e.g. [SEVNDataLog](#)). All this derived classes will have the basic properties above plus other specific properties. See next slides



Even if it is possible for the user to initialise a SEVNDataResults (and derived classes) instances, the main purpose of this classes is to be the return of functions and methods that read the SEVN outputs

SEVNpy: the io module - SEVNDataLog

The class [SEVNDataLog](#) is used to store the data from the [SEVN log messages and files](#) in a “more” human readable format. In addition to the properties of the base [SEVNDataResults](#) class:

- [data](#): the underling data as a pandas Dataframe
- [data_info](#): info about the stored data
- [columns_info](#): info (description) of the columns stored in the pandas Dataframe
- [sevn_files](#): files that have been used to get the data

It also contains the additional properties:

- [regex_pattern](#): store the regex pattern used to get the results from the log message or file

SEVNpy: the io module - read log data

The module [logreader](#) contains classes and the methods to get the properties from log files and log messages.

The core of the module is the base class [LogReader](#), it can be used to define all the instruction to read from a log messages containing one or more SEVN events. However, the user does not have to use directly this class unless they want to define a custom log reader schema (see the API documentation). The module already include a list of specialised `LogReader` classes to read all the SEVN events, the class name is *<Event_label>LogReader*, so for example [SNLogReader](#) is the specialised class to get the information from the [SN log message](#).

Even in this case, the users do not have to directly use one of the specialised `LogReader` classes since the module provide a number of useful “wrapper” functions to facilitate the log reading operation (see next slide).

The detailed documentation of the logreader API can be found here:

https://sevn.readthedocs.io/en/latest/api/_autosummary/sevnpy.io.logreader.html#

SEVNpy: the io module - read log data

The module [logreader](#) contains two “wrapper” function to read a logfile/logmessage:

- [readlogfiles](#): read from a SEVN logfile retrieving the information of [SEVN events](#) as [SEVNDataLog](#) ([see here](#)). If multiple events are give in input, the output will be a dictionary contains the pair event_name:SevnDataLog of each event.
- [readlogstring](#): read a log message from a string retrieving the information of [SEVN events](#) as pandas Dataframe. If multiple events are give in input, the output will be a dictionary contains the pair event_name:Dataframe of each event.

The functions have similar inputs (except for few parameters):

- **logfiles** (only for *readlogfiles*): path of the logfile to read, or least of path of logfiles to read. It accepts wildcard * to consider all the file of a given kind, e.g. logfile_*.dat to consider all the logfiles produced by a multithreads SN run.
- **string** (only for *readlogstring*): string with logmessage(s)
- **events**: event for which we want to retrieve the information, it could be a single event or a list of events. If you want to get the list of available events use the io function [logevents_list](#)
- **capturing_cols**: what properties to retrieve. It possible to use special string values, or property names or integer indexes. The special values are: “default” (default value): return only columns containing str, int or float values, “all”: return all the properties (including the header), “header”, return only the properties of the header. The indexes are relative to the position in the SEVN log description (starting from 0 as the first element of the header, [see here](#)). To check the names of the properties use the io function [logcolumns_info](#) giving as input the name of the events.
- **nproc** (only for *readlogfiles*): number of processes to be used for a parallel reading on multiple files.
- **names**: List of names to consider in the logfile. If None (default) do not filter the names.
- **IDs**: List of IDs to consider in the logfile. If None (default) do not filter the IDs.

[Examples](#)



SEVNpy: the io module - read log data

Examples

```
In [1]: from sevnpy.io import readlogfiles, logevents_list, logcolumns_info
```

```
In [2]: #See the available events
print(logevents_list())
```

```
['SN', 'WD', 'NS', 'HENAKED', 'CONAKED', 'BSN', 'RLO_BEGIN', 'RLO_END', 'CIRC', 'CE', 'COLLISION', 'MERGER', 'SWALLOWED']
```

```
In [5]: #See the columns name of a given event
print(logcolumns_info("BSN"))
```

```
{0: 'logtype', 1: 'name', 2: 'ID', 3: 'event', 4: 'time', 5: 'SID_SN', 6: 'Mass_preSN_SN', 7: 'MHE_preSN_SN', 8: 'MCO_preSN_SN', 9: 'Phase_preSN_SN', 10: 'RemnantType_postSN_SN', 11: 'SID_Other', 12: 'Mass_preSN_Other', 13: 'MHE_preSN_Other', 14: 'MCO_preSN_Other', 15: 'Phase_preSN_Other', 16: 'RemnantType_postSN_Other', 17: 'Semimajor_preSN', 18: 'Eccentricity_preSN', 19: 'Semimajor_postSN', 20: 'Eccentricity_postSN', 21: 'CosNu', 22: 'Vcom'}
```

```
In [8]: #Get the properties reading the log files
out=readlogfiles("logfile_*.dat",events=["BSN"],capturing_cols=["name",2,"time",6])
print(out.data)
```

	name	ID	time	Mass_preSN_SN	IDfile
0	600688921382265	0	15.76611	2.6920	0036001354
0	428321860166908	1	207.22690	0.5015	c41e87b88e

```
In [10]: #Get the proprieties for multiple events
dicout=readlogfiles("logfile_*.dat",events=["BSN","RLO_BEGIN"])
print(dicout["BSN"].data)
print(dicout["RLO_BEGIN"].data)
```

	name	ID	time	...	CosNu	Vcom	IDfile
0	600688921382265	0	15.76611	...	0.983899	18.827600	0036001354
0	428321860166908	1	207.22690	...	1.000000	0.544003	c41e87b88e

[2 rows x 22 columns]

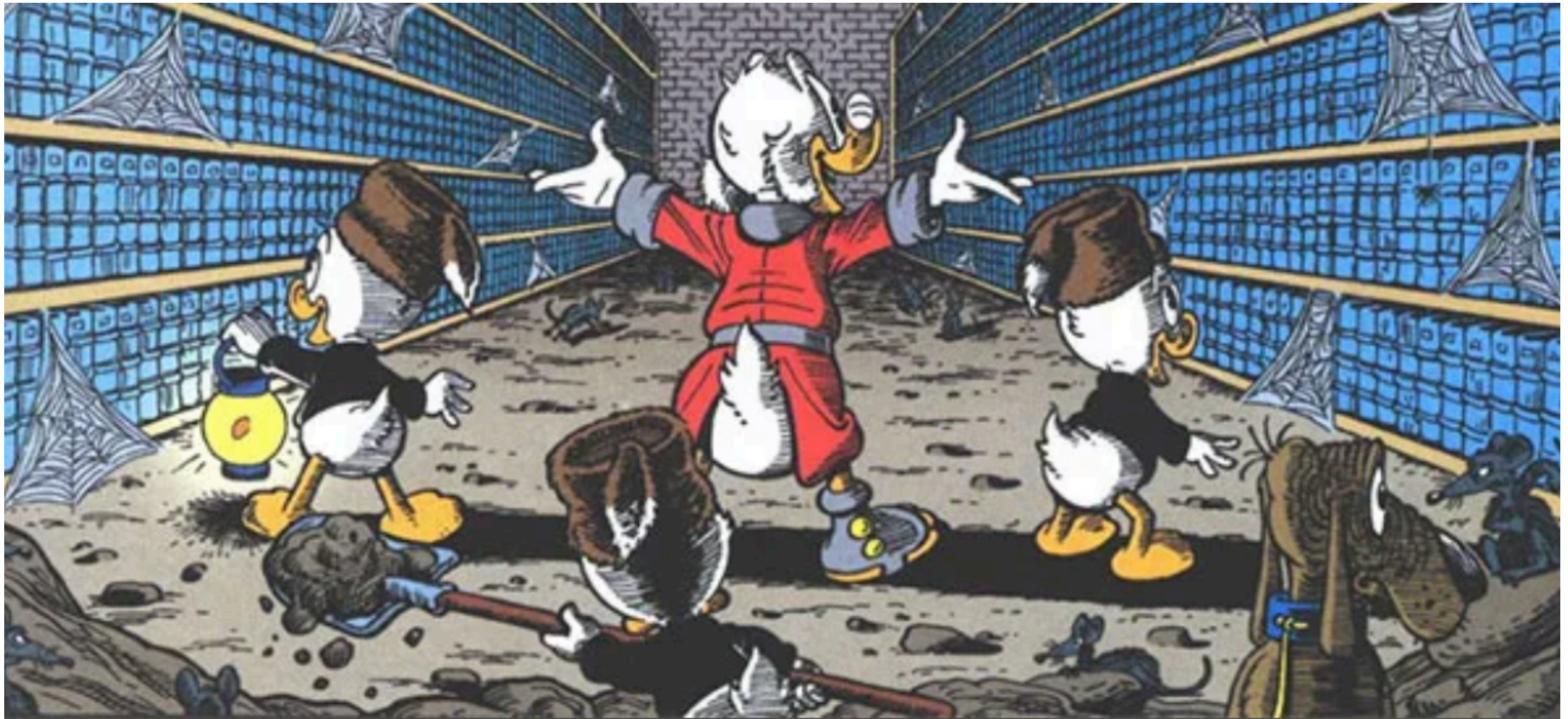
	name	ID	...	Eccentricity_RLOB	IDfile
0	600688921382265	0	...	0.046904	0036001354
1	600688921382265	0	...	0.000000	0036001354
2	600688921382265	0	...	0.207284	0036001354
0	428321860166908	1	...	0.000000	c41e87b88e
1	428321860166908	1	...	0.000000	c41e87b88e

[5 rows x 24 columns]

Advanced Guide



SEVN as a library



All the classes and functions defined in SEVN are collected inside a C++ library. The standard compilation link directly the main SEVN executables and the SEVN library.

However, the SEVN library can be used as any other C++ library to include the stellar and binary evolution from SEVN in other C++ codes or even in codes written in other languages (e.g. Python).

In order to facilitate the usage of the SEVN library, we introduce advanced options in the Cmake (see [here](#)), we produce a Cmake SEVN package and as an alternative a script to properly set the C++ flags needed to link the SEVN library as environment variables ([see here](#)).

In order to install the libraries, after running **make**, you need to run also **make install**, you can decide the installation path with the cmake option *-DCMAKE_INSTALL_PREFIX* ([see here](#))

Compiling - extra options for SEVN lib

```
cmake .. [-Dexe=ON*/OFF] [-Dsharedlib=ON*/OFF]  
[-Dstaticlib=ON/OFF*] [-Dlinklib=auto*/static/shared]  
[-DCMAKE_INSTALL_PREFIX=]
```

- Dexe** If disabled do not compile the executables (i.e. just compile the SEVN libraries)
 - Dshared** Enable the compilation of the shared (dynamic) version of the SEVN libraries (NB: at least one between Shared and
 - Dstatic** Enable the compilation of the static version of the SEVN libraries
- NOTICE:** At least one between shared and static needs to be enabled
- Dlinklib** Select what version of the SEVN library (static or dynamic) is used to compile the SEVN executable. The chosen version needs to be enabled
 - DCMAKE_INSTALL_PREFIX** Path for the installation of the libraries (if not included, the library will be installed in the standard GNU space, see [here](#))

Installing the library

In order to install the SEVN library:

1. follow the usual instruction to compile SEVN (see [here](#)). If not needed you can compile only the SEVN libraries setting `-Dexe=OFF` when calling `CMake` (see [here](#)).
2. After running `Make`, type `make install`. This will install the libraries.
3. DONE! You are ready to use the SEVN library in your projects.

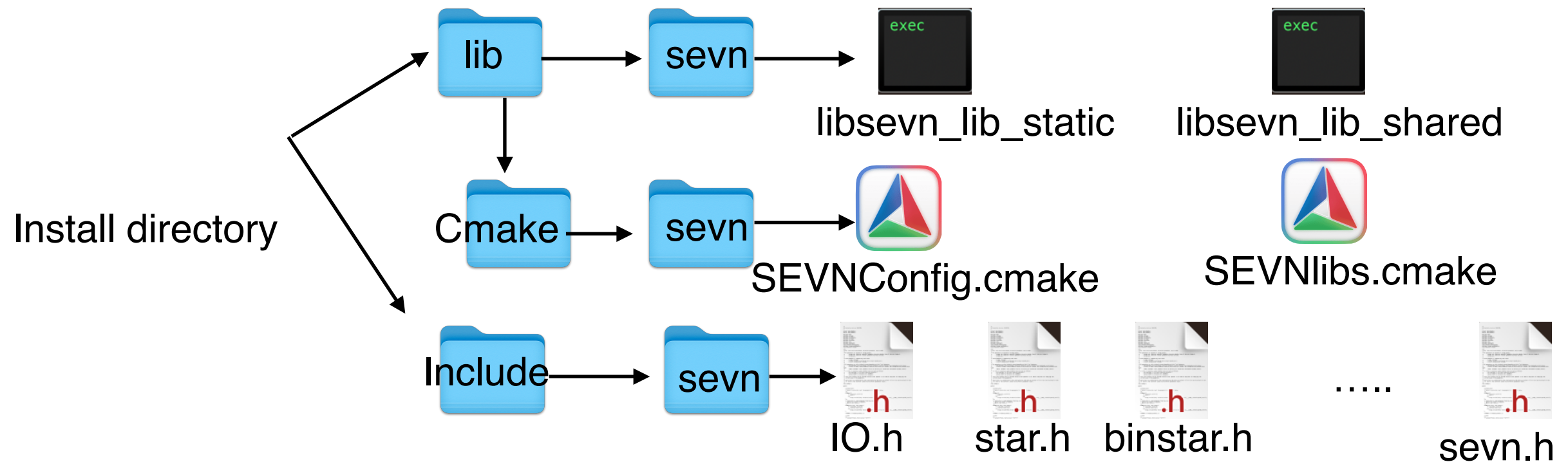
```
Install the project...  
-- Install configuration: ""  
-- Installing: /usr/local/lib/sevn/libsevn_lib_static.a
```



In some cases, the command `make install` produces an error because you don't have the permission to move files in the installing directory. The default installation directory is `/usr/local`. In such cases try to run `sudo make install`. If you don't have the privileged to run the command as `sudo` just change the installation directory with the `cmake` option `-DCMAKE_INSTALL_PREFIX` (see [here](#) and [here](#)).

Installing the libraries: installation path

The install command move the compiled sevn libraries (*libsevn_lib_static* and/or *libsevn_lib_shared*, the extension .a, .so, .dylib depends on the operative system) and the SEVN headers in the installation directory:



The default install directory is */usr/local/*, but it can be changed setting the *CMAKE* option *-DCMAKE_INSTALL_PREFIX*

For example

```
cmake . -DCMAKE_INSTALL_PREFIX=.
```

will install the libraries exactly in the folder in which you are calling cmake (usually build), therefore after *make* and *make install* two subfolder (*lib* and *include*) will appear (if not already present)

Using the SEVN library - main headers

In order to use the functions/methods/classes from the SEVN library, you should include the appropriate header in the .cpp file.

The main headers are:

- **star.h**: contains the class star (for the SSE)
- **binstar.h**: contains the class binstar (for the BSE)
- **IO.h**: contains the class IO that is used to handle the I/O operations (i.e. read the runtime parameters and load the tables)
- **sevnlog.h**: contains the SEVN class that can be used to produce log informations
- **errand.h**: introduces the various SEVN errors
- **utilities.h**: contains various functions used in SEVN and not belonging to specific method
- **static_main.h** (see next slide): it does not contains code that can be used, but it is extremely important since it initialises all the quantities necessary to use the SEVN library. **It must be** the first header included from the SEVN library
- **sevn.h** (see next slide): this is header is a wrapper of all the most important SEVN libraries (the one discussed before), it also properly include the static_main.h at the beginning. **It is strongly suggested** to use this library to include SEVN.

Using the SEVN library - special headers

sevn.h (suggested)

In the previous slide, we stress to use **sevn.h** to include SEVN in your project. It will automatically include all the main SEVN libraries in the right order.

```
#include <sevn.h>

#include <vector>
#include <string>
```

static_main.h (be careful..)



Your FIRST include needs to be ALWAYS *static_main.h*

```
#include <static_main.h>
#include <IO.h>
#include <binstar.h>
```

If you are not using the **sevn.h** header, but rather import the individual SEVN headers, **BE CAREFUL**, you need to always include **static_main.h** as the first SEVN header

Be careful if you need to include sevn in multiple files in your code, see next slide

Using the SEVN library - special headers



**BE CAREFUL IF YOUR CODE IS SPLIT IN
MULTIPLE FILE INCLUDING THE SEVN
LIBRARIES**

The **static_main.h** contains the initialisation of many static variables. If it is included in more than one file used to produce your final executables, during the compilation, at the linker stage, you will receive the error on multiple definitions of the same file.

To solve this issue, **include** the **static_main.h** or **sevn.h** ([see here](#)) **only in the main cpp file** and include only the specific headers (e.g. `star.h` or `IO.h`) in the other files using the SEVN API.

Using the SEVN library - static variables



**READ BELOW IF YOUR CODE DEFINES
STATIC VARIABLES THAT DEPENDS ON
SEVN STATIC VARIABLES DEFINED IN `static_main.h`**



The **`static_main.h`** contains the initialisation of many static variables that are initialised at runtime. The C++ standards assure that the static variable in the same translation unit are initialised in the order of definition. However there are not standards for the initialisation order of static variable define in multiple translation units.

If your code defines static variables that depends that depends on SEVN static variables (e.g. a list of properties ID) it is not assured that the SEVN static variables will be initialised before the initialisation of the static variable defined in your code. This could represents a source of runtime errors, or even worst, bugs and unpredictable behaviour.

It is better to avoid to define static variables depending on the SEVN static variables, but it is necessary a solution to the issue is to initialise the variable in functions or class initialisations the first time they are required. An example of this solution can be found in the SEVNpy wrapper looking at the class *Star_property* and its static variable *propertiesID_list* containing the list of the SEVN property ID. The list is initialised as empty, and then in the class constructor we check if it is empty it is then filled with the proper values.

Using the SEVN library - special headers

`sevn_version.h`

This header includes a C++ struct (*SEVNinfo*) storing the information about the current version of SEVN and the last commit it belongs to. All their members and methods are static so they can be retrieved simply using *SEVNinfo* as a namespace (i.e. *SEVNinfo::.....*). The header is already included in the “glue” header `sevn.h` ([see here](#)).

The members containing info about the code version are:

- `SEVNinfo::VERSION_MAJOR` (unsigned int) major code version
- `SEVNinfo::VERSION_MINOR` (unsigned int) minor code version
- `SEVNinfo::VERSION_PATCH` (unsigned int) patch code version
- `SEVNinfo::VERSION` (const char*) major.minor.patch

The members containing the information about the git repository are listed in the next slide

Using the SEVN library - main headers

sevn_version.h

The members containing info about the git repository are:

- SEVNinfo::GIT_BRANCH (const char*) name of the current git branch
- SEVNinfo::GIT_SHA (const char*) hash code of the last **local** commit
- SEVNinfo::GIT_SHATIME (const char*) date of the last **local** commit
- SEVNinfo::GIT_SHATIMESTAMP (unsigned long int) timestamp of the last **local** commit
- SEVNinfo::GIT_COUNTER (unsigned int) progressive commit counter related to the last **local** commit
- SEVNinfo::GIT_SHA_REMOTE (const char*) hash code of the last **remote** commit
- SEVNinfo::GIT_SHATIME_REMOTE (const char*) date of the last **remote** commit
- SEVNinfo::GIT_SHATIMESTAMP_REMOTE (unsigned long int) timestamp of the last **remote** commit
- SEVNinfo::GIT_COUNTER_REMOTE (unsigned int) progressive commit counter related to the last **remote** commit

The **local** commits are the one not yet pushed to the remote repository.



NOTICE:

If SEVN has not been cloned from the git repository, these quantities will not be available and are set to NOT AVAILABLE (the strings) or to 0 (the numbers)

Linking the SEVN library

In order to properly compile the executable that is using the SEVN library we have to:

1. Use the same C++ compiler used to compile the SEVN library
2. Tell the compiler the name of the library we are using: *sevn_lib_static* if we are using the static version, *sevn_lib_shared* if we are using the shared version. We can do this adding the compiler flag *-lsevn_lib_static* or *-lsevn_lib_shared*
3. Tell the compiler where to find the library (if not already inside the library path) with the command *-L<path_of_the_library_folder>*, e.g. *-L/usr/local/lib/sevn*
4. Tell the compiler where to find the library headers with the command *-I<path_of_the_header_folder>*, e.g. *-I/usr/local/include/sevn*
5. Use the *-fPIC* flag if we are linking the shared library
6. Use the *-fopenmp* flag if the libraries have been linked with the OpenMP support

Considering a default installation of the SEVN library in */usr/local* and assuming the executable is called *main.cpp* we will have, e.g.:

```
g++ main.cpp -lsevn_lib_static -L/usr/local/lib -I/usr/local/include -fopenmp
```



NOTICE:

All this process can be simplified using [CMake](#) or the [SEVNENV script](#)

Linking the SEVN library - CMake

The [library installation](#) produces a [CMake SEVN package](#).

A custom executable can be link to the SEVN libraries with few additions to the CMake file used to compile executable (*CMakeList.txt*)

1. Add the command `find_package(SEVN)`
2. To link the executable using the static lib just use `target_link_libraries(executable_name SEVNLIBS::sevn_lib_static)`
3. To link the executable using the shared lib just use `target_link_libraries(executable_name SEVNLIBS::sevn_lib_shared)`

The SEVN CMake module defines the following variables:

- `SEVN_FOUND` ON if the package has been found, OFF otherwise
- `SEVN_INCLUDE_DIR` path to the installed SEVN headers
- `SEVN_STATICLIB` ON if the static lib has been compiled, OFF otherwise
- `SEVN_SHAREDLIB` ON if the shared lib has been compiled, OFF otherwise
- `SEVNLIBS::sevn_lib_static` static libraries
- `SEVNLIBS::sevn_lib_shraed` static libraries

Give a look to the compilation scripts for the examples in the SEVN folder
sevnlib_examples/binary_evol_cmake

Linking the SEVN library - CMake

The [library installation](#) produces a [CMake SEVN package](#).

A custom executable can be link to the SEVN libraries with few additions to the CMake file used to compile executable (*CMakeList.txt*)

1. Add the command `find_package(SEVN)`
2. To link the executable using the static lib just use `target_link_libraries(executable_name SEVNLIBS::sevn_lib_static)`
3. To link the executable using the shared lib just use `target_link_libraries(executable_name SEVNLIBS::sevn_lib_shared)`



If Cmake is not able to find the SEVN module:

- check if you properly [installed the SEVN libraries](#)
- If you installed the SEVN library in a custom directory (using the command `-DCMAKE_INSTALL_PREFIX`) add the path of the directory to the CMake search path with the cmake option `-DCMAKE__PREFIX_PATH`

Give a look to the compilation scripts for the examples in the SEVN folder
sevnlib_examples/binary_evol_cmake

Linking the SEVN library - The SEVNENV script

In order to simplify the compilation of codes using the SEVN library, *CMAKE* produces a bash script *sevnenv.sh*.

When *make install* is run, the script is made executable and moved inside the bin folder in the install directory (e.g. *usr/local/bin*).

The script is executed with *source sevnenv* (see the Warning below)

When the script is executed the script set the following env variable:

- *SEVNCXX*, compiler used to compile the SEVN library
- *SEVNFLAGS*, contains all the flags needed to properly link the SEVN library to the executable

The code can be compiled simply exploiting such variables:

```
$SEVNCXX main.cpp $SEVNFLAGS
```

Give a look to the compilation scripts for the examples in the SEVN folder
sevnlib_examples/binary_evol



If the bin folder in the install directory is in the PATH, it is possible to run *source sevnenv* from anywhere. If it is not in the PATH, you should go in the install directory then in the bin folder and then run *source sevnenv*. Remember, that the environment variables set through the script are local to the shell you are using. To make them always available copy the content of the script inside your *.bashrc* or *.bash_profile*